PROYECTO CHOLESKY HPC 2026

Factorización de Cholesky

memoria experimental ordenada

De la referencia secuencial y la validación a la comparación final entre MPI y LAPACK/OpenBLAS.

Edición cronológica de 20 informes · 90 páginas originales preservadas

Criterio de esta edición

Esta edición conserva las noventa páginas de los veinte PDF de origen. No elimina resultados, tablas, figuras, límites, referencias ni decisiones. El cambio es exclusivamente estructural: algunas páginas de diseño se trasladan a la pregunta experimental que preparan.

Movimientos editoriales principales

1. La referencia Python y su validación aparecen antes de las decisiones de rendimiento en C.
2. La razón para elegir right-looking y la discusión de repartos estáticos/dinámicos se ubican justo antes del primer experimento MPI.
3. El modelo de costos para decidir cuándo conviene MPI se ubica inmediatamente antes de la campaña que mide ese cruce.
4. Desde la generación distribuida en adelante se mantiene la secuencia experimental original.

Las numeraciones impresas en cada página pertenecen a los informes originales. Los marcadores del PDF y los separadores de partes muestran la nueva ubicación cronológica.

Mapa de lectura

PARTE I — **Fundamentos, validación y decisiones locales** — 4–15

Primero se fija qué se calcula y cómo se comprueba; después se comparan las variantes escalares.

PARTE II — **Diseño MPI y primera implementación distribuida** — 16–31

Las decisiones sobre propiedad de columnas aparecen inmediatamente antes de la campaña que las pone a prueba.

PARTE III — **Cruce de rendimiento y comparación de algoritmos MPI** — 32–47

El modelo de costos se presenta cuando ya existe un candidato MPI y justo antes de medir cuándo compensa.

PARTE IV — **Flujo de datos y cierre local** — 48–70

La investigación pasa del kernel al recorrido completo: generación, reunión, distribución y programa candidato.

PARTE V — **Campañas en el clúster y comparación final** — 71–98

La evidencia local se contrasta con el entorno real, el cruce formal, la escalabilidad y LAPACK.

I

PARTE I

Fundamentos, validación y decisiones locales

¿La implementación es correcta y qué organización local conviene antes de introducir MPI?

Separador editorial; las páginas siguientes conservan su contenido original.

Antes de acelerar, ¿qué debemos calcular?

Fundamentos, referencia secuencial y primeras mediciones

Proyecto universitario de computación de alto rendimiento · 21 de septiembre de 2026

El objetivo del proyecto es construir y evaluar una factorización de Cholesky paralela. Comenzamos por una versión secuencial cuyo resultado podamos verificar. Cada avance seguirá una pregunta, una hipótesis, una implementación y un experimento; los resultados decidirán la siguiente mejora.

1. La pregunta inicial

Dada una matriz real A, simétrica y definida positiva (SPD), buscamos una matriz triangular inferior L con diagonal positiva que cumpla:

$$A = LL^T$$

Simétrica significa que A coincide con su transpuesta. Definida positiva significa que su forma cuadrática es positiva para cualquier vector real no nulo:

$$A = A^T, \qquad x^T Ax > 0 \quad \text{para } x \neq 0$$

La factorización sirve, por ejemplo, para resolver un sistema Ax = b: primero se resuelve Ly = b y después L^Tx = y. En esta etapa implementamos solamente la factorización.

2. Qué esperamos observar

Hipótesis: si la matriz es SPD y respetamos las dependencias del algoritmo, el producto de L por su transpuesta reconstruirá A, salvo los errores de redondeo de la aritmética de punto flotante.

En aritmética exacta, las condiciones SPD garantizan los pivotes positivos y la existencia de esta factorización. En una computadora verificaremos los pivotes y los resultados no finitos: una entrada mal condicionada puede presentar dificultades numéricas.

3. Alcance de esta primera etapa

Usamos Python como referencia didáctica porque permite relacionar directamente las ecuaciones con el código. No representa una implementación HPC optimizada. La referencia de rendimiento para MPI se construirá en C; no calcularemos speedup comparando Python con C.

De las ecuaciones a una columna de L

Usamos índices desde 0, como en el programa. Al expandir el producto matricial, la triangularidad limita la suma:

$$a_{ij} = \sum_{k=0}^{\min(i,j)} l_{ik} l_{jk}$$

4. Primero la diagonal; después, debajo de ella

Para la columna j, los términos de las columnas anteriores ya son conocidos. Despejamos primero el elemento diagonal y luego cada elemento inferior:

$$l_{jj} = \sqrt{a_{jj} - \sum_{k=0}^{j-1} l_{jk}^2}$$

$$l_{ij} = \frac{a_{ij} - \sum_{k=0}^{j-1} l_{ik} l_{jk}}{l_{jj}}, \qquad i > j$$

Cuando j = 0, la suma vacía vale cero. Los elementos bajo la diagonal de una misma columna pueden calcularse de forma independiente una vez disponibles el pivote y las columnas anteriores. Esta observación será útil al estudiar paralelismo, aunque no determina por sí sola una estrategia eficiente.

5. Un ejemplo que podemos resolver a mano

$$A = \begin{bmatrix} 4 & 12 & -16 \\ 12 & 37 & -43 \\ -16 & -43 & 98 \end{bmatrix} \qquad L = \begin{bmatrix} 2 & 0 & 0 \\ 6 & 1 & 0 \\ -8 & 5 & 3 \end{bmatrix}$$

$$l_{00} = \sqrt{4} = 2, \qquad l_{10} = 12/2 = 6, \qquad l_{20} = -16/2 = -8$$

$$l_{11} = \sqrt{37 - 6^2} = 1, \qquad l_{21} = \frac{-43 - (-8)\cdot 6}{1} = 5$$

$$l_{22} = \sqrt{98 - (-8)^2 - 5^2} = 3$$

El producto LL^T recupera la matriz A del ejemplo. El trabajo aritmético dominante de Cholesky denso es aproximadamente $n^3/3$ operaciones de punto flotante; el almacenamiento denso crece como n^2. En Python también intervienen el intérprete y la representación de los objetos.

Cómo comprobamos que el resultado es correcto

6. Decisiones de implementación

La función **cholesky()** conserva A y construye una L nueva, con ceros por encima de la diagonal. Esto facilita la comparación y evita factorizar una entrada modificada en la siguiente repetición. La referencia exige una matriz cuadrada, no vacía, finita y exactamente simétrica.

En cada paso comprobamos que el pivote sea positivo y finito. También rechazamos resultados no finitos. Una ejecución que no supera estas comprobaciones no se acepta como medición válida.

7. Generación reproducible de matrices SPD

Generamos valores uniformes entre −1 y 1 fuera de la diagonal y los reflejamos para asegurar simetría. Luego fijamos:

$$a_{ii} = 1 + \sum_{j \neq i} |a_{ij}|$$

La diagonal es positiva y estrictamente dominante. Por simetría, los autovalores son reales; los intervalos de Gershgorin quedan en el semieje positivo, por lo que la matriz es SPD. La semilla permite reproducir la entrada. Esta familia inicial no cubre todos los problemas ni matrices arbitrariamente mal condicionadas.

8. Medida de reconstrucción

$$r = \frac{\|A - LL^T\|_F}{\|A\|_F}, \qquad \|M\|_F = \sqrt{\sum_i \sum_j m_{ij}^2}$$

Usamos el residuo relativo para comparar el error con la magnitud de A. Para estas matrices pequeñas adoptamos $r < 10^{-12}$ como criterio práctico. No es un umbral universal: un residuo pequeño tampoco asegura un error pequeño al resolver cualquier sistema mal condicionado.

9. Pruebas realizadas

Resultado: cinco pruebas automatizadas aprobadas. Comprueban el factor 3×3 conocido y que A se conserve; el caso escalar y la identidad; doce combinaciones generadas (tamaños 2, 7, 16 y 31 con semillas 0, 42 y 2026); entradas inválidas; y la detección de un factor deliberadamente incorrecto.

Entre las entradas rechazadas están las matrices vacías, no cuadradas, asimétricas, indefinidas, semidefinidas sin positividad estricta y aquellas con NaN o infinito. El ejemplo conocido produjo exactamente el factor esperado.

Qué medimos y qué podemos concluir

10. Protocolo de medición

Para cada tamaño n = 32, 64 y 128 usamos la semilla 2026, un calentamiento y siete repeticiones medidas sobre la misma entrada. El reloj monótono **perf_counter** mide la llamada completa a cholesky(): comprobaciones de entrada, reserva de L y factorización. Generación de A, validación y escritura de archivos quedan fuera del cronómetro.

Reportamos mediana, primer y tercer cuartil (Q1 y Q3), y rango intercuartílico IQR = Q3 − Q1. Se usa el método de cuartiles inclusivo de Python. El IQR describe la dispersión observada; no es un intervalo de confianza. La validación entre repeticiones puede modificar la caché.

11. Resultados locales observados

Windows 11 · Python 3.12.14 · 21 de septiembre de 2026. Tiempos en milisegundos; siete observaciones por fila.

n	Mediana	Q1	Q3	IQR	Residuo máx.
32	1.2313	1.2255	1.2627	0.0372	1.08e-16
64	7.3433	7.3293	7.4244	0.0951	9.68e-17
128	55.7097	55.3702	55.9313	0.5611	1.19e-16

Los residuos quedan muy por debajo del criterio establecido. Al duplicar n, las medianas aumentan aproximadamente 5,96 y 7,59 veces. Es compatible con la tendencia cúbica esperada, pero tres tamaños pequeños en Python no demuestran una ley de escalabilidad ni permiten predecir tiempos del clúster.

12. Conclusión y siguiente pregunta

La referencia reconstruye correctamente las matrices ensayadas y el protocolo conserva mediciones individuales y dispersión. Esto respalda los casos probados, no constituye una prueba exhaustiva de estabilidad. Todavía no hay una variante paralela comparable ni un speedup que reportar.

¿Qué pasaría si llevamos el mismo algoritmo a C y separamos los costos de reserva, generación, factorización y validación? Esa pregunta abre la etapa 2. Después estudiaremos cómo agrupar operaciones por bloques y cuáles pueden ejecutarse simultáneamente.

Trazabilidad. Código: desde_cero/cholesky.py; pruebas: test_cholesky.py; medición: benchmark.py. Datos: desde_cero/resultados/20260921T170357862059Z/ (mediciones.csv, resumen.csv y entorno.json). Esta edición utiliza los resultados existentes; no representa una nueva campaña de medición.

Referencia. LAPACK: documentación de DPOTRF. Describe la factorización SPD y una versión de producción por bloques con BLAS de nivel 3. Nuestra referencia es escalar.

Cholesky: el problema que vamos a resolver

Queremos factorizar una matriz real A, simétrica y definida positiva, como el producto de una matriz triangular inferior L y su transpuesta:

$$A = LL^T$$

Por encima de la diagonal de L todos los elementos son cero. Elegimos su diagonal positiva; con esa condición el factor es único. Esta factorización permite resolver Ax = b mediante los sistemas triangulares Ly = b y $L^T x = y$.

Condiciones de entrada

$$A = A^T, \qquad x^T A x > 0 \quad \text{para todo } x \neq 0$$

La primera condición es simetría; la segunda es definición positiva. En aritmética exacta garantizan que los pivotes de Cholesky sean positivos, sin necesidad de intercambiar filas. El código comprueba los pivotes para detectar entradas o resultados inválidos.

De dónde salen las dependencias

Al desarrollar el producto matricial, con índices desde cero, obtenemos:

$$a_{ij} = \sum_{k=0}^{\min(i,j)} l_{ik} l_{jk}$$

Una columna nueva utiliza contribuciones de las anteriores. Left-looking y right-looking cambian cuándo se aplican esas contribuciones, no el problema matemático. Ambas variantes escalares realizan aproximadamente $n^3/3$ operaciones de punto flotante.

Las tres decisiones del trabajo

Primera: qué variante conviene estudiar y qué muestran sus tiempos. **Segunda:** cuándo el ahorro de cálculo justifica usar más procesos. **Tercera:** cómo repartir los datos para mantener trabajo útil en esos procesos.

Todo el cálculo, la generación de matrices y la futura implementación MPI se realizan en C. Python procesa los CSV y genera estadísticas y gráficos. En este informe se distingue entre mediciones ejecutadas, conteos analíticos y experimentos MPI todavía pendientes.

[1] Ballard et al. (2009), Communication-optimal Parallel and Sequential Cholesky Decomposition.

Left-looking y right-looking: ¿dónde está el paralelismo?

Left-looking: actualizar la columna que necesitamos

Al llegar a la columna j, recuperamos los aportes de las columnas ya calculadas. Primero obtenemos el pivote y luego los elementos debajo de él:

$$l_{jj} = \sqrt{a_{jj} - \sum_{k=0}^{j-1} l_{jk}^2}$$

$$l_{ij} = \frac{a_{ij} - \sum_{k=0}^{j-1} l_{ik} l_{jk}}{l_{jj}}, \qquad i > j$$

También permite paralelismo: conocido el pivote y disponibles los datos anteriores, distintos elementos l_{ij} pueden calcularse simultáneamente. Incluso una suma interna puede repartirse si luego se combinan correctamente sus resultados. En memoria distribuida debemos decidir dónde están esos datos y cuánto cuesta reunirlos.

Right-looking: actualizar la región que todavía falta

Usamos una matriz de trabajo S, inicialmente igual a A. Al llegar a j, las contribuciones anteriores ya se aplicaron:

$$l_{jj} = \sqrt{s_{jj}}, \qquad l_{ij} = s_{ij}/l_{jj} \quad (i > j)$$

Luego la columna calculada actualiza la submatriz restante:

$$s_{ik} \leftarrow s_{ik} - l_{ij} l_{kj}, \qquad i \geq k > j$$

El paralelismo aparece en esas actualizaciones: cada pareja (i, k) modifica una posición diferente y lee una columna ya terminada. Las actualizaciones de la siguiente columna deben completarse antes de factorizarla.

Ambas variantes tienen dependencias entre columnas y admiten versiones por bloques. En right-looking nos interesa repartir las actualizaciones; no ejecutar arbitrariamente todas las columnas a la vez. [1]

Experimento 1: comparar las dos variantes en C

La pregunta y el experimento

¿Right-looking es más rápido que left-looking en nuestra implementación secuencial? Implementamos ambas variantes escalares en el mismo archivo C, con double, almacenamiento por filas y el mismo compilador. No usamos BLAS ni MPI en esta comparación.

Matrices iguales para las dos variantes

El generador C, con semilla 2026, crea valores simétricos fuera de la diagonal en [−1, 1). La diagonal se define como:

$$a_{ii} = 1 + \sum_{j \neq i} |a_{ij}|$$

Esto produce una matriz simétrica con diagonal positiva y estrictamente dominante; por tanto es definida positiva. Cada llamada recibe una copia nueva de la misma matriz. Se exige un residuo relativo de reconstrucción no mayor que 10^{-12}.

Condiciones de medición

WSL / Ubuntu 26.04, AMD Ryzen 5 2600, GCC 15.2.0 y opciones -O3 -std=c11 -Wall -Wextra -Wpedantic. Ambas variantes se fijan a la misma CPU lógica. Para cada tamaño hacemos un calentamiento por variante y siete repeticiones, alternando cuál se ejecuta primero.

El tiempo incluye solo la factorización. Generación, copia y residuo quedan fuera. Conservamos las observaciones individuales y calculamos medianas y cuartiles inclusivos. La copia y validación siguen el mismo protocolo para ambas variantes, pero no garantizan un estado idéntico de caché.

n	Left (ms)	Right (ms)	Right / Left
64	0.0238	0.0226	0.95
128	0.1999	0.2680	1.34
256	1.8608	5.9010	3.17
512	16.2481	130.1355	8.01
1024	137.6874	1109.6237	8.06

Medianas de siete repeticiones por variante y tamaño. Un cociente Right / Left mayor que 1 indica que right-looking tarda más. No es un speedup MPI. Datos nuevos: resultados/left_right_20260921/.

Qué dicen los datos y qué no demuestran

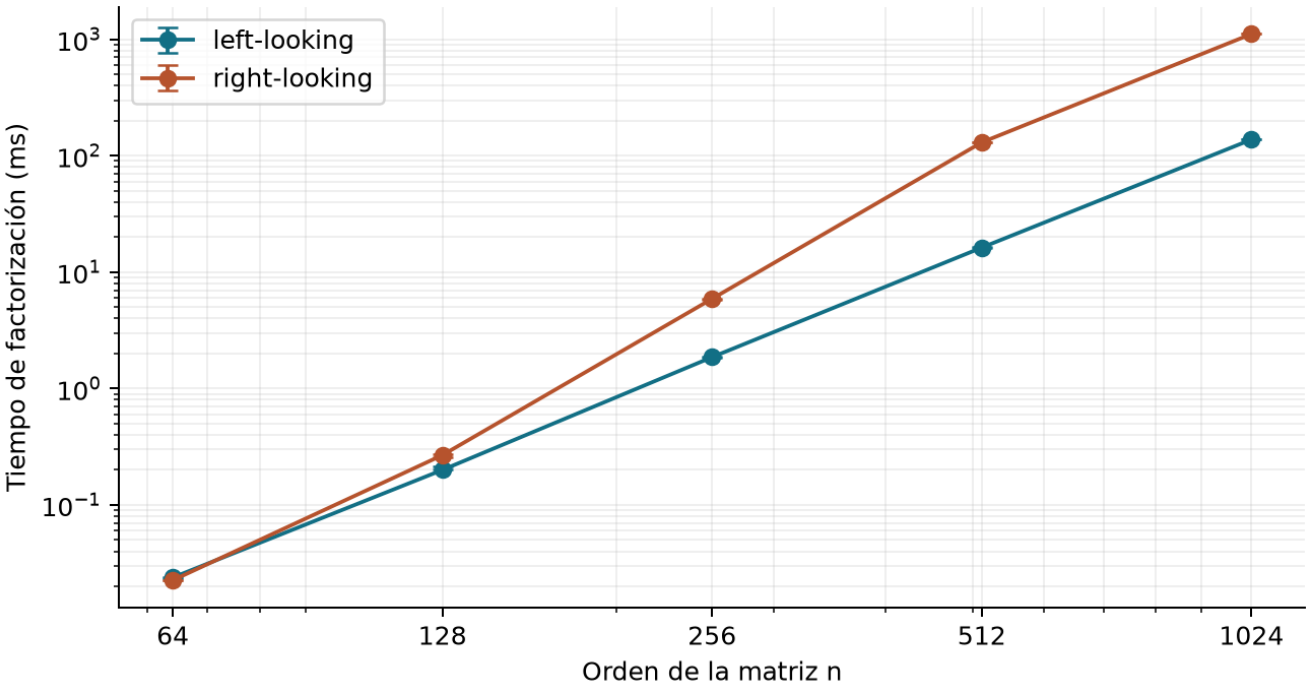


Figura 1. Comparación secuencial. Ambos ejes son logarítmicos. Los puntos indican la mediana y las barras abarcan Q1 a Q3; algunas barras son pequeñas a esta escala.

Resultado observado

En esta primera campaña, right-looking tiene una mediana ligeramente menor para n = 64. Left-looking es más rápido en los otros cuatro tamaños ensayados. Para n = 1024, right-looking tarda aproximadamente 8,06 veces más. Por lo tanto, **estos resultados no justifican elegir la versión original de right-looking por velocidad secuencial**.

Una explicación que debemos investigar

El código de left-looking calcula productos escalares recorriendo filas contiguas. El de right-looking relee elementos de una columna con saltos de n posiciones. La inspección del código muestra esa diferencia de acceso; el experimento no aísla cuánto explica del tiempo. Haría falta variar almacenamiento o implementar bloques para comprobarlo.

La conclusión corresponde a estas versiones escalares y a este equipo. No prueba que todas las implementaciones de left-looking superen a todas las de right-looking, ni que MPI vaya a invertir el resultado.

La pregunta siguiente

¿Qué pasaría si copiamos la columna activa a un vector contiguo antes de reutilizarla? Esa pregunta abre el experimento 2. Antes de agregar comunicación con MPI, comprobamos si un cambio pequeño en los accesos a datos mejora el cálculo local.

Experimento 2: cambiar el acceso, conservar el algoritmo

Hipótesis

El kernel right-looking original recorre la matriz por filas, pero lee la columna activa con saltos. Proponemos copiar esa columna a un vector contiguo y reutilizarlo durante la actualización. La hipótesis es que el ahorro en esas lecturas repetidas compense el costo de copiar.

Versión	Lectura dentro del bucle sobre k
Right original	a[k*n+j]: al aumentar k, la dirección avanza n valores.
Right + vector	column[k]: al aumentar k, la dirección avanza un valor.

$$c_k \leftarrow l_{kj}, \qquad s_{ik} \leftarrow s_{ik} - l_{ij} c_k$$

Mantenemos los bucles en el orden j, i, k, los mismos destinos y las mismas multiplicaciones y restas. El pivote y el cálculo de la columna no cambian. Esto sigue siendo Cholesky right-looking; no es aún una versión por bloques ni una implementación MPI.

Costo de la modificación

$$\text{memoria extra} = 8n \text{ bytes}, \qquad \text{copias} = \frac{n(n-1)}{2}$$

El vector reserva n valores double y se reutiliza. Para n = 1024 ocupa 8 KiB. El total de valores copiados crece como n^2, mientras que las actualizaciones mantienen su trabajo cúbico. Reservamos el vector antes del cronómetro, pero **todas las copias de columnas se incluyen en el tiempo medido**.

Comparación controlada

Volvemos a medir left, right original y right con vector en la misma campaña, sobre las mismas matrices y la misma CPU lógica. Hacemos un calentamiento por variante y doce repeticiones: las seis permutaciones del orden de ejecución se usan dos veces. Así cada variante aparece igual cantidad de veces en cada posición.

Conservamos el equipo, GCC 15.2.0 y las opciones -O3 del experimento anterior. Medimos n = 64, 128, 256, 512, 1000 y 1024; n = 1000 evita limitar la muestra a potencias de dos. Las reservas, la copia de entrada y la validación quedan fuera del tiempo en las tres variantes.

Resultados: ¿compensa copiar la columna?

n	Left (ms)	Right (ms)	Vector (ms)	Right / vector	Left / vector
64	0.0239	0.0230	0.0202	1.14	1.19
128	0.2004	0.2570	0.1725	1.49	1.16
256	1.8380	5.8159	1.3914	4.18	1.32
512	18.0215	185.6965	12.3247	15.07	1.46
1000	131.6738	95.7353	59.8545	1.60	2.20
1024	144.7909	1159.0888	76.9183	15.07	1.88

Medianas de doce repeticiones por variante y tamaño. Un cociente mayor que 1 favorece a right con vector. Cada comparación usa exclusivamente observaciones de esta segunda campaña; no mezclamos tiempos nuevos con los del experimento 1.

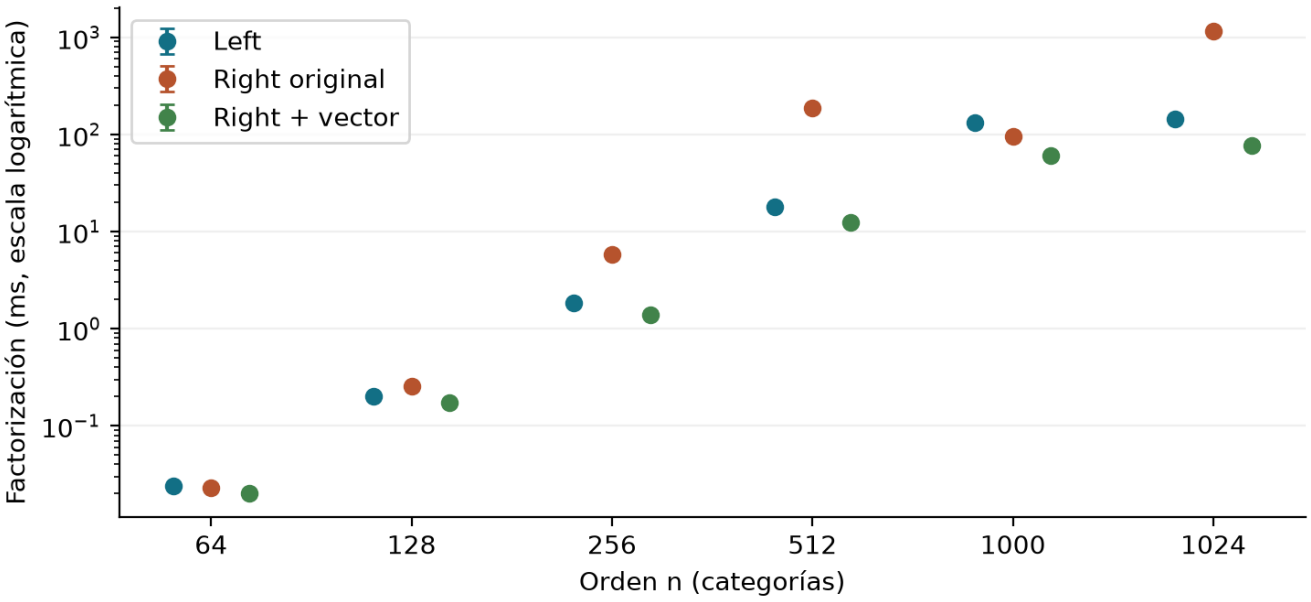


Figura 2. Medianas y barras Q1–Q3. El eje vertical es logarítmico; el horizontal muestra tamaños como categorías. Las barras representan dispersión, no intervalos de confianza.

El resultado que cambia nuestra decisión

La variante con vector obtiene menor mediana en los seis tamaños probados. Para n = 1024, pasa de 1159,09 ms a 76,92 ms frente a right original: una mejora de 15,07 veces. En la misma campaña supera a left, que tarda 144,79 ms, por un factor de 1,88.

La comparación conserva y contabiliza el costo del empaquetado. El trabajo adicional de copia se compensa en estas condiciones. Son mediciones secuenciales locales; todavía no representan un speedup MPI.

Interpretación y decisión después del experimento

¿Se conserva la corrección?

Sí en los casos ensayados. La variante con vector produce el factor esperado para el ejemplo conocido y supera quince matrices generadas de distintos tamaños y semillas. Se compara su factor con right original, se comprueban pivotes inválidos y se prueba que no lea zonas del vector todavía sin inicializar ni escriba fuera de él.

Las pruebas con AddressSanitizer y UndefinedBehaviorSanitizer pasaron. En la campaña se aceptaron 216 observaciones: seis tamaños, tres variantes y doce repeticiones. Todas cumplieron el criterio de residuo relativo no mayor que 10^{-12}; el máximo registrado fue aproximadamente $1{,}21 \times 10^{-15}$.

Qué respalda y qué no aísla esta evidencia

Mantener el algoritmo y cambiar el acceso a la columna produjo una reducción clara del tiempo. Esto respalda que la organización de esos accesos era una limitación importante de esta implementación. Sin contadores de hardware ni análisis adicional no atribuimos un porcentaje específico a fallos de caché: también pueden cambiar la vectorización y otras decisiones del compilador.

Hay una señal útil: right original tarda 95,74 ms para n = 1000 y 1159,09 ms para n = 1024, aunque los tamaños son cercanos. La nueva versión tarda 59,85 y 76,92 ms. La sensibilidad al tamaño se reduce, pero no afirmamos haber identificado el mecanismo exacto que causó el salto.

Qué conservamos

Adoptamos right-looking con vector como referencia local para el siguiente experimento MPI. Ahora la decisión combina una organización clara para repartir actualizaciones con una mejora secuencial medida. Mantenemos left y right original como referencias y preservamos los resultados anteriores: pertenecen a implementaciones distintas.

No concluimos que right-looking sea universalmente superior. La selección vale para estas versiones, matrices y máquina, y debe revisarse en el clúster. Para evaluar MPI volveremos a medir la mejor referencia secuencial en el mismo entorno.

La pregunta siguiente

¿La ganancia de repartir estas actualizaciones alcanza para pagar el envío de columnas? Esa será la prueba MPI. Un kernel local más rápido puede hacer que necesitemos matrices más grandes para que la comunicación se amortice; mejorar lo secuencial no garantiza más speedup paralelo.

Trazabilidad: src/compare_memory.c, scripts/comparar_memoria.sh y resultados/memoria_20260921/. Python produce resumen.csv y memoria.png a partir de los CSV individuales.

II

PARTE II

Diseño MPI y primera implementación distribuida

¿Cómo repartir columnas y actualizar datos locales sin perder corrección ni introducir desequilibrio innecesario?

Separador editorial; las páginas siguientes conservan su contenido original.

La razón para estudiar right-looking con MPI

Partimos de una propiedad fija de las columnas

Supongamos que cada columna tiene un proceso propietario. Queremos que los datos permanezcan en ese proceso y que el trabajo importante pueda repartirse sin migrar continuamente la matriz.

Con ese reparto	Left-looking	Right-looking
Columna j	Su dueño necesita aportes de columnas anteriores que pueden estar en otros procesos.	Su dueño encuentra la columna ya actualizada, calcula el pivote y los valores inferiores.
Trabajo repartido	Para repartir el cálculo de esa columna habría que distribuir sumas o filas y reunir resultados, o usar otra planificación.	La columna calculada se comunica; cada proceso actualiza las columnas futuras de las que ya es dueño.
Dependencias	Los aportes y el pivote deben estar disponibles.	La columna debe estar lista antes de actualizar; las actualizaciones necesarias preceden al siguiente pivote.

La tabla describe una primera organización sencilla con columnas de dueño fijo. No afirma que todas las implementaciones left-looking necesiten esa misma comunicación ni que no puedan ser competitivas.

Dónde encontramos trabajo independiente

Con right-looking, terminada la columna j, dos procesos pueden actualizar destinos diferentes usando los mismos valores recibidos. La región activa contiene muchas actualizaciones, mientras que el pivote sigue siendo parte de la cadena de dependencias. Esta secuencia tiene una traducción directa a MPI: propietario → difusión de columna → actualizaciones locales. [2]

Decisión defendible en el informe

Elegimos right-looking con vector para la primera implementación MPI por su correspondencia con un reparto estático de columnas, sus actualizaciones independientes y la mejora local medida en el experimento 2. Conservaremos left y el mejor secuencial disponible al evaluar la utilidad real de MPI.

La ventaja secuencial observada no demuestra escalabilidad distribuida. La adopción de MPI por rendimiento queda condicionada a medir su costo completo de factorización, incluidas comunicaciones y esperas.

[2] Netlib, Cholesky Factorization: panel y actualización distribuida right-looking.

Repartos estáticos: contiguo, cíclico y por bloques

En un reparto estático, el propietario de cada columna se fija antes de factorizar. **Round-robin por columnas es exactamente el reparto cíclico**: se asignan por turnos a los procesos 0, 1, …, P−1 y se repite.

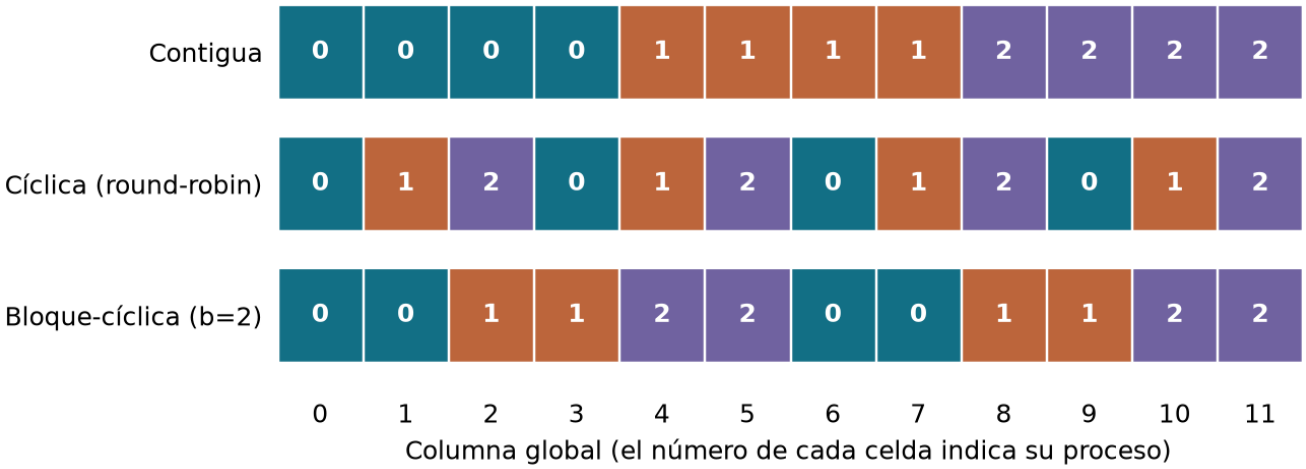


Figura 3. Ejemplo de asignación con n = 12 y P = 3. Los colores y números indican propietarios, no tiempos de ejecución.

Contiguo: un intervalo de columnas por proceso

P0 recibe 0–3, P1 recibe 4–7 y P2 recibe 8–11. La asignación es sencilla. El problema aparece cuando termina el primer intervalo: P0 deja de tener columnas futuras que actualizar mientras otros procesos siguen trabajando. Igual cantidad de columnas no implica igual cantidad de operaciones.

Cíclico: columnas alternadas

$$\text{owner}(j) = j \bmod P$$

P0 recibe 0, 3, 6 y 9; P1 recibe 1, 4, 7 y 10; P2 recibe 2, 5, 8 y 11. Al reducirse la región activa, sus columnas quedan repartidas entre varios procesos durante más etapas. No elimina el desequilibrio ni la escasez de trabajo al final. [3]

Bloque-cíclico: repartir grupos de columnas

$$\text{owner}(j) = \lfloor j/b \rfloor \bmod P$$

Agrupa b columnas consecutivas antes de pasar al siguiente proceso. Con b = 1 recuperamos el cíclico. Con bloques mayores se busca combinar reparto y operaciones locales más grandes; el tamaño b se elegirá midiendo. Cambiar el reparto por sí solo no convierte el kernel escalar en BLAS-3.

[3] ScaLAPACK Users' Guide, Two-dimensional Block-Cyclic Distribution.

Por qué el cíclico es nuestra primera hipótesis

Contemos el trabajo, sin inventar tiempos

En el kernel escalar right-looking, una columna global k recibe una actualización por cada columna anterior j. Cada una modifica sus n−k elementos inferiores. El número de actualizaciones de esa columna durante toda la factorización es:

$$U(k) = k(n-k), \qquad k = 0, \ldots, n-1$$

Sumando U(k) según los propietarios de la figura anterior obtenemos este conteo exacto para n = 12 y P = 3:

Reparto	P0	P1	P2	Máx. / media
Contigua	58	138	90	1.45
Cíclica (round-robin)	90	98	98	1.03
Bloque-cíclica (b=2)	82	106	98	1.11

Se cuentan restas de productos sobre elementos, no pivotes, divisiones, bytes ni segundos. Un valor Máx./media cercano a 1 indica mejor equilibrio de este trabajo acumulado. No representa una medición de rendimiento ni captura todas las esperas entre etapas.

Qué respalda este ejemplo

El cíclico distribuye mejor las actualizaciones que el contiguo en este caso. Para matrices densas y procesos similares, eso motiva comenzar con **propiedad estática cíclica**. Después compararemos bloque-cíclico cuando agrupemos operaciones. La decisión final dependerá del tiempo y de la comunicación medidos, no solo del conteo.

Distribución contigua no significa memoria contigua

El reparto indica qué índices globales pertenecen a un proceso. El almacenamiento local indica cómo los guarda en su RAM. Un proceso con columnas cíclicas puede empaquetarlas en un arreglo local contiguo; no necesita reservar los huecos de las columnas ajenas. [5]

Cuándo asignamos las columnas

En el esquema estático las asignamos al inicio. Durante cada etapa se comunica la columna lista y se trabaja sobre datos locales. No esperamos a un número de columna mágico para empezar a delegar. Cerca del final hay menos destinos independientes; reducir procesos o migrar datos solo convendría si su ahorro supera ese costo adicional.

[5] ScaLAPACK Users' Guide, Local Storage Scheme and Block-Cyclic Mapping.

Reparto dinámico y criterio para continuar

Qué decide una distribución dinámica

Un planificador asigna tareas durante la ejecución según qué procesos están libres. En Cholesky no puede entregar cualquier columna sin más: solo puede despachar tareas cuyas dependencias estén resueltas. Una tarea podría actualizar un bloque usando un panel ya calculado. [4]

Si el proceso elegido no tiene los datos, debe recibirlos o acceder a ellos. Además, dos tareas que modifican el mismo bloque deben respetar su dependencia o combinar sus resultados de forma segura. Por eso distinguimos la propiedad de datos de la planificación de tareas: es posible fijar una y hacer dinámica la otra.

Cuándo podría ayudar y qué cuesta

Aspecto	Estático cíclico	Dinámico
Asignación	Se conoce de antemano; no consulta una cola por tarea.	Se decide al quedar libre un trabajador y estar lista una tarea.
Desigualdad entre procesos	Un proceso lento puede retrasar etapas.	Puede asignar más tareas a trabajadores rápidos.
Datos y coordinación	Propiedad fija; comunicación del panel predecible.	Colas, mensajes, dependencias y posibles transferencias de datos.
Granularidad	Columnas o grupos con tamaño fijado.	Tareas pequeñas equilibran mejor, pero aumentan coordinación; tareas grandes reducen ese costo.

No podemos descartar lo dinámico como siempre más lento: un planificador por tareas puede ser eficaz. Para nuestro primer MPI de matrices densas en recursos similares, el cíclico estático ofrece un punto de partida más sencillo. Lo dinámico se justificaría si medimos esperas importantes por heterogeneidad o trabajo irregular que compensen su costo.

El siguiente experimento, en orden

1. Conservar right con vector como referencia local, junto con left y right original.
2. Implementar MPI con propiedad estática, comparando cíclico y contiguo con el mismo kernel.
3. Medir por separado P y n dentro de un nodo y entre nodos; registrar tiempo total de factorización, cómputo por proceso y espera en MPI.
4. Incorporar bloques o planificación dinámica solo si los resultados explican qué problema resolverían.

La medición paralela usará el máximo de los tiempos locales con inicio coordinado; excluirá validación y mantendrá el mismo criterio en todas las variantes. El tiempo dentro de MPI puede incluir espera, no solo transferencia. No se presentan tiempos de MPI ni de distribuciones como si ya se hubieran medido.

[4] Dorris et al., Task-Based Cholesky Decomposition: tareas listas y planificación dinámica.

¿Cuándo conviene repartir el trabajo?

En la etapa anterior mejoramos el acceso a memoria de right-looking. Ahora damos el primer paso hacia memoria distribuida: implementamos MPI en C y comparamos propiedad estática contigua y cíclica. Python analiza los tiempos y prepara gráficos; no realiza la factorización.

Hipótesis inicial

Con matrices pequeñas, comunicar cada columna puede costar más que el cálculo ahorrado. Al crecer n, esperamos que varios procesos resulten útiles. El reparto cíclico debería equilibrar mejor las actualizaciones, pero eso **no garantiza** que sea el más rápido en esta implementación.

Qué hace cada proceso

1. Asignar: repartimos las columnas antes de factorizar. Cada proceso guarda solo las suyas, empaquetadas por filas.
2. Calcular: el dueño de la columna j calcula el pivote y los elementos inferiores de esa columna.
3. Difundir: todos llaman a MPI_Bcast; el dueño envía el tramo de la columna desde j hasta n−1.
4. Actualizar: cada proceso modifica las posiciones de sus columnas k > j, usando el vector recibido. Luego avanzamos a j+1.

$$a_{ik} \leftarrow a_{ik} - l_{ij} l_{kj}, \qquad i \geq k > j$$

Los destinos pertenecen a procesos distintos: las actualizaciones de una etapa son independientes. Las columnas sucesivas siguen teniendo dependencias. Usamos comunicaciones bloqueantes y no incorporamos solapamiento ni tareas dinámicas.

La misma matriz, dos propietarios posibles

n=8, P=3	Columnas de P0	Columnas de P1	Columnas de P2
Contiguo	0, 1, 2	3, 4, 5	6, 7
Cíclico	0, 3, 6	1, 4, 7	2, 5

En contiguo, las primeras n mod P tareas reciben una columna adicional. En cíclico, owner(k)=k mod P. Incluso si P > n, los procesos sin columnas participan en las comunicaciones. El reparto no cambia durante la factorización.

MPI_Bcast distribuye un mensaje desde el dueño a todos los procesos del comunicador. Fuente: documentación oficial de Open MPI.

Cómo medimos y qué comparamos

Decisión	Protocolo ejecutado
Equipo	Un equipo, WSL2 / Ubuntu 26.04, AMD Ryzen 5 2600; Open MPI 5.0.10 y GCC 15.2.0.
Compilación	-O3 -std=c11 -Wall -Wextra -Wpedantic; double, sin BLAS.
Tamaños y procesos	n = 64, 256, 512, 1024; P = 1, 2, 3, 4. Un proceso por núcleo distinto, según la topología expuesta por WSL.
Repeticiones MPI	Un calentamiento por reparto y 8 repeticiones medidas. Contiguo y cíclico alternan el orden. El orden de P rota entre tamaños.
Referencias secuenciales	Left, right original y right con vector. Seis repeticiones antes y seis después de MPI para cada tamaño; misma semilla 2026.

El tiempo principal incluye cálculo y comunicación

$$T_P = \max_{r=0,\ldots,P-1} (t_{r,\text{fin}} - t_{r,\text{inicio}})$$

Una barrera coordina el inicio. Cada proceso mide su duración con MPI_Wtime; al finalizar tomamos el máximo. No restamos marcas de reloj de procesos diferentes. El cronómetro incluye el armado del vector, las difusiones y las actualizaciones; excluye generación, reserva, distribución inicial, reunión del factor y comprobaciones. Tampoco incluye iniciar mpirun.

Por tanto, medimos el kernel de factorización, no el tiempo de la aplicación completa. El proceso 0 genera y reúne matrices completas para este banco de pruebas; esa preparación también limita la escalabilidad futura.

Dos aceleraciones responden preguntas distintas

$$S_{\text{MPI}} = \frac{\widetilde{T}_{\text{MPI},1}}{\widetilde{T}_{\text{MPI},P}}, \qquad S_{\text{util}} = \frac{\min_v \widetilde{T}_{v,\text{seq}}}{\widetilde{T}_{\text{MPI},P}}$$

La primera compara un reparto consigo mismo al aumentar P. La segunda comprueba si supera al mejor secuencial de esta campaña; esa es la relevante para decidir si nos conviene usar MPI. Reportamos medianas y cuartiles inclusivos. Los cuartiles describen dispersión; no son intervalos de confianza.

MPI_Wtime mide tiempo local. Fuente: documentación oficial de Open MPI.

Resultado: más procesos no siempre ayudan

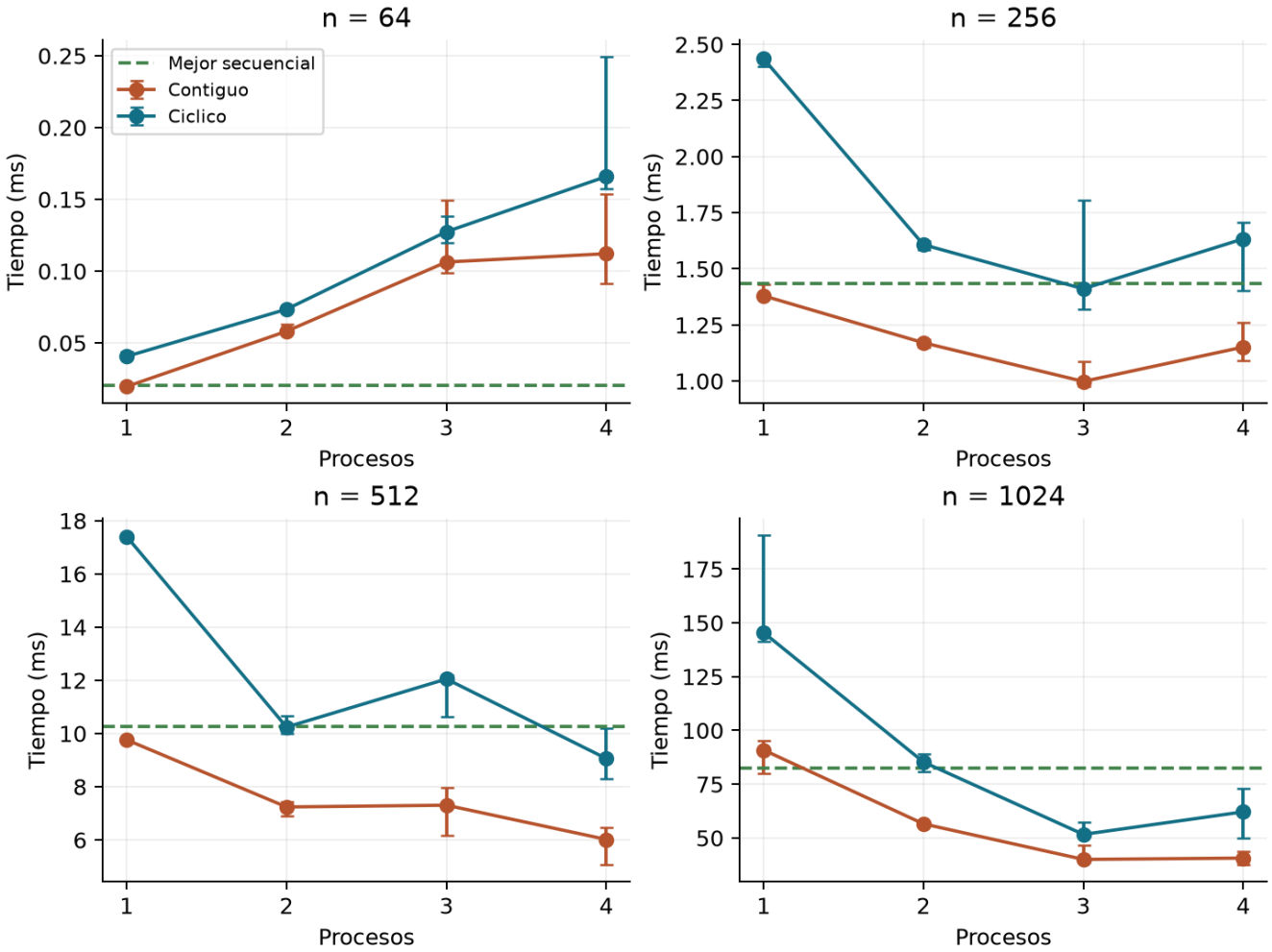


Figura 1. Medianas de 8 repeticiones por configuración; barras Q1-Q3. La línea verde muestra la mejor mediana secuencial contemporánea. Cada panel tiene su propia escala vertical.

Lo que vemos

n=64: usar más de un proceso aumenta el tiempo en ambos repartos. Hay demasiado poco trabajo para amortizar la coordinación.
n=256 y n=512: el contiguo ya obtiene mejoras útiles; el cíclico presenta resultados cercanos al secuencial o más lentos en varias configuraciones.
n=1024: ambos repartos mejoran al pasar de 1 a 3 procesos. Agregar un cuarto no reduce la mediana en esta campaña.

Contiguo y cíclico comparten el código de factorización y las operaciones matemáticas. Sin embargo, el reparto cambia los índices, los accesos al vector difundido y el código que puede optimizar el compilador. Esta prueba compara las implementaciones completas, no aísla exclusivamente el balance de carga.

¿Cuándo empieza a convenir MPI?

n	Mejor sec. (ms)	Mejor MPI P≥2	MPI (ms)	S útil
64	0,021	Contiguo, P=2	0,058	0,35
256	1,434	Contiguo, P=3	0,998	1,44
512	10,265	Contiguo, P=4	6,019	1,71
1024	82,654	Contiguo, P=3	40,028	2,06

Selección descriptiva de la menor mediana entre configuraciones probadas, no una garantía de que siempre sea la mejor. Right con vector fue la mejor referencia secuencial en los cuatro tamaños.

Respuesta provisional para este equipo

Con n=64 no conviene paralelizar este kernel. El primer tamaño probado con mejora clara del contiguo es n=256. Para n=1024 y P=3, el contiguo tarda 40,03 ms frente a 82,65 ms del secuencial: S útil = 2,06.

No encontramos un umbral exacto. Faltan tamaños intermedios y campañas independientes. Tampoco suponemos que la conveniencia sea monótona: caché, reparto, sistema operativo y red pueden cambiar el resultado. P=3 y P=4 contiguos tienen cuartiles que se superponen en n=1024; no declaramos una superioridad concluyente de tres procesos.

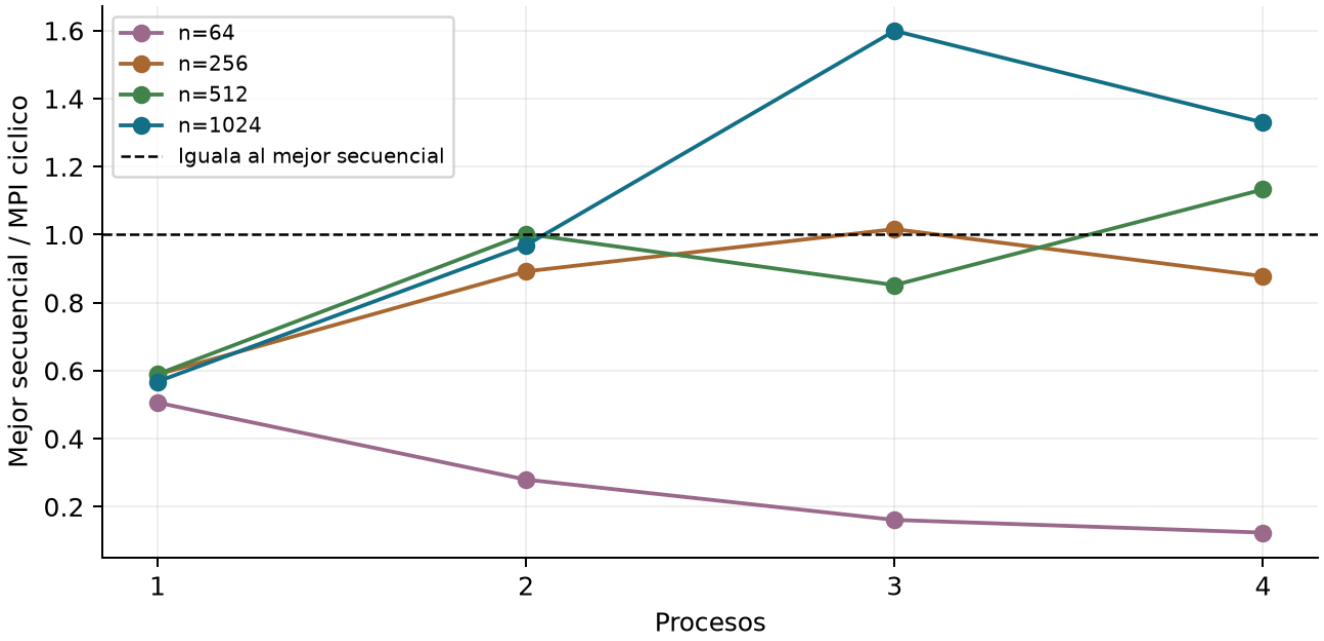


Figura 2. Aceleración útil del cíclico. Cerca de 1, una pequeña diferencia de medianas no basta para afirmar una mejora robusta. La evaluación entre nodos queda pendiente.

¿Por qué no ganó el cíclico?

La hipótesis de mejor reparto sí se cumple para el **número de actualizaciones**. Para n=1024 y P=3, los conteos exactos por proceso son:

Reparto	P0	P1	P2
Contiguo	46 434 993	86 086 814	46 434 993
Cíclico	59 652 153	59 652 153	59 652 494

Pero el tiempo depende también de cuánto cuesta cada actualización y cuándo queda disponible. En el contiguo, el vector difundido se lee en posiciones consecutivas. En el cíclico, sus índices avanzan de P en P, aunque los destinos locales estén empaquetados. El cálculo de índices también difiere.

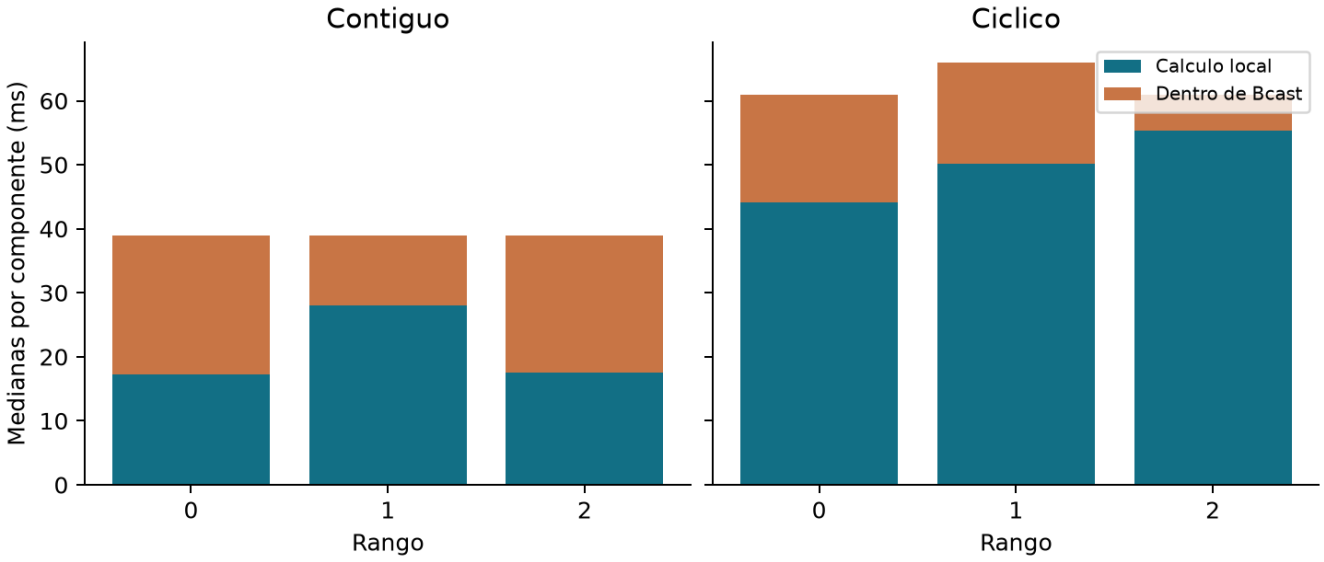


Figura 3. Perfil separado para n=1024, P=3, con 3 repeticiones e instrumentación interna. Se apilan medianas de componentes; su suma no es necesariamente la mediana del total. Bcast incluye transferencia y espera por otros procesos: no mide únicamente red.

Un control que impide una conclusión apresurada

Con P=1, ambos repartos poseen las mismas columnas. Aun así, el contiguo tarda 90,90 ms y el cíclico 145,49 ms. Por tanto, **el desequilibrio entre procesos no explica toda la diferencia**. Los caminos de indexación y optimización local ya tienen un costo distinto.

El informe de GCC registra versiones vectorizadas y no vectorizadas del bucle de actualización. Es una pista compatible con la diferencia de costo local, pero no cuantifica su causa. No medimos contadores de caché ni aislamos cada optimización del compilador.

Qué aprendimos y cómo seguimos

Decisión respaldada por esta campaña

Ya tenemos una implementación MPI correcta con dos repartos estáticos. En los tamaños probados, el contiguo obtuvo menores medianas que el cíclico para cada P. Lo conservamos como referencia práctica; mantenemos el cíclico como candidato por su mejor equilibrio de operaciones. No elegimos una distribución de forma definitiva para el clúster.

¿Y si ambos leyeran un vector local contiguo?

Esa es la próxima pregunta. Después de recibir la columna, cada proceso podría copiar los valores que necesita a un vector ordenado como sus columnas locales. El bucle interno leería posiciones consecutivas tanto para contiguo como para cíclico. La copia costaría trabajo adicional: debemos incluirlo en el tiempo y comprobar si se amortiza.

Este experimento permitiría reducir la diferencia de acceso e indexación antes de atribuir los resultados al reparto. Después repetiríamos el barrido de n y P en el clúster, distinguiendo un nodo de varios nodos. Todavía no incorporamos bloques, reparto dinámico ni comunicación solapada.

Comprobaciones realizadas

Ambos repartos pasaron casos SPD de tamaños 1, 2, 7, 16 y 31, un factor conocido 3×3 y rechazo colectivo de pivotes negativos y NaN, con P=1,2,3,4. Incluimos repartos desiguales y P > n. Las 256 mediciones MPI tienen residuo relativo ≤ 10^{-12} y coincidieron con el factor right secuencial dentro de 10^{-13}; el residuo máximo observado fue $1{,}2131\times10^{-15}$.

UndefinedBehaviorSanitizer pasó con P=4. AddressSanitizer también pasó con P=4 al desactivar memory/patcher de Open MPI solo en la prueba. Con ese componente activo fallaba dentro de MPI_Init, incluso en un programa mínimo sin Cholesky. No se comprobó detección de fugas; las mediciones usaron compilación normal y la configuración MPI predeterminada.

Archivos para reproducir y revisar

Archivo en desde_cero	Contenido
src/cholesky_mpi.c	Propietarios, índices y factorización distribuida.
src/compare_mpi.c	Banco de pruebas, validación, tiempo y perfil.
scripts/comparar_mpi.sh	Campaña con 1, 2, 3 y 4 procesos.
scripts/analizar_mpi.py	Estadísticas, aceleración, cuartiles y gráficos.
resultados/mpi_20260921/	CSV, entorno, afinidad, SHA-256 y resúmenes.

Los 144 tiempos secuenciales, 256 tiempos MPI y 60 registros de perfil por rango se conservan por separado. Las etapas anteriores permanecen en el PDF de fundamentos; este capítulo continúa su análisis.

¿Y si ambos repartos leyeran datos contiguos?

En el experimento anterior, el cíclico equilibraba mejor el número de operaciones, pero tardaba más. Había una pista: incluso con un solo proceso, cuando ambos repartos poseen las mismas columnas, sus tiempos eran distintos. Antes de descartar el reparto cíclico, decidimos reducir el costo de acceso e indexación del cálculo local.

Hipótesis

Después de recibir la columna j, cada proceso copia los valores que necesita a un vector ordenado como sus columnas propias. Esperamos que reutilizar ese vector compense el costo de copiarlo, especialmente en el cíclico. La copia se incluye en el tiempo: no damos por hecho que sea beneficiosa.

El cambio en C

Sea g(c) la columna global correspondiente a la columna local c. En la versión original, el bucle interno consulta el vector difundido mediante ese índice global. En la nueva versión lo prepara una sola vez por etapa:

$$b_c = l_{g(c),\,j}, \qquad g(c) > j$$

$$a_{ic}^{\text{local}} \leftarrow a_{ic}^{\text{local}} - l_{ij} b_c$$

Momento	Versión original	Con vector local
Antes de actualizar	Se usa la columna difundida.	local_column[c] = column[g(c)]
Dentro del bucle	lij * column[g(c)]	lij * local_column[c]

Por ejemplo, con n=8 y P=3, el rango 1 cíclico posee las columnas globales 1, 4 y 7. En la etapa j=0 copia esos tres valores a posiciones locales consecutivas. Las filas que actualiza luego reutilizan esa copia.

Cuánto cuesta y qué se conserva

Cada proceso reserva tantos doubles adicionales como columnas propias: ocho bytes por columna local en este equipo. La memoria adicional máxima por proceso es:

$$M_{\text{extra}} \leq 8\left\lceil \frac{n}{P} \right\rceil \text{ bytes}$$

Durante toda la factorización se copian n(n−1)/2 valores entre todos los procesos. Es trabajo cuadrático adicional frente al cálculo cúbico de Cholesky; eso por sí solo no garantiza una mejora para tamaños pequeños.

No cambian los propietarios, las dependencias, el orden de las operaciones aritméticas ni las llamadas a MPI_Bcast. El bucle interno accede ahora a datos consecutivos en ambos repartos. La asignación de índices fuera de ese bucle todavía difiere.

Un experimento con cuatro variantes

Reparto	Lectura original	Vector local
Contiguo	Referencia anterior	Nueva variante
Cíclico	Referencia anterior	Nueva variante

Cómo evitamos mezclar campañas

Volvimos a medir las cuatro variantes en el mismo ejecutable y sobre las mismas matrices. Los cocientes de esta etapa usan únicamente tiempos nuevos; no dividimos tiempos actuales por los del PDF anterior. La generación y todas las factorizaciones están en C. Python valida los CSV y obtiene estadísticas y gráficos.

Configuración	Protocolo ejecutado
Entorno	WSL2 / Ubuntu 26.04, AMD Ryzen 5 2600, Open MPI 5.0.10, GCC 15.2.0; -O3, double, sin BLAS.
Tamaños y procesos	n=64,256,512,1024 y P=1,2,3,4. Un proceso por núcleo distinto según la topología expuesta por WSL.
Orden y repeticiones	Un calentamiento por variante; ocho repeticiones medidas. Cada variante ocupa cada posición dos veces. Se equilibran también las parejas consecutivas dentro de cada repetición.
Tiempo MPI	Máximo de las duraciones locales, con inicio coordinado por barrera. Incluye copia del vector, cálculo y difusión.
Fuera del cronómetro	Reserva de memoria, generación, distribución inicial, reunión, validación y arranque de MPI.
Referencia secuencial	Left, right original y right con vector: seis ejecuciones antes y seis después de MPI para cada n.

Qué reportamos

$$G_{\text{copia}} = \frac{\widetilde{T}_{\text{original}}}{\widetilde{T}_{\text{local}}}, \qquad S_{\text{util}} = \frac{\widetilde{T}_{\text{mejor secuencial}}}{\widetilde{T}_{\text{MPI}}}$$

Un cociente mayor que 1 indica una mejora. Usamos medianas y cuartiles inclusivos Q1-Q3. Las barras de dispersión no son intervalos de confianza; ocho repeticiones en una campaña no sustituyen repeticiones independientes en otros momentos.

Se conservaron 512 tiempos MPI, 144 tiempos secuenciales y un perfil separado de 48 registros por rango (n=1024, P=3, cuatro variantes y cuatro repeticiones). El perfil incorpora cronómetros internos y no se usa para calcular la aceleración principal.

Resultado: la copia ayuda sobre todo al cíclico

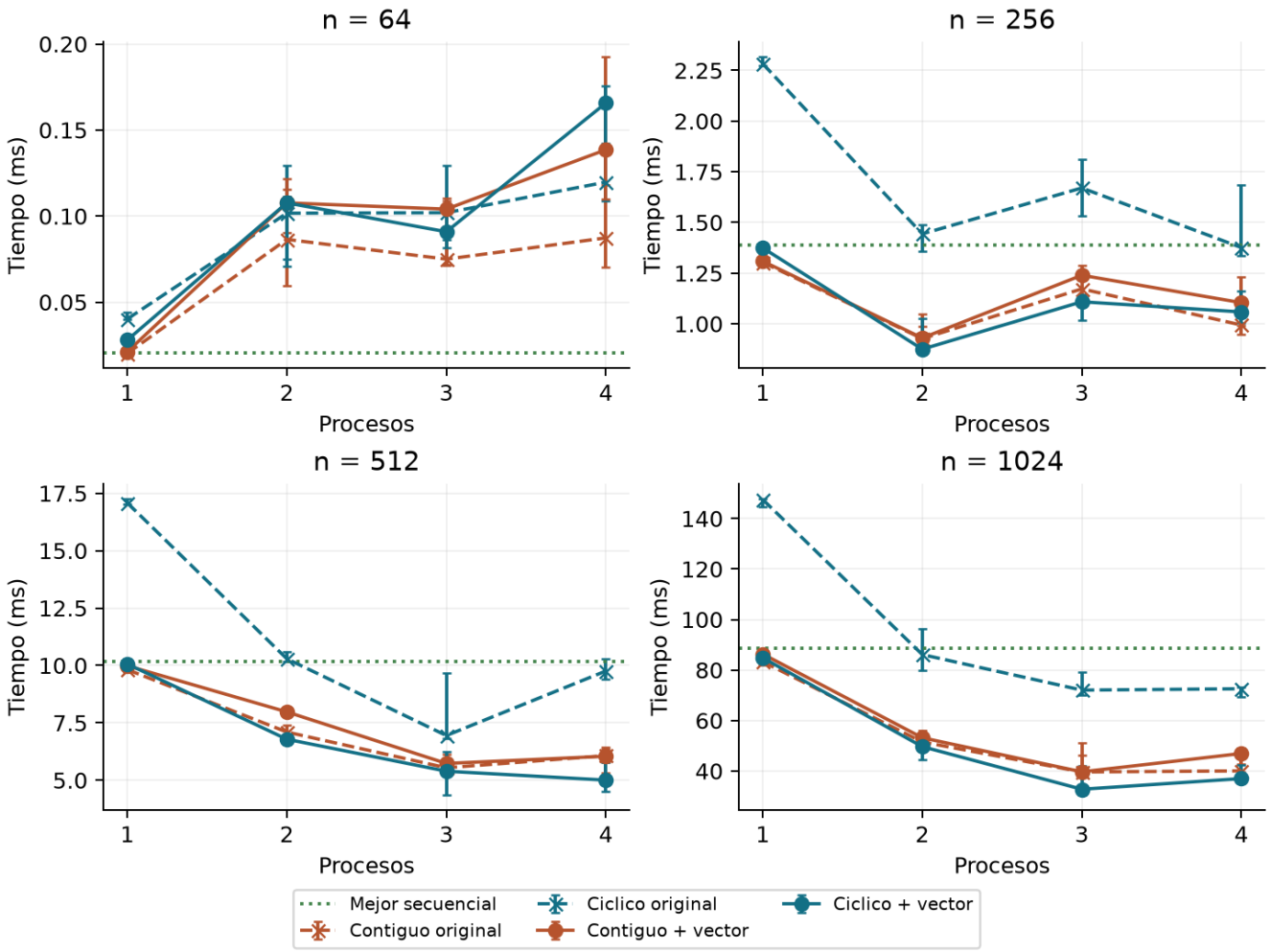


Figura 1. Medianas de ocho repeticiones; barras Q1-Q3. Líneas discontinuas: acceso original. Líneas continuas: vector local. La referencia secuencial es la mejor mediana de las tres variantes secuenciales de esta campaña. Cada panel tiene su propia escala.

La hipótesis se cumple con límites

n=64: todos los casos con P>1 son más lentos que el mejor secuencial; agregar una copia no resuelve la falta de trabajo útil.
n≥256: el cíclico con vector tiene menor mediana que el cíclico original en todos los P probados.
Contiguo: copiar no aporta una mejora sistemática; su lectura original ya era consecutiva y en varios casos la copia agrega costo.

Entre las nuevas variantes, el cíclico presenta menor mediana que el contiguo para P=2,3,4 y n≥256. Sin embargo, varias distribuciones de tiempos se superponen: esto respalda una elección provisional, no una superioridad universal ni una prueba de significación estadística.

El caso n=1024, explicado con números

P	Contiguo original	Contiguo + vector	Cíclico original	Cíclico + vector
1	83,28	86,33	147,55	84,81
2	51,49	53,17	86,11	49,64
3	39,62	39,66	72,04	32,77
4	40,03	46,93	72,57	36,99

Todos los tiempos están en milisegundos. Mediana del mejor secuencial: 88,62 ms, correspondiente a right con vector.

¿Se amortizó la copia?

Con P=3, el cíclico pasa de 72,04 ms a 32,77 ms: mejora 2,20 veces respecto de su versión original, incluyendo el tiempo de copiar. Frente al mejor secuencial, la aceleración útil es 2,70.

El control con un solo proceso

Antes de copiar, P=1 da 83,28 ms en contiguo y 147,55 ms en cíclico. Con el vector local, quedan en 86,33 y 84,81 ms, con cuartiles superpuestos. La gran diferencia previa se redujo: el reparto entre procesos no podía explicar esa diferencia inicial.

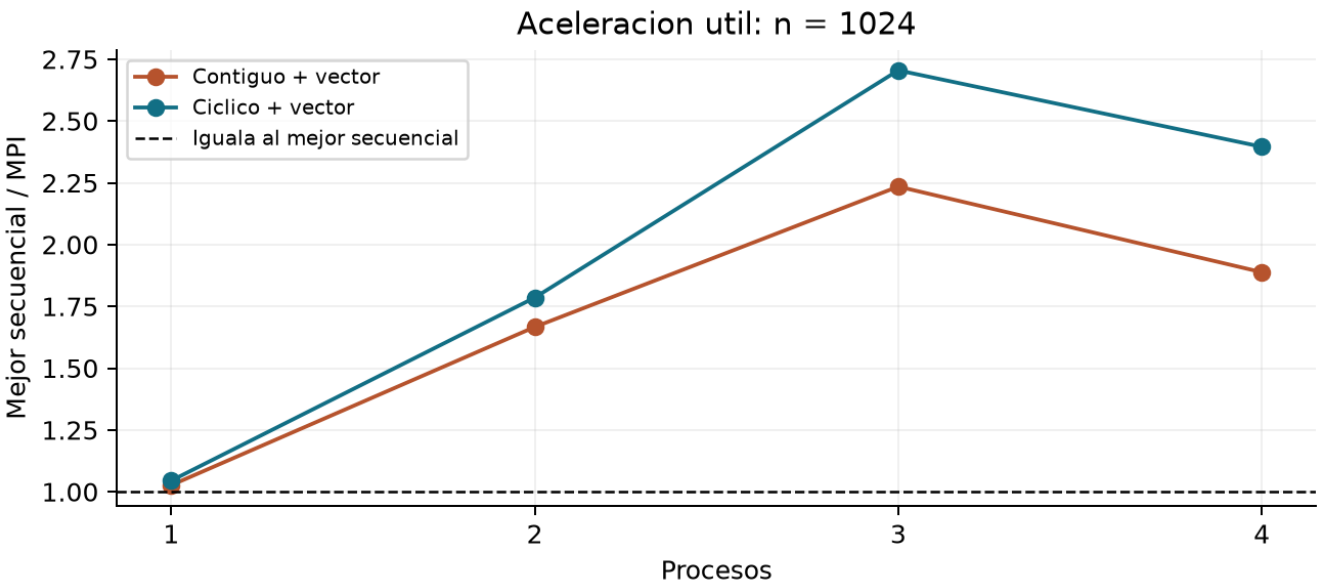


Figura 2. Aceleración útil de las nuevas variantes. Tres procesos cíclicos dieron la menor mediana para n=1024; sus cuartiles se superponen con los de cuatro procesos y con el contiguo de tres. Debemos repetir antes de afirmar cuál es la mejor configuración de forma estable.

Qué decidimos y cuál es la próxima pregunta

Qué podemos concluir

Conservar **right-looking MPI cíclico con vector local** como candidato principal está ahora respaldado por una mejora medida, no solo por el conteo de operaciones. También conservamos el contiguo original como referencia: la copia no lo mejora sistemáticamente. Para matrices pequeñas, mantenemos el secuencial como opción práctica.

El informe del compilador registra vectorización del nuevo bucle interno. El perfil separado muestra menor tiempo de cálculo local en el cíclico con vector para los tres rangos de n=1024, P=3. Es evidencia compatible con la mejora del acceso y la indexación; no cuantifica cuánto proviene de caché, instrucciones vectoriales u otros efectos.

El tiempo dentro de MPI_Bcast incluye esperas además de transferencia. El perfil tiene instrumentación y se ejecuta en otra pasada: sus tiempos no deben sustituir las medianas principales. No medimos comunicación entre nodos ni el tiempo completo de la aplicación.

Validación y trazabilidad

Las cuatro variantes pasaron los casos SPD, el factor conocido, repartos desiguales, P>n y fallos colectivos con P=1,2,3,4. También verificamos los límites del vector local y ejecutamos AddressSanitizer/UndefinedBehaviorSanitizer con P=4. Se mantuvo la adaptación de Open MPI descrita en la etapa 2 solo para los sanitizadores, con detección de fugas desactivada.

Las 512 observaciones MPI fueron aceptadas. El residuo máximo fue $1{,}2131\times10^{-15}$, inferior al límite 10^{-12}. La máxima diferencia registrada del factor frente a right secuencial fue cero en esta campaña. Se conservaron entradas del protocolo, afinidad, CSV y huellas de las fuentes.

¿En qué tamaño empieza a compensar esta versión?

Para responder mejor, el próximo paso razonable es probar tamaños intermedios entre 64 y 256, repetir campañas independientes y comparar pocos valores de P. Así evitamos elegir un umbral por una sola mediana. Después repetiremos el protocolo en el clúster, separando un nodo de varios nodos; los bloques y la planificación dinámica quedan para cuando los datos justifiquen su complejidad.

Archivo en desde_cero	Función
src/cholesky_mpi.c	Conserva el kernel original y añade cholesky_mpi_local.
src/compare_mpi_local.c	Compara las cuatro variantes y valida sus factores.
scripts/comparar_mpi_local.sh	Ejecuta la campaña reproducible.
scripts/analizar_mpi_local.py	Valida observaciones y genera estadísticas y gráficos.
resultados/mpi_local_20260921/	Datos, perfil, fuentes y comprobaciones.

III

PARTE III

Cruce de rendimiento y comparación de algoritmos MPI

¿En qué tamaños MPI empieza a ayudar y qué variante paralela conviene conservar?

Separador editorial; las páginas siguientes conservan su contenido original.

¿Cuándo conviene usar más procesos?

Repartir trabajo ahorra cálculo por proceso, pero agrega comunicación, coordinación y posibles esperas. Un modelo orientativo, sin solapamiento, es:

$$T_P \approx T_{\text{dep}} + \frac{T_{\text{upd}}}{P} + T_{\text{comm}} + T_{\text{espera}}$$

Tdep representa trabajo dependiente no repartido; Tupd, las actualizaciones repartibles. En este modelo, usar P procesos compensa frente al mismo algoritmo secuencial cuando:

$$T_{\text{upd}}\left(1 - \frac{1}{P}\right) > T_{\text{comm}} + T_{\text{espera}}$$

Para un mensaje de m bytes, el modelo α + βm distingue el costo de iniciar el envío y el costo por volumen. Un broadcast depende además de P y de la implementación MPI; no lo deduciremos únicamente de un ping-pong. [1]

La comparación que realmente decide

$$T_{\text{base}}(n) = \min(T_{L,1}(n), T_{R,1}(n), T_{R_v,1}(n))$$

$$S_{\text{util}}(n, P) = \frac{T_{\text{base}}(n)}{T_{R_v,P}(n)}$$

Rv indica right con vector. Conviene MPI si reduce de forma repetible el tiempo frente a esa base. También compararemos el mismo kernel MPI con P = 1 y P mayor para aislar su escalabilidad. Si importa el tiempo completo de una tarea, debemos sumar arranque y distribución inicial.

Para n = 1024, el experimento 2 deja una referencia local de 76,92 ms. Una versión MPI tendría que mejorar ese tiempo total de factorización en condiciones comparables. No usaremos los 1159,09 ms de right original para presentar un speedup artificialmente favorable.

Cómo obtendremos el umbral

Para cada n probaremos P = 1, 2, 3, 4 y otros valores permitidos, con varias ejecuciones. Elegiremos el menor tiempo estable; si la diferencia es pequeña frente a la dispersión, preferiremos menos recursos. El primer n favorable será un cruce observado para esa configuración, no una constante universal.

Pendiente: aún no ejecutamos MPI en esta etapa ni tenemos resultados entre nodos. WSL no mide la red del clúster. Separaremos ensayos dentro de un nodo y entre nodos, manteniendo explícitos procesos, núcleos y afinidad.

¿Cuándo empieza a compensar MPI?

Después de mejorar el cíclico con un vector local, observamos que MPI podía ganar con matrices grandes y perder con pequeñas. Faltaba precisar la transición. Este experimento conserva el código C del anterior y cambia únicamente el plan de medición.

La pregunta concreta

Entre n=64 y n=256, ¿qué tamaños permiten al cíclico con vector superar al mejor secuencial de manera consistente con dos o tres procesos? Incluimos P=1 como control y conservamos las otras variantes MPI en los datos, sin usarlas para seleccionar el resultado que más nos favorezca.

Protocolo fijado antes de medir

Decisión	Ejecución
Tamaños	64, 96, 128, 160, 192, 224 y 256.
Rondas	Tres rondas con lanzamientos separados en la misma sesión. Orden de tamaños ascendente, descendente y rotado; orden de P también rotado.
Repeticiones	Ocho tiempos por variante MPI y configuración, tras calentamiento. Doce tiempos por variante secuencial: seis antes y seis después.
Entradas	Mismo generador SPD en C, misma semilla 2026 y mismo n para todas las variantes.
Entorno	WSL2, AMD Ryzen 5 2600, GCC 15.2.0 y Open MPI 5.0.10. -O3; un proceso por núcleo distinto según WSL.

Criterio conservador de mejora

$$S_r = \frac{\min_v \widetilde{T}_{v,\,\mathrm{seq},\,r}}{\widetilde{T}_{\mathrm{MPI},\,r}}$$

$$S_r > 1.10 \quad \text{y} \quad Q_{3,\,\mathrm{MPI},\,r} < Q_{1,\,\mathrm{seq},\,r}, \qquad r = 1, 2, 3$$

Exigimos aceleración superior a 1,10 y separación entre los cuartiles centrales en las tres rondas. Los cuartiles secuenciales corresponden a la variante con menor mediana en esa ronda. El margen del 10 % es una decisión práctica explícita, no una constante de Cholesky.

Esta regla es descriptiva: Q1-Q3 no es un intervalo de confianza y su separación no demuestra significación estadística. Las tres rondas comparten equipo y sesión; no representan días, máquinas o condiciones independientes.

La transición observada entre 64 y 256

n	S con P=2	Criterio	S con P=3	Criterio
64	0,27 - 0,51	0/3	0,15 - 0,22	0/3
96	0,53 - 0,69	0/3	0,38 - 0,39	0/3
128	0,72 - 0,88	0/3	0,66 - 0,77	0/3
160	0,72 - 0,96	0/3	0,57 - 0,87	0/3
192	0,87 - 0,91	0/3	0,70 - 0,93	0/3
224	0,73 - 0,99	0/3	0,83 - 1,06	0/3
256	1,13 - 1,43	2/3	1,12 - 1,49	3/3

Cada intervalo muestra el mínimo y máximo de las tres aceleraciones calculadas por ronda. No es un intervalo de confianza. “Criterio” cuenta las rondas que cumplen simultáneamente ambas condiciones.

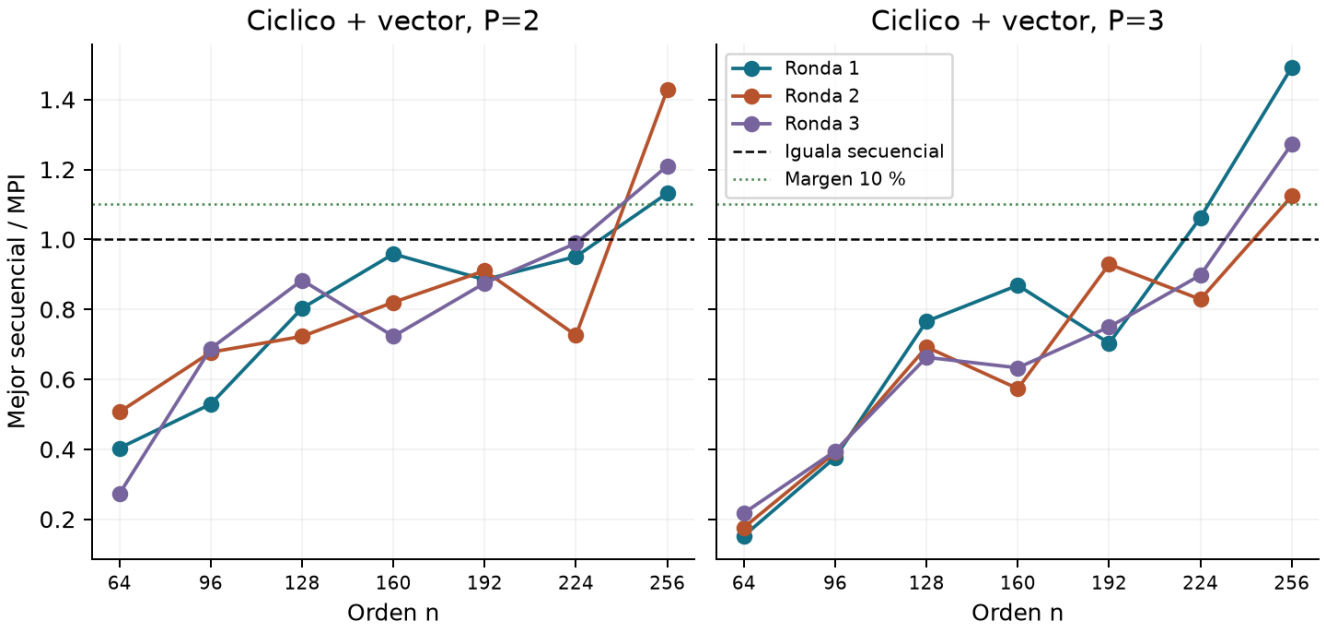


Figura 1. Aceleración del cíclico con vector frente al mejor secuencial contemporáneo. Por encima de 1 gana MPI; por encima de 1,10 supera el margen elegido, aunque todavía debe cumplir la condición de cuartiles.

Qué muestran las repeticiones

n≤192: MPI con dos y tres procesos tuvo mayor mediana que el secuencial en las tres rondas.
n=224: tres procesos ganaron apenas en una ronda y perdieron en las otras dos.
n=256: ambos valores de P ganaron por mediana en las tres rondas; solo P=3 cumplió también el criterio completo en las tres.

Por tanto, n=256 con P=3 es el **primer punto de la grilla probada** que satisface nuestra regla. No medimos un umbral exacto y no demostramos que todo n mayor que 256 tenga el mismo comportamiento.

Por qué dos y tres procesos no reciben la misma conclusión

Ronda	Secuencial (ms)	MPI P=2 (ms)	MPI P=3 (ms)
1	1,431	1,264	0,959
2	1,388	0,972	1,235
3	1,398	1,156	1,099

Medianas para n=256. Right con vector fue la referencia secuencial elegida en estas tres comparaciones. No se mezclan rondas para calcular una aceleración única.

Una mediana favorable puede convivir con dispersión

En la primera ronda, dos procesos tienen aceleración 1,13, superior a 1,10. Sin embargo, su Q3 es 1,452 ms y el Q1 secuencial es 1,380 ms. Los intervalos centrales se superponen y la configuración no pasa nuestra segunda condición.

Con tres procesos, las tres rondas cumplen ambas condiciones. Sus aceleraciones son 1,49, 1,12, 1,27. Eso respalda una mejora consistente dentro de esta sesión, pero no demuestra que tres procesos siempre sean mejores que dos: en la segunda ronda, P=2 tuvo menor mediana que P=3.

Qué podemos decir sobre el cruce

Hay un cambio de comportamiento entre los puntos probados n=224 y n=256. Es una zona candidata para investigar, no un intervalo matemáticamente garantizado del umbral. Tamaños, caché, compilador y carga del sistema pueden generar un comportamiento no monótono.

Tampoco convertimos el dato en una regla automática “si n≥256, usar tres procesos”. Eso requeriría probar otros tamaños, repetir en otras sesiones y definir qué costo de ejecución queremos minimizar.

Seguimos midiendo solo la factorización

El tiempo incluye cálculo, copia del vector local, comunicación de columnas y esperas dentro de MPI. Excluye iniciar MPI, generar la matriz, distribuirla al principio, reunir el factor y validarlo. El cruce de una aplicación completa puede ser distinto al del kernel medido aquí.

Resultado de esta etapa y siguiente problema

Decisión provisional

Para los tamaños ensayados hasta n=224, no tenemos evidencia consistente para elegir el cíclico MPI sobre el secuencial en este equipo. En n=256, tres procesos sí superan la regla prefijada. Conservamos ambos caminos; aún no introducimos selección automática ni trasladamos esta decisión al clúster.

Validación de los datos

Se verificaron 2016 tiempos MPI y 756 tiempos secuenciales. Todas las observaciones cumplieron el límite de residuo relativo 10^{-12}. El máximo observado fue $5{,}7136\times10^{-16}$; la diferencia máxima del factor MPI frente a right secuencial fue cero en estos casos.

El análisis comprueba configuraciones, semilla, cantidad de repeticiones, orden equilibrado, tiempos finitos y resultados numéricos. Las huellas SHA-256 coinciden con las fuentes usadas. No se modificó el algoritmo: se reutilizaron los ejecutables y pruebas de la etapa anterior, y se validó cada nueva factorización fuera del cronómetro.

¿Y si contamos también repartir y reunir la matriz?

Ese es el próximo experimento útil: medir por separado la distribución inicial, la factorización y la reunión, además del total de esas fases. Con esa información podremos decidir si la aceleración del kernel se conserva cuando el programa debe entregar un resultado completo.

El lanzamiento de procesos puede medirse aparte para aplicaciones que los crean por cada problema. En un programa que mantiene procesos MPI activos, ese costo no se paga en cada factorización. Debemos explicitar el caso de uso antes de elegir qué tiempo define la conveniencia.

Después podremos repetir las mediciones en el clúster, primero dentro de un nodo y luego entre nodos. La comunicación local de WSL no estima el costo de su red. No hace falta añadir todavía bloques o planificación dinámica para responder estas preguntas.

Cómo reproducirlo

Archivo en desde_cero	Uso
scripts/medir_umbral.sh	Ejecuta las tres rondas y registra el protocolo previo.
scripts/analizar_umbral.py	Valida datos y evalúa el mismo criterio en cada punto.
docs/generar_informe_umbral.py	Genera este capítulo desde la campaña registrada.
resultados/umbral_20260922/	CSV por ronda, resúmenes, gráfico, protocolo y fuentes.

La fecha del experimento es 22/09/2026. Las etapas previas conservan sus resultados históricos; las conclusiones de esta etapa se basan solo en las nuevas rondas.

¿Cuál variante nos conviene paralelizar?

Hasta ahora habíamos elegido right-looking por su organización natural de las actualizaciones, pero eso era una razón de diseño. Para responder con evidencia implementamos también left-looking con MPI y comparamos ambas variantes bajo las mismas condiciones.

La dependencia que comparten

En ambas variantes, la columna j solo puede finalizar cuando están disponibles las contribuciones que la afectan. Right-looking no elimina esa dependencia ni permite calcular arbitrariamente todas las columnas a la vez.

Dónde aparece el trabajo paralelo

Variante	Antes de calcular j	Trabajo repartido después
Left-looking	Reúne desde las columnas 0,...,j-1 las contribuciones necesarias para j.	Con el pivote listo, calcula en paralelo distintos elementos Lij de la columna actual.
Right-looking	Las columnas anteriores ya aplicaron sus aportes sobre la matriz de trabajo.	Con j lista, actualiza en paralelo muchas posiciones de las columnas futuras.

$$l_{ij} = \frac{a_{ij} - \sum_{k<j} l_{ik} l_{jk}}{l_{jj}} \quad \text{(left)}$$

$$s_{ik} \leftarrow s_{ik} - l_{ij} l_{kj}, \quad i \geq k > j \quad \text{(right)}$$

Una formulación precisa

Left-looking reúne el pasado para construir la columna actual. Right-looking propaga la columna actual hacia el futuro. Ambos admiten paralelismo; right-looking expone un conjunto amplio de actualizaciones sobre la región restante. En versiones por tareas se puede priorizar la próxima columna y continuar otras actualizaciones como look-ahead.

Nuestra versión right-looking es sincronizada por columnas: difunde j y cada proceso actualiza sus columnas propias antes de la etapa siguiente. Por eso esta comparación evalúa dos implementaciones concretas, no el máximo paralelismo teórico posible de cada familia.

Cómo hicimos comparable el experimento

La misma propiedad de datos

Ambas variantes usan distribución cíclica estática. Cada proceso almacena sus columnas en memoria local contigua y recibe exactamente la misma matriz SPD. La distribución inicial y la reunión final son idénticas y quedan fuera del cronómetro del kernel.

Left-looking MPI	Right-looking MPI
Cada proceso calcula sumas parciales con sus columnas anteriores.	El propietario calcula la columna j.
MPI_Reduce reúne el pivote y los elementos inferiores en el propietario de j.	MPI_Bcast difunde el tramo terminado de j.
El propietario guarda la nueva columna; se difunde el pivote.	Cada proceso actualiza sus columnas futuras con vector local contiguo.

Diferencia de coordinación

Por columna, left usa una reducción escalar, una reducción vectorial -salvo en la última columna- y una difusión escalar: hasta 3n-1 colectivas. Right usa una difusión vectorial por columna: n colectivas. Los volúmenes exactos dependen de la implementación de MPI, pero left introduce más puntos de coordinación.

Protocolo

Decisión	Valor
Tamaños y procesos	n=64,128,256,512,1024; P=1,2,3,4.
Repeticiones	Un calentamiento y ocho mediciones por algoritmo y configuración. Orden left/right alternado.
Entorno	WSL2, AMD Ryzen 5 2600, Open MPI 5.0.10, GCC 15.2.0, -O3, double, sin BLAS.
Tiempo	Máximo de las duraciones locales tras una barrera. Incluye cálculo y colectivas; excluye Scatterv, Gatherv y validación.
Correctitud	Residuo relativo <=10^-12 y comparación con el factor right secuencial.

También ejecutamos un perfil separado para n=1024 y P=3. Sus cronómetros internos alteran el tiempo total; se usan para interpretar componentes, no para sustituir las medianas principales.

Resultado: right fue más rápido en todos los casos

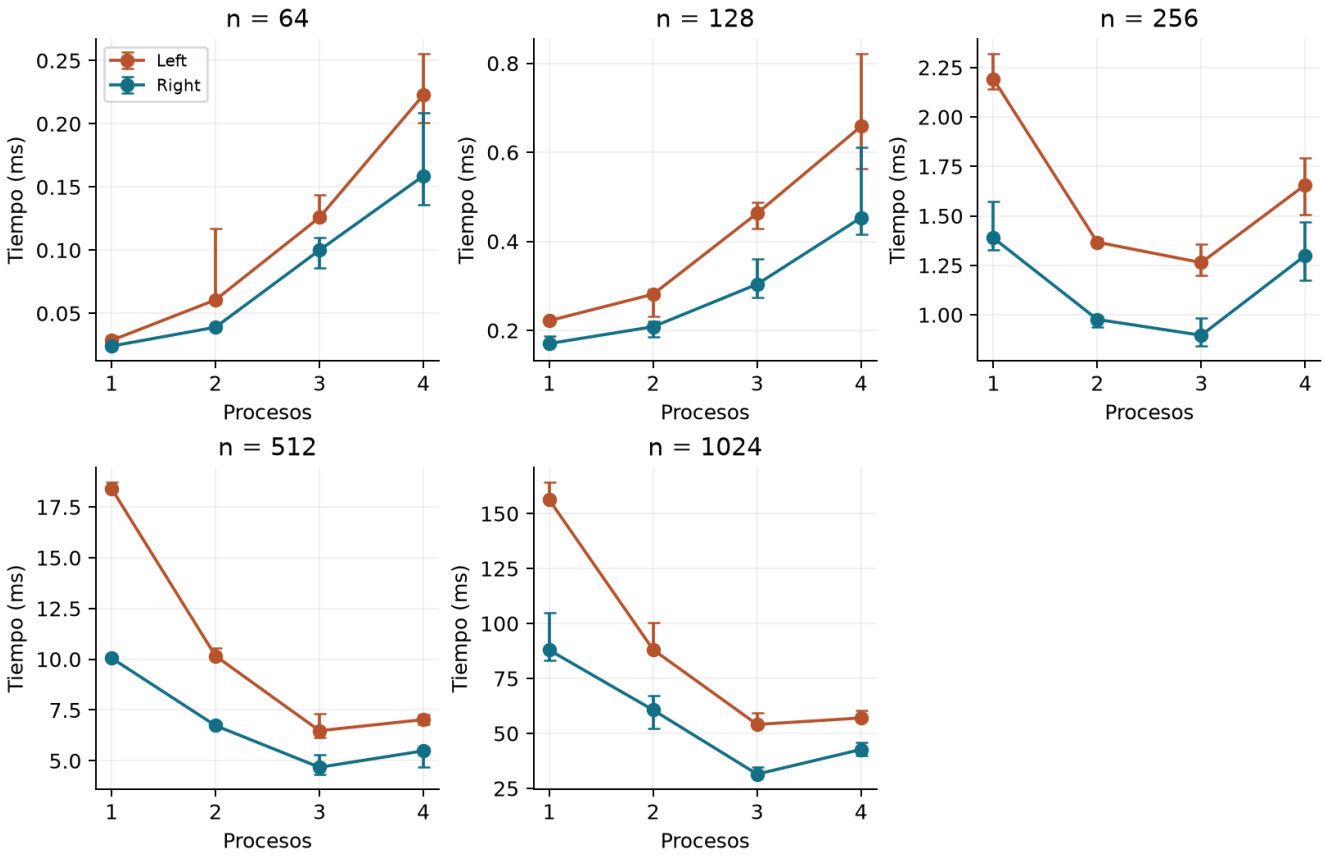


Figura 1. Medianas de ocho repeticiones; barras Q1-Q3. Se usa una escala propia por panel.

n	P	Left (ms)	Right (ms)	Left / Right
64	1	0,03	0,02	1,19
128	1	0,22	0,17	1,30
256	3	1,26	0,90	1,41
512	3	6,48	4,68	1,38
1024	3	54,08	31,48	1,72

La tabla usa, para cada n, el P que minimizó la mediana de right. No selecciona P por separado para favorecer a left. En las 20 configuraciones completas, right tuvo menor mediana.

Caso principal: n=1024

Con tres procesos, left tardó 54,08 ms y right 31,48 ms: left fue 1,72 veces más lento. Respecto de su propio P=1, left alcanzó 2,89x y right 2,79x. Ambos mejoraron al paralelizar y ambos empeoraron al pasar de tres a cuatro procesos.

Por qué right ganó en nuestra implementación

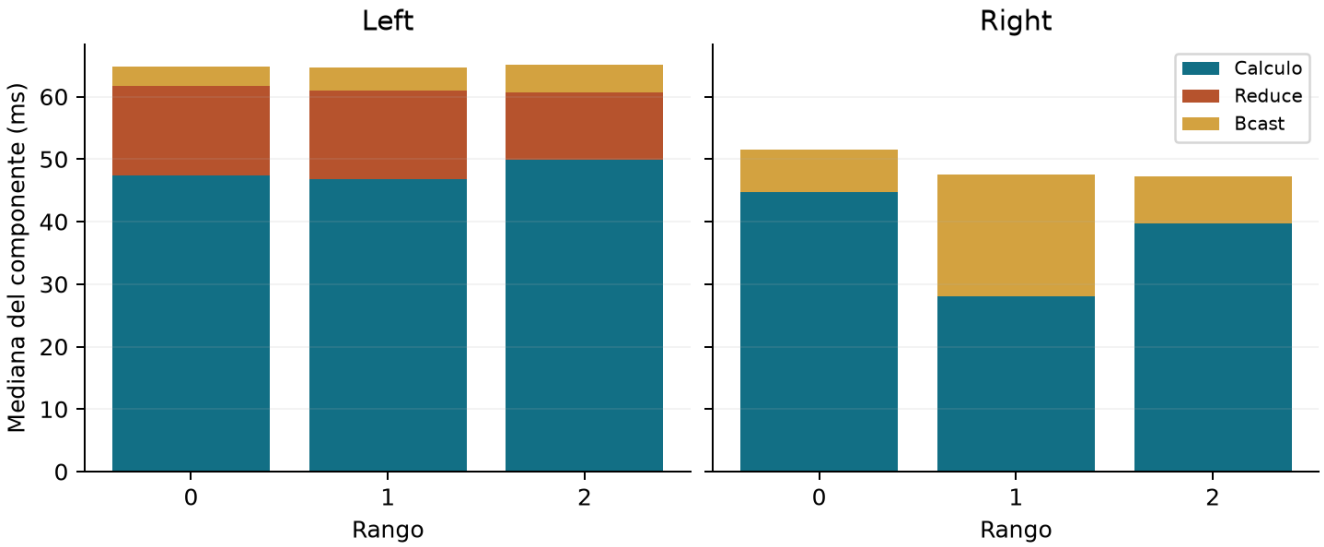


Figura 2. Perfil separado de n=1024, P=3, cuatro repeticiones por algoritmo. Se apilan medianas de componentes; su suma no tiene por qué coincidir exactamente con la mediana del total. Las colectivas incluyen espera.

La comunicación de left es más fragmentada

Left necesita reunir sumas parciales en cada columna. Aunque reparte bien los productos locales, paga reducciones vectoriales y escalares además de difundir cada pivote. Right difunde la columna terminada una vez y realiza actualizaciones locales sobre los datos que cada proceso posee.

En el perfil, los tres rangos de left acumulan aproximadamente entre 10,80 y 14,29 ms dentro de MPI_Reduce, además de sus difusiones. Right no usa reducciones en el kernel. Esta observación explica parte de la diferencia, pero el tiempo dentro de una colectiva mezcla transferencia y espera.

No es una prueba universal

La conclusión válida es: **para nuestra implementación escalar, almacenamiento por filas, distribución cíclica, Open MPI local y tamaños probados, conviene continuar paralelizando right-looking con vector local.** No demostramos que toda implementación right sea superior a toda left.

Una versión left por bloques, con BLAS, menos reducciones o comunicación no bloqueante podría cambiar el resultado. Una versión right con look-ahead también podría mejorar. Esas comparaciones requieren otra etapa y no deben darse por realizadas.

Validación

Se aceptaron 320 tiempos del experimento y 24 registros de perfil. El residuo máximo fue 1,2131×10^-15. La máxima diferencia relativa elemento a elemento frente al factor secuencial fue 2,6578×10^-15. Las pruebas con P=1,2,3,4 y los sanitizadores con P=4 finalizaron correctamente.

Decisión y explicación para la consulta

Qué elegimos

Continuamos con right-looking MPI cíclico y vector local. La elección tiene ahora dos fundamentos: su estructura permite repartir las actualizaciones futuras entre propietarios de columnas, y la comparación directa mostró menor tiempo que el left-looking MPI implementado con el mismo reparto.

Cómo explicarlo oralmente

“Left-looking también puede paralelizarse: cada proceso forma contribuciones con las columnas anteriores y estas se combinan para calcular la columna actual. Right-looking calcula una columna y reparte las actualizaciones de la matriz restante. Implementé ambas con la misma distribución cíclica. En mis pruebas, right fue más rápido en todas las configuraciones; con n=1024 y tres procesos tardó 31,48 ms frente a 54,08 ms de left. El perfil muestra que left necesita más colectivas de reducción. Por eso elijo right para continuar, aclarando que es una conclusión sobre estas implementaciones.”

Qué sigue pendiente

Todavía medimos el kernel de factorización. El siguiente paso práctico es incluir distribución inicial y reunión final para comprobar si la ventaja se mantiene en el tiempo completo. Luego corresponde repetir dentro de un nodo y entre nodos del clúster.

Archivo en desde_cero	Contenido
src/cholesky_left_mpi.c	Left-looking distribuido y perfil de reducciones.
src/compare_mpi_algorithms.c	Comparación controlada, reunión y validación.
scripts/comparar_left_right_mpi.sh	Campaña reproducible.
scripts/analizar_left_right_mpi.py	Estadísticas, validación y gráficos.
resultados/left_right_mpi_20260922/	CSV, protocolo, perfil, huellas y fuentes.

¿La mejora se mantiene al mover la matriz?

Los experimentos anteriores medían el kernel de factorización. Sin embargo, una aplicación puede comenzar con la matriz A completa en un proceso y necesitar que el factor L vuelva completo al mismo proceso. En ese caso, repartir y reunir los datos también forman parte del costo útil.

Caso de uso que medimos

Fase	MPI	Secuencial
Entrada	El rango 0 empaqueta las columnas cíclicas y ejecuta MPI_Scatterv.	Copia A al arreglo que se sobrescribirá.
Factorización	Right con vector local o left distribuido.	Right-looking con vector contiguo.
Salida	MPI_Gatherv y desempaquetado del factor completo en rango 0.	El factor ya está completo en el arreglo local.

Qué queda fuera

La reserva de memoria, la generación de A, la comprobación del resultado y el arranque de los procesos MPI quedan fuera. El lanzamiento puede pagarse una vez si una aplicación mantiene trabajadores activos, o en cada problema si crea un nuevo trabajo; por eso debe estudiarse por separado.

Tiempo total observado

$$T_{\text{total}} = T_{\text{entrada}} + T_{\text{factor}} + T_{\text{salida}}$$

Cada proceso registra marcas consecutivas. Para reportar cada fase y el total tomamos el máximo entre procesos. Debido a que los máximos de las fases podrían pertenecer a rangos distintos, la suma de los máximos por fase no tiene que coincidir exactamente con el máximo total.

Esta definición representa un flujo centralizado de entrada y salida. No representa aplicaciones donde la matriz ya nace distribuida o el factor continúa utilizándose de forma distribuida.

Protocolo de la comparación completa

Decisión	Valor
Algoritmos	Right MPI cíclico con vector local; left MPI cíclico; right secuencial con vector.
Tamaños y procesos	n=64,128,256,512,1024; P=1,2,3,4.
Repeticiones	Un calentamiento y 12 mediciones por algoritmo y configuración. Las seis permutaciones del orden se repiten dos veces.
Entrada	La misma matriz SPD, generada en C con semilla 2026, comienza completa en el rango 0.
Salida	El factor se reúne completo en rango 0 y se compara con la referencia secuencial.
Entorno	WSL2, Ryzen 5 2600, Open MPI 5.0.10, GCC 15.2.0, -O3, double, sin BLAS.

Comparación contemporánea

La referencia secuencial se ejecuta dentro del mismo lanzamiento y ocupa todas las posiciones posibles del orden. Para cada n y P comparamos MPI con esa referencia contemporánea. No dividimos por tiempos de PDF anteriores.

Dos preguntas distintas

$$S_{\text{kernel}} = \frac{\widetilde{T}_{\text{factor, seq}}}{\widetilde{T}_{\text{factor, MPI}}}, \qquad S_{\text{total}} = \frac{\widetilde{T}_{\text{total, seq}}}{\widetilde{T}_{\text{total, MPI}}}$$

La primera indica cuánto acelera Cholesky una vez repartidos los datos. La segunda decide si conviene el flujo completo descrito. Una optimización puede mejorar el kernel y aun perder cuando la matriz es pequeña.

Las barras de los gráficos muestran Q1-Q3 de 12 repeticiones. Son medidas descriptivas de dispersión, no intervalos de confianza.

Resultado: el cruce práctico se desplaza

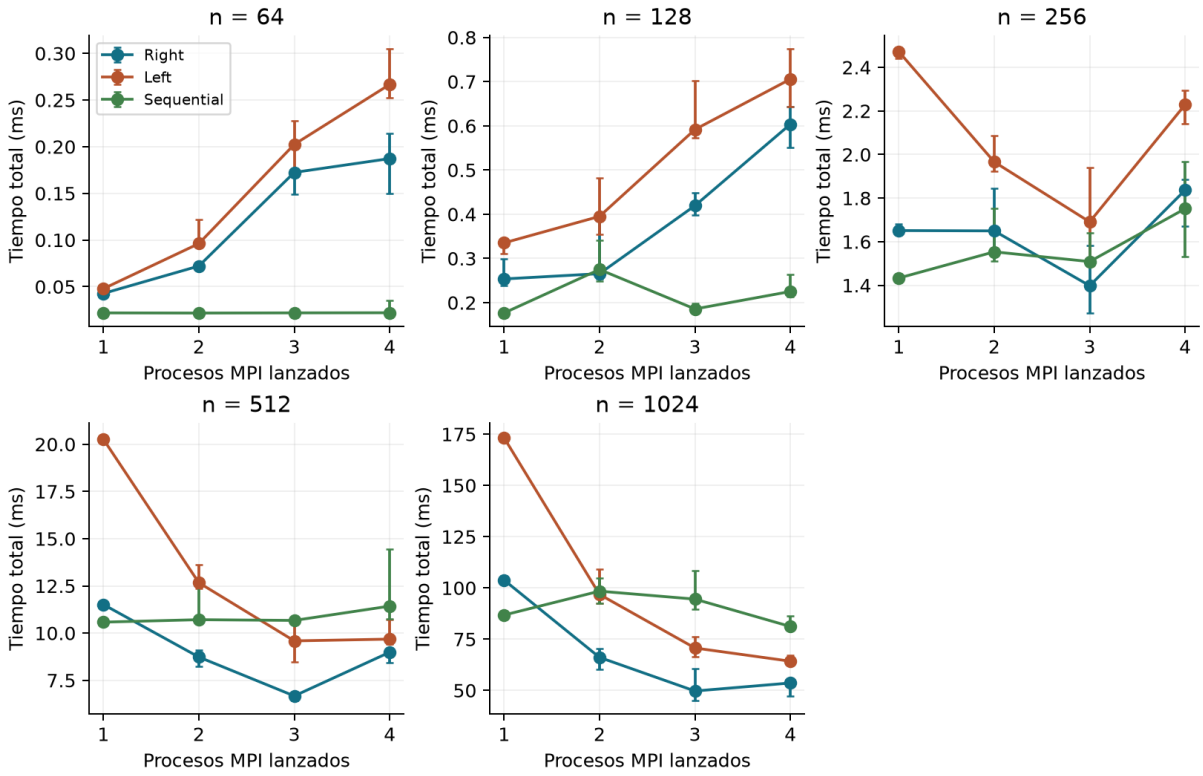


Figura 1. Tiempo completo: entrada, factorización y salida. Cada panel usa su propia escala. La referencia secuencial se repite para cada P porque se mide dentro del mismo lanzamiento.

n	Mejor P≥2	Sec. (ms)	Right total (ms)	Aceleración
64	2	0,022	0,072	0,30
128	2	0,275	0,266	1,04
256	3	1,509	1,398	1,08
512	3	10,670	6,671	1,60
1024	3	94,523	49,699	1,90

Qué cambia respecto del kernel

n=64: la coordinación y las copias dominan.
n=128: la mejor configuración paralela apenas iguala al secuencial.
n=256: right con P=3 mejora solo 1,08x, por debajo del margen práctico de 10 % usado en la etapa 4.
n=512 y n=1024: la ventaja total ya es clara: 1,60x y 1,90x con P=3.

En esta grilla, **512 es el primer tamaño con una mejora total clara**. Es una observación de esta campaña, no un umbral universal.

Qué costo agregan la entrada y la salida

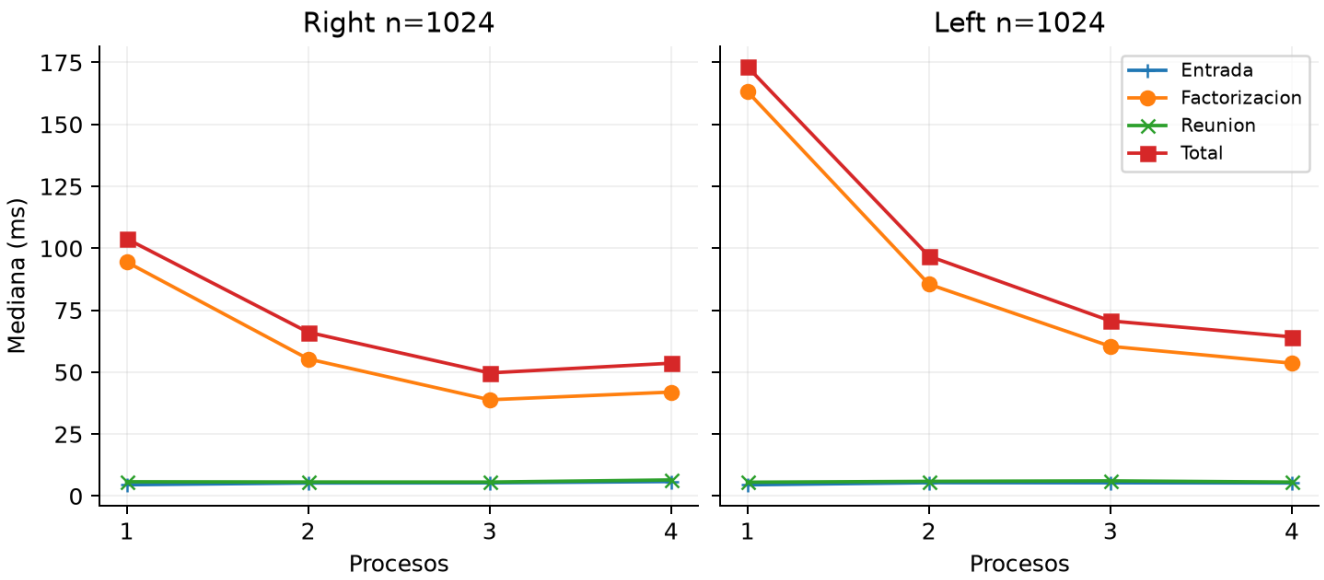


Figura 2. Medianas de cada fase y del total para n=1024. Las líneas de fases no deben sumarse como si provinieran del mismo rango y repetición.

Right con n=1024 y P=3

Entrada	Factorización	Salida	Total
5,323 ms	38,848 ms	5,630 ms	49,699 ms

El kernel right mejora 2,41x frente al secuencial. Al incluir entrada y salida, la aceleración queda en 1,90x. Los movimientos de datos reducen la ganancia, pero no la eliminan en este tamaño.

Right mantiene la ventaja frente a left

Con n=1024 y P=3, left tarda 70,678 ms en total frente a 49,699 ms de right. Como ambos usan el mismo reparto y las mismas fases externas, la diferencia continúa concentrándose en la factorización: 60,417 ms para left y 38,848 ms para right.

Una limitación visible

La entrada y la salida permanecen alrededor de cinco a seis milisegundos cada una para n=1024 y no disminuyen al aumentar P en esta máquina. El rango 0 empaqueta y desempaqueta en forma secuencial; ese diseño se convertirá en cuello de botella al escalar.

Una aplicación real podría evitar esas conversiones generando o leyendo directamente el formato distribuido, o conservar el factor distribuido. Eso cambiaría el caso de uso y debe medirse como otro experimento.

Conclusión y siguiente etapa

Decisión actual

Right-looking MPI cíclico con vector local sigue siendo la variante elegida. Fue más rápido que left en el kernel y en el flujo completo. Con matrices pequeñas conviene la versión secuencial; para n=512 y n=1024, tres procesos dieron una mejora total clara en esta campaña.

Qué afirmamos y qué no

Afirmamos resultados sobre una entrada y salida centralizadas en WSL. No sabemos todavía cómo cambia el costo dentro de un nodo del clúster ni, especialmente, entre nodos. Tampoco medimos lectura desde archivo, escritura, lanzamiento del trabajo o varias factorizaciones consecutivas.

Validación

Se aceptaron 720 observaciones. El residuo relativo máximo fue 1,2131×10^-15 y la diferencia relativa máxima frente al factor secuencial fue 2,6578×10^-15. Las pruebas funcionales y los sanitizadores con P=4 terminaron correctamente; las huellas de las fuentes medidas coinciden.

Siguiente experimento cuando tengamos el clúster

Repetiremos right secuencial y right MPI con P=1,2,3,4 dentro de un nodo. Luego compararemos una asignación equivalente entre nodos, registrando nodos, procesos por nodo, afinidad, compilador y biblioteca MPI. Mediremos entrada, kernel, salida y total con el mismo criterio.

Si la red hace dominar la comunicación, el siguiente cambio razonable será agrupar operaciones por bloques para reducir la frecuencia de mensajes y aprovechar BLAS. No conviene implementar esa complejidad antes de observar el problema en el clúster.

Archivo en desde_cero	Contenido
src/compare_mpi_total.c	Cronometra fases, reúne y valida.
scripts/comparar_tiempo_total.sh	Campaña completa reproducible.
scripts/analizar_tiempo_total.py	Estadísticas y gráficos por fase.
resultados/tiempo_total_20260922/	CSV, protocolo, afinidad, huellas y fuentes.

PARTE IV

IV

Flujo de datos y cierre local

¿La mejora persiste cuando se mide el trabajo necesario para obtener el factor completo?

Separador editorial; las páginas siguientes conservan su contenido original.

Etapa 7: ¿quién genera la matriz?

Primera etapa posterior a la consulta docente. Algoritmos y MPI en C; análisis y gráficos en Python. Esta campaña estudia la preparación de la entrada, no mide todavía toda la aplicación.

Pregunta e hipótesis

¿Podemos evitar que el proceso 0 genere y distribuya toda la matriz? Si cada proceso calcula sus columnas directamente, eliminamos el empaquetado central y Scatterv. A cambio, se recalculan los valores simétricos y aparecen accesos por columnas; la mejora debe medirse.

Implementación: exactamente la misma matriz

El generador original visita las parejas i<j en orden. Identificamos la posición de cada pareja en esa secuencia y recuperamos el estado del generador entero sin recorrer los valores anteriores. La suma diagonal conserva el mismo orden que antes. Por ello, el resultado se puede comprobar bit a bit.

$$q(i,j) = i(2n - i - 1)/2 + (j - i - 1), \quad i < j$$

$$a_{ii} = 1 + \sum_{j \neq i} |a_{ij}|$$

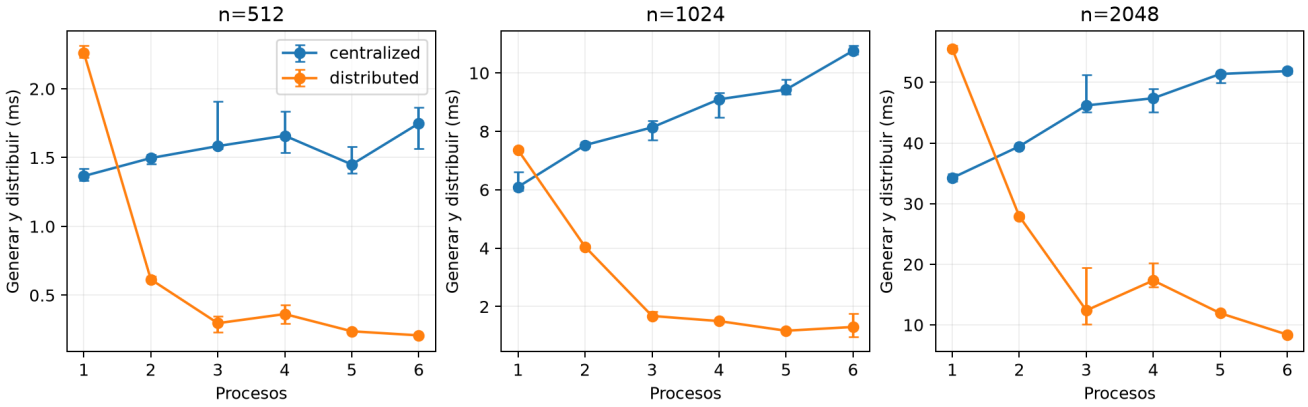
La simetría y la dominancia diagonal estricta con diagonal positiva aseguran que la matriz sea definida positiva. Cada proceso guarda sus columnas cíclicas en el formato que ya utiliza nuestro Cholesky MPI.

Protocolo

WSL, Ryzen 5 2600; 6 núcleos físicos. Tamaños 512, 1024 y 2048; P=1 a 6; semilla 2026; un calentamiento y 6 repeticiones alternadas por método. Total: 216 observaciones. Centralizado: generación en rango 0, empaquetado y Scatterv. Distribuido: generación directa local. Ambos terminan con idéntica matriz distribuida.

El cronómetro excluye reserva, arranque MPI, factorización y validación. Se toma el máximo del tiempo total entre procesos. Las fases no están sincronizadas: sus máximos incluyen esperas distintas y no deben sumarse para reconstruir el total.

Resultados y decisión provisional



Puntos: medianas. Barras: Q1-Q3; no son intervalos de confianza. Una campaña local con seis repeticiones por configuración.

n	P	Central (ms)	Distrib. (ms)	Cociente
512	1	1.366	2.262	0.60x
512	6	1.748	0.208	8.40x
1024	1	6.097	7.359	0.83x
1024	6	10.759	1.303	8.26x
2048	1	34.243	55.519	0.62x
2048	6	51.862	8.385	6.18x

Para n=2048 y P=6, el cociente entre medianas de preparación es 6.18. Este cociente compara dos métodos con el mismo P; no es el speedup de Cholesky ni el de la aplicación completa.

Comprobaciones y límites

Todas las repeticiones reprodujeron exactamente la matriz original. Se probaron también n=1,2,3,7,31 con seis procesos, incluidos procesos sin columnas. Las pruebas C y sanitizadores de las funciones nuevas pasaron. Falta repetir en sesiones independientes y medir el impacto junto con la factorización.

La reserva del banco contiene matrices adicionales para verificar igualdad; no representa la memoria mínima de una aplicación distribuida. Este experimento aún no integra el nuevo generador en el banco de tiempo completo.

¿Y si comprobar tarda demasiado?

Reconstruir todo LL transpuesta cuesta O(n³). Agregamos una comprobación complementaria con vectores de signos, reproducibles mediante una semilla separada. Para cada vector calculamos Ax y L(L transpuesta x), sin formar el producto matricial completo.

$$r_x = \frac{\|Ax - L(L^T x)\|_2}{\|Ax\|_2}$$

Con k vectores, el costo es O(kn²) y la memoria auxiliar es O(n). Esta magnitud NO es el residuo de Frobenius. Un conjunto finito de vectores puede dejar errores sin detectar: no constituye un certificado determinista ni sustituye las pruebas exhaustivas.

Las pruebas usan ocho vectores con semilla 91827, factores correctos y deliberadamente alterados, valores no finitos y elementos del triángulo superior que no pertenecen a L. También comprueban repetibilidad. El método está implementado como función independiente; los bancos históricos siguen usando el residuo completo.

Siguiente pregunta

¿La mejora al preparar la entrada reduce el tiempo que espera el usuario? El siguiente banco debe incluir reserva, generación, movimiento, factorización y salida; mantener un caso con resultado centralizado y otro distribuido; y registrar la validación por separado. Un tiempo externo adicional podrá incluir el arranque y cierre de MPI.

Ruta local pendiente

1. Integrar el flujo completo y validación escalable. 2. Matrices mayores y P=1 a 6. 3. Distribución bloque-cíclica. 4. Almacenamiento triangular. 5. Compilador y vectorización. 6. Referencia LAPACK. 7. Decidir, según evidencia y alcance defendible, algoritmo por bloques y MPI+OpenMP.

Trazabilidad y uso de IA

Fuentes: src/cholesky.c, src/compare_generation.c y tests/test_projected.c. Campaña: resultados/generacion_local_20260922, con CSV, protocolo, entorno, afinidad y copia de fuentes. Código y texto elaborados con asistencia de IA; resultados obtenidos mediante ejecución local. La defensa debe explicar las decisiones, las pruebas y sus límites.

Etapa 8: ¿mejora el tiempo completo?

La generación distribuida redujo el tiempo de preparar la entrada en la etapa anterior. Ahora preguntamos si esa mejora se mantiene al incluir la factorización y devolver el resultado al proceso 0.

Una comparación con el mismo resultado

Centralizada: rango 0 genera A, empaqueta y distribuye columnas. Distribuida: cada proceso genera directamente sus columnas de la misma A. Ambas ejecutan right-looking MPI cíclico con vector local y reúnen L completa en rango 0. La referencia secuencial genera A y aplica right-looking con vector en rango 0.

Qué llamamos total en este experimento

$$T = T_{reserva} + T_{generacion} + T_{distribucion} + T_{factorizacion} + T_{recoleccion}$$

Medimos directamente el intervalo desde el comienzo de las reservas hasta que L queda completa en rango 0, con MPI ya iniciado. Las fases se registran consecutivamente en ese mismo proceso y suman su total por ejecución. También guardamos el máximo de las duraciones locales como control.

Quedan fuera el lanzamiento y cierre de MPI, las referencias de prueba, la validación, la liberación de memoria y escribir resultados. Es el flujo de procesamiento, no el tiempo externo del comando mpirun. Las reservas se repiten; malloc no toca todas las páginas, por lo que parte del costo de memoria se paga al acceder a ellas en las fases posteriores.

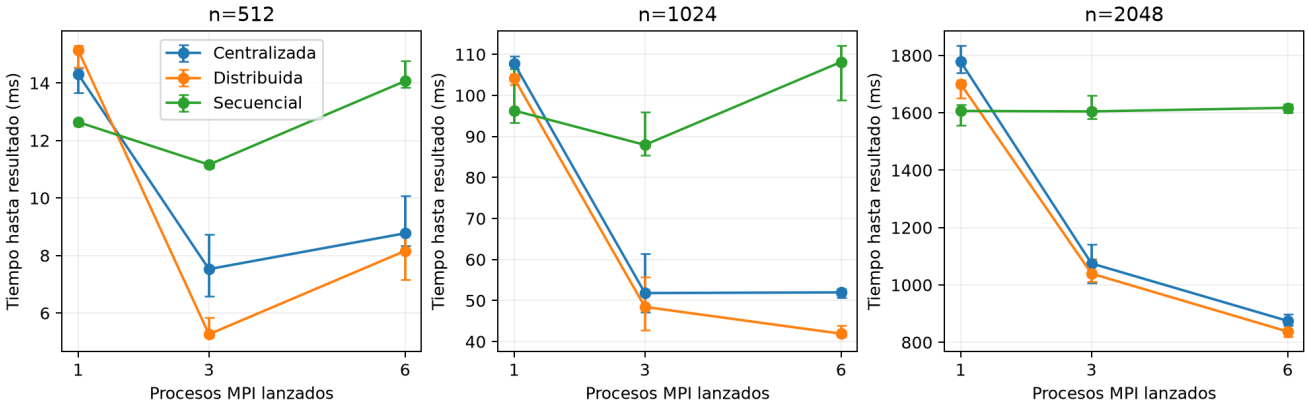
Por qué usar medianas

Calculamos la mediana de los totales observados. No sumamos medianas de fases para obtenerla. Para conocer el peso de una fase, dividimos su duración por el total de esa misma ejecución y luego calculamos la mediana de esas proporciones. Estas medianas tampoco tienen que sumar exactamente 100 %.

Protocolo

n=512,1024,2048; P=1,3,6; un calentamiento y seis repeticiones por método y configuración, usando las seis permutaciones de los tres métodos. Son 162 observaciones en una sesión WSL. Tamaños y P se recorren en orden creciente. Ryzen 5 2600, afinidad por núcleo, GCC -O3; semilla 2026. El secuencial corre dentro del lanzamiento MPI con los otros procesos esperando, condición que debe tenerse presente al interpretar su referencia.

Resultados del flujo integrado



Medianas y barras Q1-Q3. No son intervalos de confianza.

n	P	Central (ms)	Distrib. (ms)	Secuencial (ms)
512	3	7.53	5.26	11.16
512	6	8.77	8.16	14.06
1024	3	51.82	48.44	87.92
1024	6	51.98	41.91	108.15
2048	3	1074.82	1039.34	1604.65
2048	6	874.60	837.39	1617.31

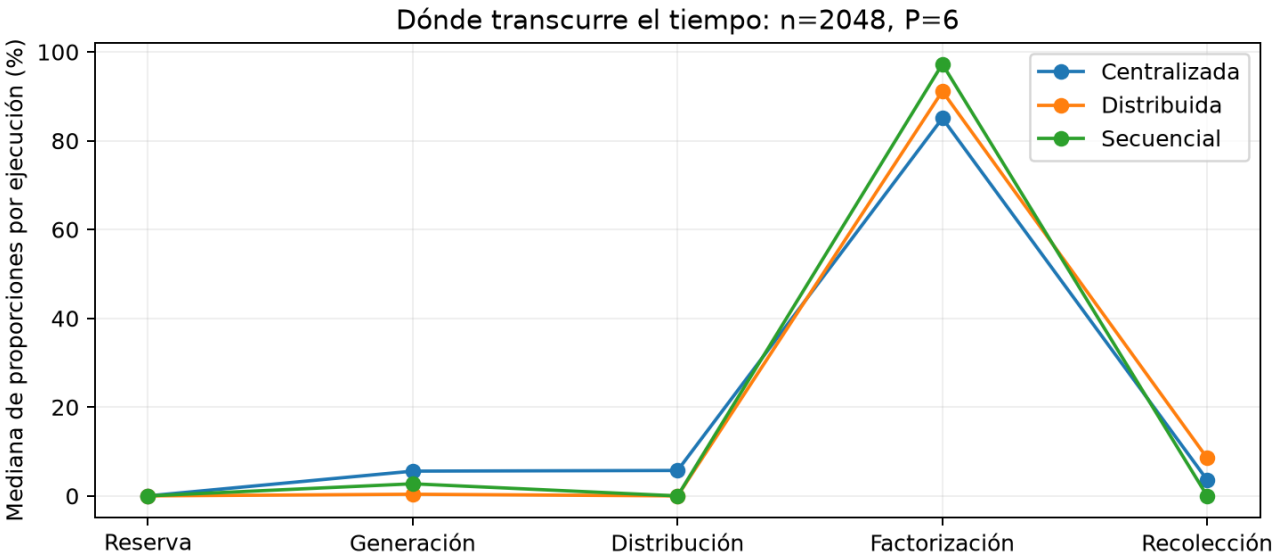
Para n=2048 y P=6: cambiar la preparación centralizada por distribuida da un cociente de 1.04x y una reducción de 4.3 % del tiempo mediano. Respecto del secuencial medido en esa configuración, el método distribuido alcanza 1.93x.

Lo que esta comparación permite afirmar

Comparamos resultados completos equivalentes y datos idénticos, sin combinar tiempos de campañas diferentes. La velocidad de la generación aislada no se traslada automáticamente al total: la factorización, la salida y las interacciones con memoria siguen costando tiempo.

No atribuimos diferencias de factorización exclusivamente al algoritmo, que es el mismo en ambas variantes MPI. La generación cambia el estado de caché, el primer acceso a memoria y cuándo llega cada proceso al kernel. Los tiempos de fases incluyen las esperas resultantes.

¿Dónde conviene trabajar ahora?



En el flujo distribuido de n=2048, P=6, la factorización representa una mediana del 91.1 % del tiempo observado y la recolección del 8.5 %. Estas proporciones sirven para priorizar el próximo experimento local; no predicen el comportamiento de la red del clúster.

Validación

Se aceptaron 162 observaciones. Máximo residuo proyectado: 1.611e-15; máxima diferencia relativa por elemento respecto del factor secuencial: 0.000e+00. Se comprobó el factor completo contra una referencia secuencial en cada repetición. El residuo Frobenius completo se usa en las pruebas pequeñas (hasta n=128).

La comprobación proyectada usa ocho vectores de signos y semilla 91827, fuera del tiempo útil, y su duración se registra por separado. No es un certificado ni el residuo Frobenius. Las pruebas incluyen n=1,2,3,7,31,128 y procesos sin columnas.

Próxima hipótesis y límites

Si el cálculo concentra el tiempo, el próximo cambio debe estudiar la factorización y el reparto, empezando por bloques de columnas cíclicos. Una distribución bloque-cíclica modifica propietarios; un algoritmo Cholesky por bloques modifica las operaciones. Debemos mantener esa distinción y cambiar una decisión por experimento.

Una sola sesión y seis repeticiones no establecen un ganador universal. Falta confirmar configuraciones elegidas en sesiones independientes, ampliar tamaños y medir lanzamiento externo si la aplicación crea procesos por cada matriz. El banco conserva matrices de referencia fuera del cronómetro: su consumo de memoria no es el mínimo del algoritmo.

Fuentes: src/compare_application.c; scripts/comparar_aplicacion.sh; scripts/analizar_aplicacion.py. Evidencia: resultados/aplicacion_local_20260922. Implementación y documentación con asistencia de IA, comprobadas mediante ejecución local.

Etapa 9: ¿agrupar columnas ayuda?

La etapa anterior mostró que la factorización concentra gran parte del tiempo. Estudiamos ahora si cambiar la propiedad de las columnas mejora el flujo completo. Se mantiene right-looking escalar: cada columna se termina y difunde antes de continuar.

Hipótesis

Un bloque de b columnas consecutivas se asigna por turnos a cada proceso. Agrupar columnas puede modificar la localidad y las esperas, pero también el reparto del trabajo. Como las columnas locales ya estaban empaquetadas por filas, b mayor no garantiza mejor acceso a memoria.

$$\text{propietario}(j) = \lfloor j/b \rfloor \bmod P$$

Con P=3 y b=2: proceso 0 recibe 0,1,6,7; proceso 1 recibe 2,3,8,9; proceso 2 recibe 4,5,10,11. b=1 reproduce los propietarios de la distribución cíclica previa.

Qué cambia y qué se conserva

Cambia el mapa global/local y la cantidad de columnas de cada proceso. Se conserva la matriz generada, el kernel de actualización con vector local, n difusiones por factorización y L reunida en rango 0. Todas las variantes usan el mismo código general de índices, incluido b=1. No comparamos contra cifras de campañas anteriores.

Esto es distribución por bloques de columnas. No es Cholesky por bloques con operaciones matriz-matriz ni agrupa las comunicaciones: sigue habiendo una difusión por columna.

Protocolo y reloj

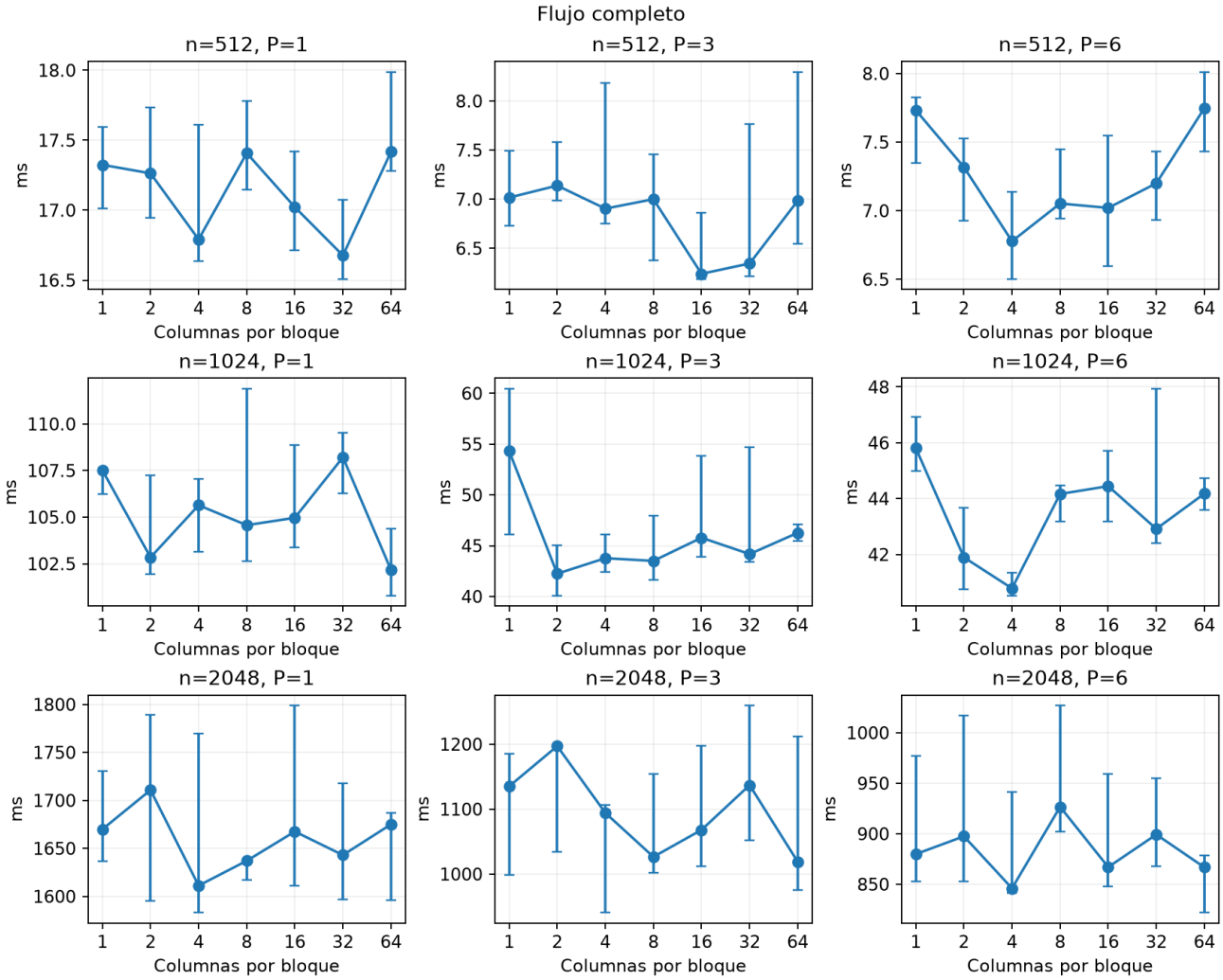
n=512,1024,2048; P=1,3,6; b=1,2,4,8,16,32,64. Un calentamiento por variante y siete repeticiones. El orden rota para que cada bloque ocupe cada posición una vez; no balancea todas las parejas de vecinos. Se rota P entre tamaños. Total: 441 observaciones en una sesión local WSL, afinidad por núcleo, GCC -O3, semilla 2026.

Total principal: reloj de rango 0 desde reserva y preparación del reparto hasta generación distribuida, factorización y recolección completa. Excluye lanzamiento/cierre MPI, referencias, validación y liberación. Las fases incluyen esperas. Se compara la mediana de totales medidos directamente.

Control con un proceso

Con P=1, las columnas y operaciones son iguales para todos los tamaños de bloque. Diferencias grandes en ese control requieren cautela: no pueden atribuirse a balance entre procesos. El mapa general introduce aritmética distinta de las versiones históricas.

Tiempo hasta obtener el resultado



Puntos: medianas; barras: Q1-Q3, no intervalos de confianza. Cada panel tiene su propia escala vertical. Comparamos bloques dentro de cada configuración.

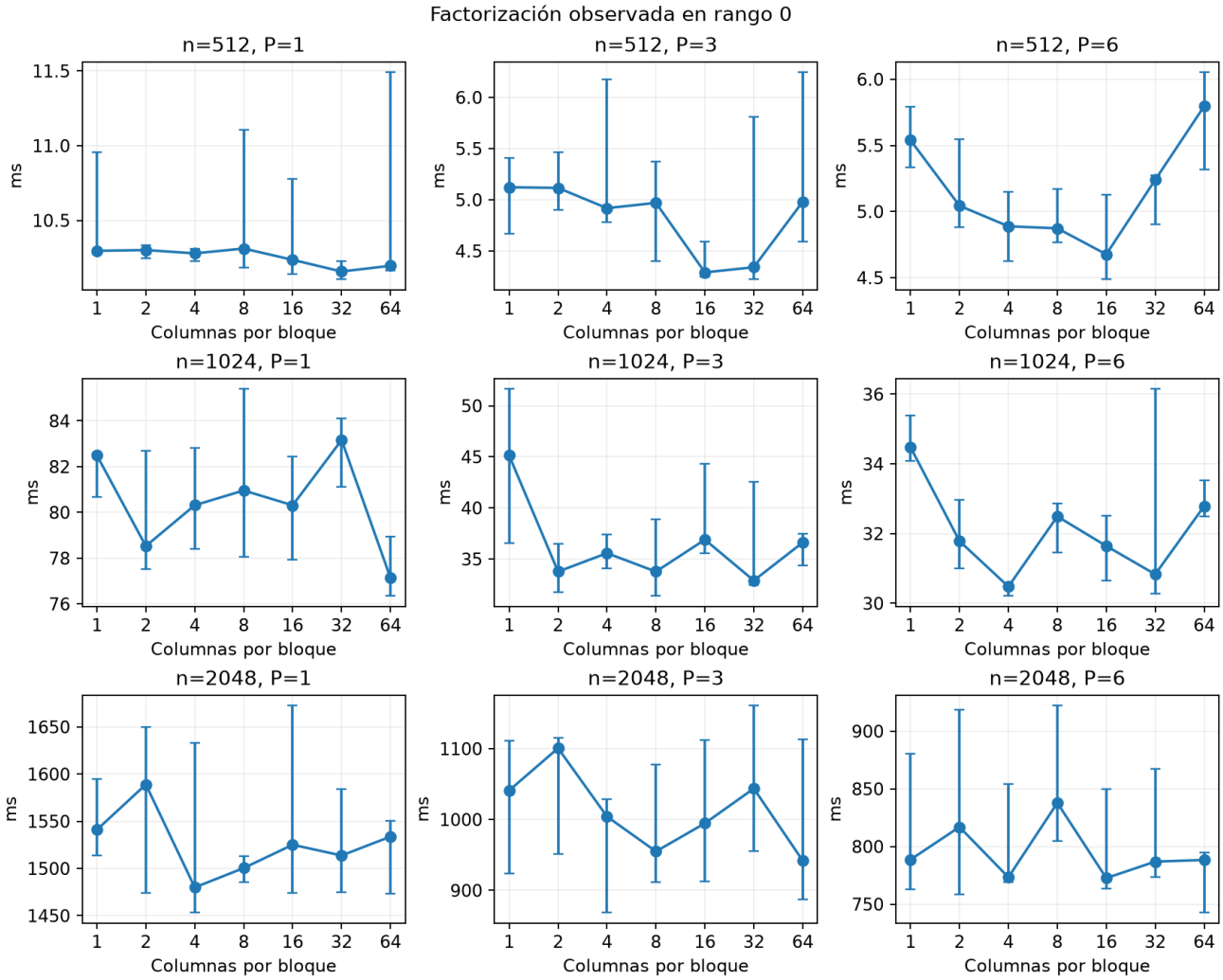
Interpretación

El total contempla los cambios de generación y desempaquetado provocados por el reparto, además del kernel. Una menor mediana aislada es un candidato para repetir, no una demostración de superioridad universal. Elegir el mínimo entre siete alternativas puede favorecer fluctuaciones de esta sesión.

Reproducibilidad

Campaña: resultados/bloques_local_20260922. Implementación: src/compare_blocks.c y mapas de src/cholesky_mpi.c. Ejecución: scripts/comparar_bloques.sh; análisis: scripts/analizar_bloques.py. Código, entorno y fuentes congeladas acompañan los resultados. Desarrollo y texto con asistencia de IA; datos obtenidos ejecutando el código localmente.

Factorización y reparto del trabajo



Factorización observada por rango 0: incluye sus esperas de comunicación. No es una medición separada de cómputo puro ni del máximo de cada proceso.

$$W_r = \sum_{k \in C_r} k(n-k), \qquad I = \frac{\max_r W_r}{\sum_r W_r / P}$$

Cada columna k recibe k actualizaciones sobre n-k elementos. I=1 indica igualdad en este conteo acumulado; no garantiza balance en cada etapa, ni incluye pivotes, indexación o comunicaciones.

n=2048, P=6: b=1: 1.0000, b=2: 1.0000, b=4: 1.0000, b=8: 1.0002, b=16: 1.0006, b=32: 1.0026, b=64: 1.0090.

Decisión provisional y siguiente pregunta

n	P	Mejor b observado	b=1 (ms)	Mejor (ms)	Cociente
512	3	16	7.02	6.24	1.124x
512	6	4	7.73	6.78	1.141x
1024	3	2	54.32	42.24	1.286x
1024	6	4	45.81	40.78	1.123x
2048	3	64	1135.03	1018.87	1.114x
2048	6	4	879.80	845.97	1.040x

Estos mínimos son descriptivos y fueron elegidos después de medir. Seleccionamos b=4 para una repetición con P=6, manteniendo b=1 como referencia. No se selecciona un tamaño universal.

Repetición del candidato

n=1024, P=6: b=1 tarda 48.93 ms y b=4 tarda 44.02 ms; cociente 1.112x. Son nuevas ejecuciones, con siete repeticiones por bloque, en la misma sesión local.

n=2048, P=6: b=1 tarda 931.67 ms y b=4 tarda 929.70 ms; cociente 1.002x. Son nuevas ejecuciones, con siete repeticiones por bloque, en la misma sesión local.

Esta repetición añade 98 observaciones. Para n=1024 se mantiene una reducción cercana al 10 % con b=4; para n=2048 la diferencia cae a aproximadamente 0,2 %, insuficiente para sostener una ventaja práctica estable. Conservamos b=1 como referencia y b=4 como candidato para n=1024, P=6. No adoptamos b=4 universalmente.

Validación

Se aceptaron 441 observaciones. Máximo residuo proyectado: 1.611e-15; máxima diferencia respecto del factor secuencial: 0.000e+00. Se comprueban propietarios, cantidad de columnas, prefijos e índices; b=1 se compara con el mapa cíclico original. Casos pequeños: n=1,2,3,7,31,65 con P=1,3,6, incluyendo bloques incompletos y procesos vacíos.

Las matrices grandes se contrastan elemento a elemento con el factor secuencial y con ocho proyecciones. Los casos pequeños también usan residuo Frobenius completo. El análisis Python verifica el conteo de trabajo mediante una enumeración independiente. El residuo proyectado por sí solo no es un certificado.

Próxima pregunta

¿La ventaja se mantiene en otras sesiones y matrices? Si la distribución aporta poco, corresponde investigar reutilización de datos dentro del cálculo, mediante una referencia LAPACK y, según el alcance, Cholesky por bloques. El reparto por bloques no sustituye esa optimización.

Etapa 10: cerrar decisiones locales

24 de septiembre de 2026. Completamos el alcance previo al clúster acordado: confirmar el reparto, incorporar una referencia LAPACK de un hilo, probar un tamaño mayor y preparar scripts configurables. Las conclusiones siguen siendo locales.

¿Se mantiene la mejora de cuatro columnas?

El 22 de septiembre b=4 pareció prometedor para n=1024 y P=6. Lo repetimos en esta nueva sesión con b=1 y b=4, ocho repeticiones por variante, orden alternado y el mismo resultado reunido en rango 0. El total contiene reserva, generación distribuida, kernel y recolección; MPI ya está iniciado.

n	P	b=1 total (ms)	b=4 total (ms)
1024	6	42.34	43.36
2048	6	767.99	798.87
3072	6	4030.13	4038.46

Para n=1024 y n=2048, b=4 no reduce la mediana total en esta sesión. No sostenemos la ventaja anterior como regla. Conservamos b=1 como referencia principal; b=4 queda disponible para comparar en el clúster.

¿Podemos aumentar el tamaño local?

n=3072 se ejecutó y validó con P=6. Solo hicimos dos repeticiones por bloque: es un piloto de factibilidad, no evidencia estadística suficiente para elegir reparto. El resultado no fija el tamaño adecuado del clúster.

Se registraron entorno, afinidad, tiempos externos y uso máximo de memoria informado por time -v. El RSS de ese comando no representa la suma de memoria de todos los rangos. La estimación previa considera las copias de referencia y validación; no mide la memoria mínima del algoritmo.

Qué cambió respecto del experimento anterior

El banco admite seleccionar únicamente b=1 y b=4. Los mapas, el kernel y las condiciones numéricas son los mismos. Se registran los datos nuevos sin sobrescribir campañas anteriores. Una mejora observada una vez no obliga a adoptarla: repetir también puede justificar descartarla.

¿Qué aporta LAPACK a la comparación?

LAPACK proporciona rutinas de álgebra lineal. DPOTRF calcula Cholesky y organiza operaciones por bloques utilizando BLAS de nivel 3. LAPACKE es su interfaz C; OpenBLAS suministra rutinas optimizadas para el procesador. Esta referencia combina algoritmo e implementación optimizada: no aísla cuánto aporta cada técnica.

La idea que debemos poder explicar

$$A = LL^T$$

$$L_{11}L_{11}^T = A_{11}, \quad L_{21}L_{11}^T = A_{21}$$

$$A_{22} \leftarrow A_{22} - L_{21}L_{21}^T$$

Esta identidad por bloques explica el trabajo: factorizar un bloque diagonal, resolver el panel inferior y actualizar una región con productos matriciales. Es un esquema conceptual right-looking; no afirmamos que reproduzca exactamente el orden interno de la biblioteca instalada. Las operaciones sobre submatrices permiten reutilizar datos y emplear kernels especializados.

Comparación controlada desde C

Nuestro right-looking con vector y LAPACK se ejecutan en el mismo programa, sin MPI, ligados a CPU 0 en esta PC. OpenBLAS se configura en un hilo y se comprueba con su API. Misma semilla 2026, precisión double, generador y política de reserva. Ambos reservan un vector n; LAPACK no necesita usarlo. Un calentamiento y seis repeticiones alternadas para n=512,1024,2048; dos para n=3072.

Tiempo de factor: llamada de factorización. Total: reserva + generación + factor. Referencias, validación y liberación están fuera. No hay conversión de almacenamiento: A simétrica por filas se interpreta por columnas, DPOTRF produce U y el mismo arreglo representa L=U transpuesta por filas. Un caso con factor conocido comprueba esa interpretación.

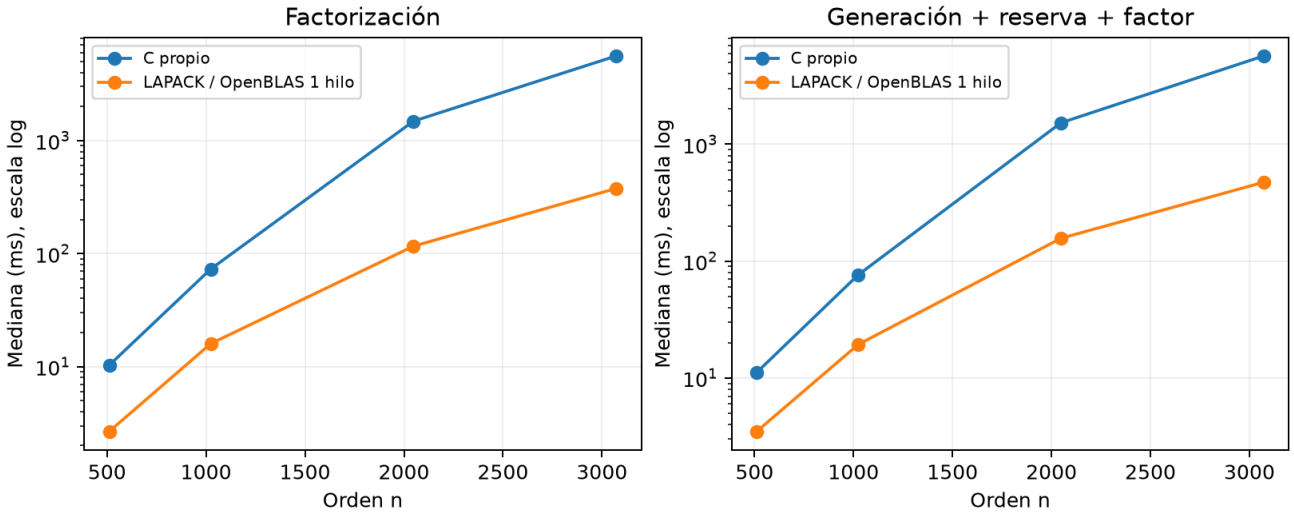
Usamos LAPACKE_dpotrf_work para evitar una transposición de la interfaz row-major y su barrido opcional de NaN. El dominio medido es entrada finita SPD generada; validamos resultados por separado. Los contratos de error de las implementaciones no son idénticos.

Bibliotecas y fuentes consultadas

LAPACKE 3.12.1; OpenBLAS 0.3.32, backend Zen, un hilo comprobado. Los logs incluyen dependencias dinámicas y configuración. La optimización interna de OpenBLAS no usa necesariamente las mismas opciones que nuestro código compilado con -O3.

DPOTRF: contrato y algoritmo; LAPACKE: interfaz C y almacenamiento; OpenBLAS: configuración de ejecución.

Resultados: factor y tiempo completo



Medianas; escala vertical logarítmica. n=3072 es exploratorio con dos repeticiones por método. No se mezclan tiempos de MPI con este experimento secuencial.

n	C factor ms	LAPACK factor ms	C total ms	LAPACK total ms
512	10.33	2.68	11.19	3.50
1024	73.03	15.96	76.37	19.37
2048	1484.35	116.30	1527.24	156.85
3072	5603.73	377.22	5701.11	473.59

Para n=2048, el cociente C/LAPACK es 12.76x en factor y 9.74x en total. Es una comparación de implementaciones con un hilo, no speedup MPI ni prueba de que un algoritmo sea siempre superior.

Qué decidimos con estos datos

Existe margen importante en el cálculo local frente a una biblioteca optimizada. Esto justifica presentar LAPACK como referencia y explicar bloques/BLAS como mejora futura. No obliga a reimplementar la biblioteca antes del clúster ni invalida el estudio educativo de MPI por columnas.

Corrección y alcance

76 observaciones nuevas aceptadas: 36 MPI y 40 secuenciales/LAPACK. Residuo proyectado máximo 1.699e-15; diferencia máxima respecto del factor secuencial 3.978e-15. Pruebas pequeñas con residuo completo, factor conocido y rechazo de una matriz indefinida. Pruebas C, MPI y sanitizadores de la interfaz propia pasaron; la biblioteca instalada no fue recompilada con sanitizadores.

Ocho proyecciones y comparación de factores en matrices grandes; el residuo proyectado no es un certificado. La generación de referencia y el cálculo secuencial de referencia continúan fuera del reloj principal y cuestan memoria y tiempo.

Preparados para probar en el clúster

Archivos operativos

scripts/campana_cluster.sh: tamaños, procesos, repeticiones, lanzador y presupuesto de memoria configurables; exporta CSV, entorno, afinidad y fuentes. scripts/analizar_pares_cluster.py: análisis con biblioteca estándar Python para grillas nuevas. docs/GUIA_CLUSTER.md: instrucciones de compilación, métricas, límites y coordinación con el administrador.

Orden de ejecución

1. Confirmar con el administrador módulos, MPI, RAM, núcleos físicos, asignación y lanzamiento. 2. Compilar en destino y ejecutar tests pequeños. 3. Pilotos en un nodo para elegir tamaños grandes que quepan y duren suficiente. 4. Comparar un nodo y varios nodos con el mismo n y P cuando sea posible. 5. Aumentar P y repetir en distintas asignaciones. Registrar ubicación real y no solo cantidad de procesos.

Qué comparación debemos preservar

Conservar datos, salida centralizada, compilación, algoritmo y definición del reloj. Medir una referencia secuencial contemporánea en el mismo hardware; identificar si es nuestra implementación o LAPACK. No calcular speedup del clúster usando tiempos WSL. La espera en MPI puede mezclar comunicación y desbalance; debe interpretarse como tiempo observado, no latencia pura de red.

Límites que siguen abiertos

No ejecutamos pruebas entre nodos. El script no solicita asignaciones ni inventa directivas del planificador. Está probado localmente con mpirun; su configuración real dependerá del administrador. El ejecutable admite n hasta 20000; tamaños mayores exigen revisar conteos MPI, índices y memoria antes de ampliar ese límite.

El control de memoria por defecto usa 60 % de MemAvailable del host de lanzamiento. En el clúster debe fijarse al presupuesto de la asignación. Las referencias y copias extra limitan este banco; no es una implementación de memoria mínima.

Conclusión local y próximo interrogante

Se conserva right-looking MPI cíclico de una columna como referencia. Cuatro columnas no demostraron una mejora estable entre sesiones. LAPACK revela margen en cálculo local. Ahora la pregunta es cómo cambia el equilibrio entre cómputo y comunicación cuando participan nodos distintos. Almacenamiento triangular, híbrido MPI+OpenMP y Cholesky por bloques quedan como extensiones, no requisitos para comenzar.

Trazabilidad y asistencia

Campaña: resultados/precluster_20260924. Fuentes y comandos acompañan los CSV. Desarrollo, análisis y texto con asistencia de IA; mediciones obtenidas ejecutando C en la PC. Las conclusiones deben poder defenderse explicando las decisiones y sus límites.

Etapa 11: un programa candidato completo

Pregunta e hipótesis

Ya elegimos una variante a partir de bancos experimentales. ¿Podemos reunirla en un programa claro, correcto y medible sin cambiar el algoritmo? La hipótesis es que la composición conserva la corrección; su rendimiento debe medirse nuevamente.

Qué conservamos

Cholesky right-looking escalar en C; MPI; propiedad 1D cíclica de columnas (b=1); vector local contiguo; generación distribuida y factor completo reunido en rango 0. No añadimos OpenMP, LAPACK ni un algoritmo por bloques.

$$\text{owner}(j) = j \bmod P, \qquad A = LL^T$$

Un programa, dos compilaciones comparables

src/main_candidate.c es la entrada. Con USE_MPI se enlaza con cholesky_mpi.c y cholesky.c. Sin USE_MPI produce una referencia secuencial autónoma con right-looking buffered. Así comparten generador, validación y definición de salida, pero el secuencial no inicia MPI ni deja procesos inactivos esperando.

Entrada: N entre 1 y 20000; semilla fija 2026. Ese límite protege los conteos MPI, no garantiza memoria disponible. No hay lectura de matrices externas. El límite local probado con siete repeticiones es N=2048; N=3072 pertenece al piloto histórico.

Cómo ejecutarlo desde desde_cero, en Linux/WSL

```
make candidate
./build/cholesky_candidate_seq 1024
mpirun --bind-to core --map-by core -np 3
./build/cholesky_candidate_mpi 1024
```

Las dos últimas líneas representan un solo comando. La salida CSV informa tiempos, residuos y validez. L queda completo en memoria del rango 0; no se escribe un archivo de matriz ni se incluye I/O de disco del factor. El triángulo inferior representa L; el superior no forma parte del factor.

Próxima pregunta

Si la integración es correcta, ¿cuántos procesos reducen el tiempo hasta obtener L y cuánto se pierde al medir el comando completo?

Comprobaciones y contrato del reloj

Comprobación	Resultado
Programa: N=1,2,7,31,128; secuencial y P=1,2,3,4,6	30 configuraciones válidas; incluye N menor que P y repartos desiguales.
Residuos	Ocho proyecciones reproducibles; residuo completo para N menor o igual a 128. Límite 1e-12 y control de valores finitos y diagonal positiva.
Biblioteca numérica propia	Pruebas existentes: factor conocido, identidad, variantes, entradas inválidas y detección de corrupción.
Interfaz	Argumentos inválidos rechazados. Errores de reserva abortan el trabajo MPI.

Los tamaños grandes usan proyecciones $O(n^2)$ por vector, no reconstrucción completa $O(n^3)$. Eso es evidencia numérica, no un certificado para cualquier entrada. Esta etapa no amplía la familia de matrices: conserva las SPD diagonalmente dominantes del proyecto.

$$T_{app} = T_{reserva} + T_{generacion} + T_{factor} + T_{reunion}$$

T_app es el intervalo medido en rango 0, desde la reserva hasta que reúne el resultado. También se registra la máxima duración local. Las fases de rango 0 suman su total, pero no son tiempos puros: por ejemplo, factor incluye comunicaciones y esperas. No sumamos máximos de fases de rangos distintos.

La comprobación regenera A en rango 0 y calcula residuos después de T_app. Es la misma operación matemática en ambas compilaciones. Se mide aparte y queda incluida en el reloj externo, junto con liberación, salida CSV y cierre. No se genera una referencia secuencial adicional en el MPI candidato.

Decisión y alcance

Consolidamos este ejecutable como candidato operativo; los compare_*.c quedan como evidencia histórica. El programa no elige P automáticamente. Tampoco sustituye el estudio de rendimiento en el clúster.

Trazabilidad: resultados/cierre_local_20260924/tests, build_tests.log y fuentes congeladas. Generación y medición en C; ejecución en Bash; análisis y documentación en Python. Desarrollo asistido por IA, con pruebas ejecutadas.

Etapa 12: speedup del candidato

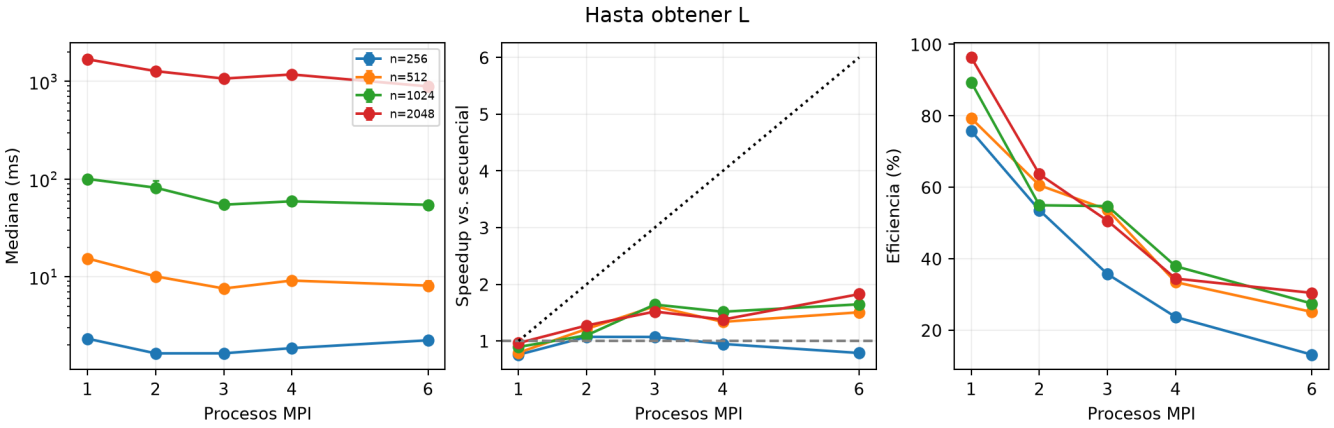
¿Cuánto ganamos hasta obtener L?

Medimos el programa elegido frente al secuencial autónomo contemporáneo, sin reutilizar tiempos anteriores. El objetivo es observar aceleración útil y eficiencia, no solamente comparar dos optimizaciones con igual P.

$$S_{app}(n,P) = \frac{\text{mediana}(T_{app,seq})}{\text{mediana}(T_{app,MPI,P})}, \qquad E_{app} = S_{app}/P$$

Protocolo: N=256,512,1024,2048; MPI P=1,2,3,4,6 y secuencial. Siete repeticiones por configuración, más una descartada; 168 observaciones medidas y 24 de preparación. Cada observación es un lanzamiento nuevo. Rotamos el orden de los seis métodos entre rondas; los tamaños siguen un orden fijo. No es una aleatorización completa.

WSL local, Ryzen 5 2600, seis núcleos físicos y doce procesadores lógicos. MPI solicita un proceso por núcleo con afinidad; secuencial usa taskset CPU 0. GCC -O3, double, sin BLAS. Una sola sesión: no inferimos estabilidad entre días ni entre nodos.



Puntos: medianas. Barras en tiempos: Q1-Q3 inclusivos. Los cuartiles describen dispersión, no intervalos de confianza. Las curvas de speedup son cocientes de medianas, sin incertidumbre inferencial calculada.

Lectura de la figura

S mayor que 1 significa que MPI reduce el tiempo frente a nuestro secuencial; E muestra la fracción del ideal lineal respecto de esa referencia. MPI con P=1 no tiene por qué coincidir con el secuencial: cambia almacenamiento y agrega coordinación y reunión.

Resultados y decisión local

N	P	MPI ms	S app	E app
256	1	2.32	0.76	75.7%
256	2	1.64	1.07	53.6%
256	3	1.64	1.07	35.7%
256	4	1.86	0.95	23.7%
256	6	2.23	0.79	13.1%
512	1	15.41	0.79	79.3%
512	2	10.09	1.21	60.6%
512	3	7.58	1.61	53.7%
512	4	9.14	1.34	33.4%
512	6	8.12	1.51	25.1%
1024	1	100.43	0.89	89.4%
1024	2	81.68	1.10	55.0%
1024	3	54.69	1.64	54.7%
1024	4	59.25	1.52	37.9%
1024	6	54.54	1.65	27.4%
2048	1	1686.75	0.96	96.3%
2048	2	1276.59	1.27	63.6%
2048	3	1070.06	1.52	50.6%
2048	4	1180.06	1.38	34.4%
2048	6	890.19	1.83	30.4%

Referencia secuencial (ms): N=256: 1.76; N=512: 12.22; N=1024: 89.78; N=2048: 1624.85.

En N=2048, el menor tiempo MPI observado fue con P=6: 890.19 ms y S=1.83. Es el mínimo de esta muestra, no una regla automática para elegir procesos.

El CSV también conserva T_MPI,1/T_MPI,P como escalabilidad interna. No la confundimos con aceleración frente al secuencial. La siguiente pregunta cambia el reloj: ¿esa ganancia sobrevive al arranque, comprobación y cierre?

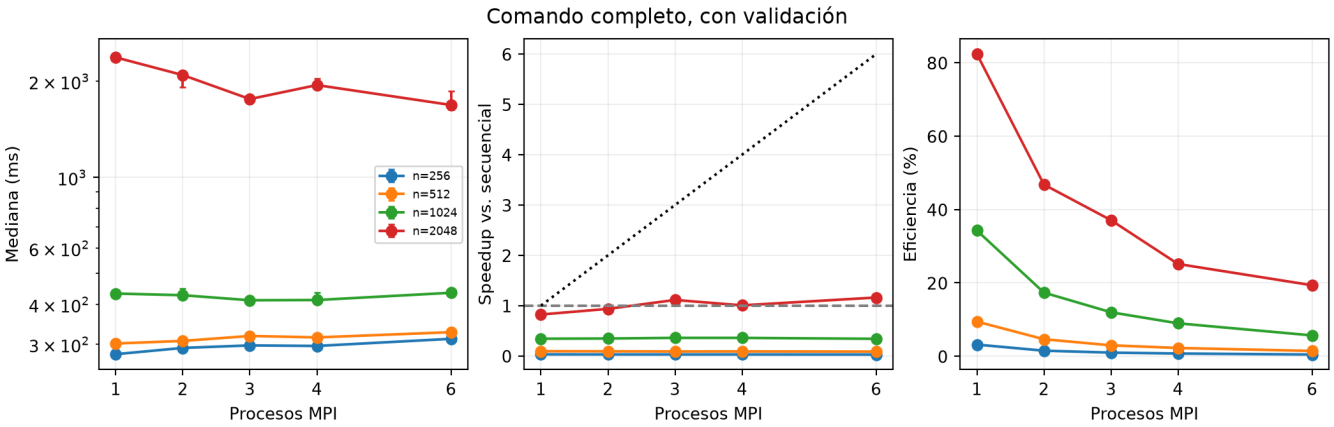
Etapa 13: el comando completo

¿El usuario termina antes?

El ejecutable measure_command, escrito en C, mide con reloj monotónico desde antes de fork hasta que termina el comando hijo. Para MPI el hijo es mpirun; para el secuencial, taskset. La escritura del CSV del cronómetro queda fuera. El arranque de WSL y la cola de un planificador no se miden.

$$S_{ext}(n,P) = \frac{\text{mediana}(T_{ext,seq})}{\text{mediana}(T_{ext,MPI,P})}$$

T_ext incluye lanzamiento, reserva, generación, factorización, reunión, regeneración para validar, residuos, salida CSV, liberación y cierre. No contiene impresión o escritura del factor. Las mismas 168 observaciones proporcionan ambos relojes: no hay que sumar medianas ni unir campañas.



N	Seq ext ms	MPI P6 ext ms	S ext P6
256	8.89	312.08	0.03
512	28.53	327.38	0.09
1024	148.30	434.99	0.34
2048	1957.48	1685.41	1.16

Qué podemos atribuir y qué no

external - application - validation se calcula por ejecución y después se resume. No es tiempo puro de arranque ni tiempo puro de red: combina inicialización, barreras, reducciones de medición, cierre, liberación y salida. Aislar cada causa necesitaría otro experimento.

Cierre de la investigación local

Respuesta a la pregunta completa

Para N=2048, el menor tiempo externo MPI de la muestra aparece con P=6: 1685.41 ms, frente a 1957.48 ms secuenciales; S externo=1.16. El veredicto del reloj externo puede diferir del tiempo hasta obtener L.

La validación se hace en rango 0 mientras los demás esperan una difusión del estado final. Su duración puede variar entre ejecuciones MPI y secuenciales, aunque el cálculo sea el mismo. Contención, esperas activas y condiciones del equipo son hipótesis; esta campaña no aísla sus aportes.

Corrección y evidencia

Se verificaron 192 lanzamientos de campaña sin fallo (incluidos los descartados para rendimiento). Residuo proyectado máximo: 1.611e-15. El analizador exige tiempos finitos, salidas válidas, cantidad esperada de muestras y coherencia entre reloj externo y fases. Los 30 casos pequeños son adicionales.

Qué queda decidido

Conservamos la implementación 1D right-looking cíclica b=1 como candidata. La campaña no compara nuevos algoritmos y no justifica cambiarla por el menor tiempo aislado. Conservamos como resultados principales ambos relojes, speedup, eficiencia y dispersión. No afirmamos que seis procesos sean siempre mejores.

Hasta dónde llegamos y qué dejamos abierto

Cerramos el alcance local acordado: programa integrado, comprobación, campaña con 1-6 procesos y medición externa. No agotamos toda optimización imaginable. Se mantienen como extensiones: validar otras familias SPD, estudiar las esperas durante validación, Cholesky por bloques, OpenMP, 2D y entrada/salida real de matrices.

En el clúster: compilar el mismo candidato, verificar afinidad y memoria con el administrador, aumentar N con pilotos seguros, repetir dentro de un nodo y entre nodos, y medir una referencia secuencial del mismo entorno. La cola se informa aparte. Ningún tamaño ni número de procesos local se extrapola automáticamente.

Reproducibilidad y siguiente pregunta

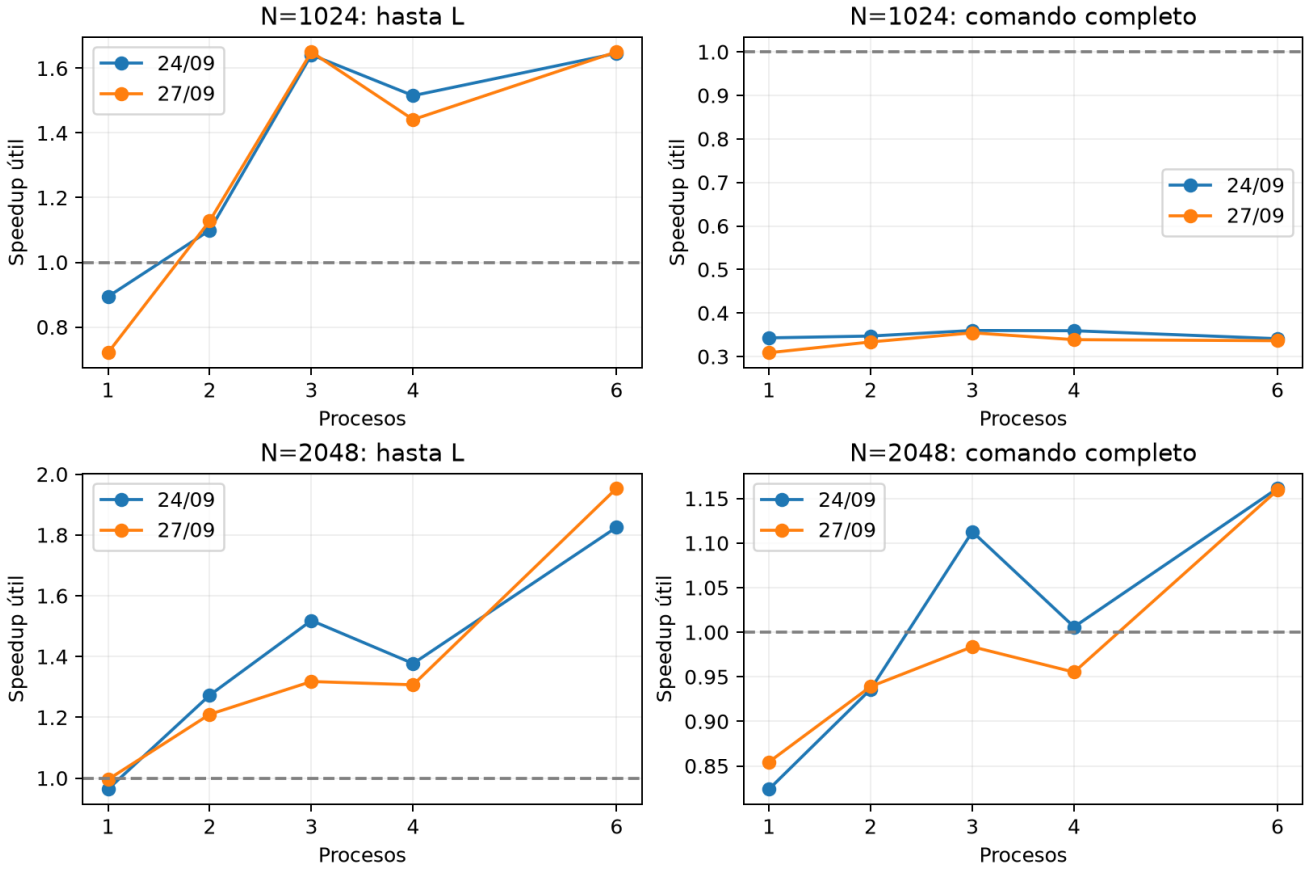
scripts/cierre_local.sh ejecuta; scripts/analizar_cierre.py analiza; resultados/cierre_local_20260924 conserva CSV, entorno y fuentes. ¿Cómo cambian las ganancias al crecer N y pasar de comunicación local a comunicación entre nodos? Esa será la siguiente etapa de investigación.

Etapa 14: repetir en otra sesión

¿Se mantiene la conclusión local?

El 24/09 el candidato logró S=1,83 hasta obtener L y S=1,16 para el comando completo en N=2048, P=6. La segunda ventaja era modesta. Repetimos el 27/09 para comprobar sensibilidad a otra sesión, sin buscar un resultado favorable ni modificar el algoritmo.

Mismo código y script, fuentes congeladas idénticas byte por byte. N=256,512,1024,2048; secuencial autónomo y MPI P=1,2,3,4,6. Siete observaciones por configuración y una descartada: 192 lanzamientos, 168 medidos. Orden rotado por ronda y mismos dos relojes. Conservamos datos separados y calculamos cada speedup con el secuencial de su propia sesión.



Cocientes de medianas por sesión, sin bandas de confianza. Los cuartiles de los tiempos están en resumen.csv de cada campaña. Dos sesiones permiten contrastar tendencias, pero no establecen estabilidad universal ni equivalen a 14 repeticiones bajo las mismas condiciones.

N, P=6	S app 24/09	S app 27/09	S ext 24/09	S ext 27/09
256	0.79	0.75	0.03	0.03
512	1.51	1.40	0.09	0.08
1024	1.65	1.65	0.34	0.34
2048	1.83	1.95	1.16	1.16

Interpretación y versión de referencia

Resultado de la nueva sesión

N=2048/P=6: hasta L, 1770.61 ms secuenciales frente a 906.42 ms MPI (S=1.95; eficiencia 32.6%). Comando completo: 2095.30 frente a 1807.18 ms (S=1.16; eficiencia 19.3%).

La mediana externa con seis procesos vuelve a mejorar frente al secuencial en N=2048. La dirección se repite, pero la magnitud debe citarse por sesión: no declaramos una aceleración fija ni un umbral universal.

N=1024: el menor tiempo externo MPI observado en esta sesión fue con P=3, 409.99 ms (S=0.35). Elegir el mínimo entre configuraciones es descriptivo; no prueba que esa cantidad sea siempre la mejor.

N=2048: el menor tiempo externo MPI observado en esta sesión fue con P=6, 1807.18 ms (S=1.16). Elegir el mínimo entre configuraciones es descriptivo; no prueba que esa cantidad sea siempre la mejor.

Corrección y límites

No todas las configuraciones repiten su ventaja: N=2048/P=3 pasa de S externo=1.11 a 0.98. Es un ejemplo de por qué una mejora pequeña en una sesión no basta para recomendar una cantidad de procesos.

Las 192 ejecuciones de campaña pasaron validación; residuo proyectado máximo 1.611e-15. También pasaron los 30 casos pequeños del programa y las pruebas numéricas previas. El análisis revisó fases, estados de salida, muestras y tiempos finitos. Matrices grandes: ocho proyecciones; pequeñas: además residuo completo.

No aislamos frecuencia, temperatura, carga del sistema ni espera activa. No atribuimos las variaciones a una causa concreta. El reloj externo incluye validación y arranque/cierre; no mide solo comunicaciones. Todo ocurre en WSL local, sin comunicación entre nodos.

Qué significa congelar el candidato

Fijamos en Git una referencia identificable del programa que medimos. No significa dejar de trabajar ni prohibir correcciones. Podemos preparar defensa, documentación, scripts y configuración del clúster. Un cambio que afecte cálculo o medición crea una versión nueva y se valida antes de comparar. Los datos anteriores conservan sus fuentes originales.

La referencia sigue siendo right-looking MPI 1D cíclico b=1, vector local y generación distribuida. Con esta repetición se completa el control local acordado. El paso siguiente es adaptar la ejecución a los recursos del clúster y repetir allí con referencias contemporáneas, tamaños mayores y separación entre uno y varios nodos.

Trazabilidad: resultados/cierre_local_20260927, comparacion_sesiones.csv y comparacion_fuentes.json. C mide; Python analiza y documenta. Trabajo asistido por IA. No se descartan resultados desfavorables.

V

PARTE V

Campañas en el clúster y comparación final

¿Qué mejora sobrevive en el clúster y cuánto rendimiento local queda sin explotar?

Separador editorial; las páginas siguientes conservan su contenido original.

Etapa 15: primeros resultados del clúster

Cholesky en C y MPI | Experimentos del 28/09/2026. Edición revisada con archivos originales del clúster. Continuación de las catorce etapas locales. Estado: exploración completada; campaña formal de cruce no incluida en la evidencia recibida.

Pregunta y respuesta provisional

¿El candidato conserva corrección y obtiene ganancias en el nuevo entorno? Las ejecuciones documentadas pasan validación. El control repetido de N=4096 muestra un cociente de medianas de aproximadamente 2,50 frente al secuencial. El escaneo sugiere un cruce entre N=128 y N=256 y un comportamiento no monótono con P. Todavía no hay una curva de escalabilidad confirmada con repeticiones para cada configuración.

Qué evidencia recibimos

Conjunto	Ejecuciones descritas	Uso
Validación N=128	2	Corrección secuencial y MPI P6
Piloto N=512..4096	8	Elegir tamaños
Control N=4096	6 (3 por método)	Dispersión dentro de la sesión
Escaneo mayor y pequeño	24 + 12	Localizar preguntas
Diagnóstico de cuota	1	Comprobar throttling
Campaña formal de cruce	0; 240 previstas	Plan, sin resultados todavía

Se recibieron 53 CSV originales, sus 53 archivos de error, dos logs de compilación, cinco fuentes y un manifiesto SHA-256 en evidencia_preformal_20260928.tar.gz. Los 113 hashes del manifiesto coinciden. Los gráficos y las estadísticas de esta edición se recalculan desde los CSV originales, manteniendo separados los conjuntos. Las 50 observaciones de las tablas anteriores coinciden salvo redondeo; las conclusiones se mantienen.

Continuidad del proyecto

Conservamos right-looking, vector local, reparto cíclico 1D b=1, generación distribuida y L en rango 0. Los cinco SHA-256 de la bitácora coinciden con las fuentes actuales, cuya copia queda archivada con este análisis. No se cambiaron los algoritmos. Las decisiones previas explican el candidato; los tiempos de WSL no se usan para calcular ganancias en el clúster.

Entorno, corrección y significado de los relojes

La sesión declara AMD EPYC 7B12, GCC 13.3 y Open MPI 4.1.6. Ve 128 CPUs y dos nodos NUMA. cpu.max=600000 100000 limita la capacidad agregada a seis CPUs equivalentes; memory.max=25769803776 corresponde a 24 GiB. Esto no acredita exclusividad sobre seis núcleos.

Pregunta: ¿se preservó la corrección?

N=128	Secuencial	MPI P=6
Residuo proyectado	3,8964e-16	3,8964e-16
Residuo completo	3,7903e-16	3,7903e-16
valid	1	1

El máximo residuo proyectado informado en los tamaños explorados es 2,274918e-15, unas 440 veces menor que 1e-12. Para N>128, exact_residual=-1 indica que no se calculó el residuo completo. Ocho proyecciones son evidencia numérica, no una prueba exhaustiva. Residuos iguales no demuestran que los factores sean idénticos elemento a elemento. El alcance es la familia SPD diagonalmente dominante generada con semilla 2026.

Contrato comprobado en main_candidate.c

Campo	Interpretación
application_s	Rango 0: reserva + generación + factor + reunión/reconstrucción. Excluye validación y arranque/cierre MPI.
factor_s	Tiempo de la fase de factor en rango 0; incluye MPI y esperas, no solo operaciones aritméticas.
gather_s	MPI_Gatherv y reconstrucción serial de L en rango 0; no es tiempo puro de red.
max_local_s	Máximo del intervalo local completo entre rangos; no son máximos por fase.
validation_s	Comprobación posterior al tiempo de aplicación. No sustituye una medición externa.

Las fases consecutivas suman el total en cada ejecución. Debe resumirse application_s directamente: sumar medianas de fases no reconstruye necesariamente su mediana. Se verificó la suma en los 53 CSV originales dentro de su precisión impresa, además de tiempos finitos y no negativos, valid=1 y residuos dentro del criterio.

Conclusión y próxima pregunta

El traslado tiene evidencia de corrección para las entradas ensayadas. Falta medir de manera repetida cuánto cuesta producir L bajo una afinidad controlada y cuánto tarda el comando completo.

Escaneo mayor: dónde aparecen las preguntas

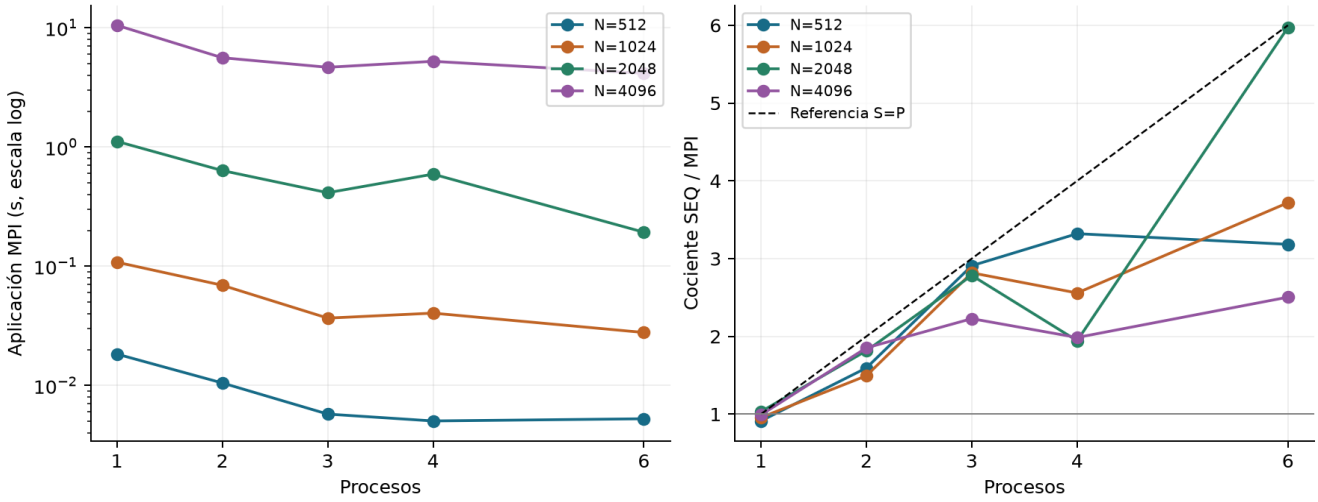


Figura 1. Sección 14 de la bitácora: una observación por configuración, sin barras de dispersión. Las líneas conectan puntos medidos; no describen tamaños ni procesos intermedios. El secuencial de este escaneo no tenía afinidad fijada.

N	SEQ (s)	P del mínimo MPI	MPI (s)	SEQ/MPI
512	0.016721	4	0.005035	3.32
1024	0.103386	6	0.027800	3.72
2048	1.151211	6	0.192791	5.97
4096	10.390189	6	4.148838	2.50

Observaciones que merecen confirmación

Para N=2048, P=4 tarda 43,1% más que P=3; pasar de P=4 a P=6 reduce el tiempo 67,4%. En N=4096, P=4 también empeora frente a P=3. P=6 produce el menor tiempo observado en tres de los cuatro tamaños, mientras N=512 favorece P=4. Son mínimos seleccionados de observaciones únicas, no recomendaciones definitivas.

Al duplicar N de 2048 a 4096, el total secuencial crece 9,03 veces y MPI P=6 crece 21,52 veces dentro de este escaneo. El total contiene varias fases: no debe exigirse un factor ocho exacto. La diferencia justifica estudiar ambos tamaños; no demuestra por sí misma un problema de caché, NUMA, comunicación o cuota.

Cómo leer el resultado cercano a seis

N=2048/P=6 da 5,97 en el escaneo y 6,15 en el piloto separado. Se conservan ambos sin combinarlos. El cociente es aceleración frente al secuencial propio, que usa otra ruta de almacenamiento; no prueba escalabilidad lineal o superlineal sostenida. Comparar MPI P1/MPIP responde una pregunta distinta y también deberá informarse.

N=4096: qué dicen las tres repeticiones

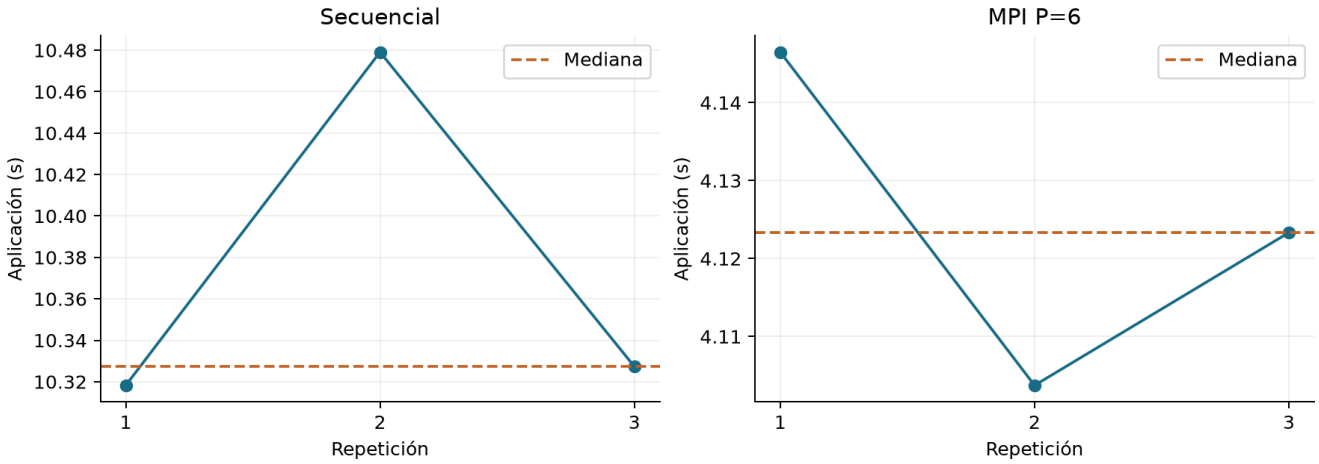


Figura 2. Tres pares consecutivos de la sección 13, con SEQ antes de MPI en todos ellos. Paneles con escalas verticales distintas para mostrar variación. Son repeticiones de una misma sesión, con el secuencial sin afinidad explícita.

Estadística (s)	SEQ	MPI P=6
Mediana	10.327355	4.123326
Mínimo	10.318244	4.103728
Máximo	10.479087	4.146479
Q1 inclusivo	10.322799	4.113527
Q3 inclusivo	10.403221	4.134902
Desvío muestral	0.090348	0.021400

Cociente de medianas: 2.5046. Reducción relativa del tiempo: 60.07%. Eficiencia descriptiva S/6: 41.74%. Coeficiente de variación: SEQ 0.87% y MPI 0.52%. Q1 y Q3 se calculan con interpolación inclusiva; con tres datos su utilidad es descriptiva, sin inferencia de significación.

Conclusión respaldada

El tiempo MPI cercano a 4,12 s reaparece en los tres pares. No parece un valor aislado del primer piloto. La ganancia alrededor de 2,50 se sostiene en este control, pero no acredita estabilidad entre sesiones ni una configuración óptima. No se añaden el piloto, el escaneo o el diagnóstico de cuota a esta muestra para aumentar artificialmente n.

Siguiente pregunta

¿Se mantienen la ganancia y la no monotonía al fijar también el secuencial, rotar métodos y repetir P=1,2,3,4,6? Esa prueba es más informativa que aumentar ahora el número de variantes algorítmicas.

Fases, afinidad y cuota: evidencia y límites

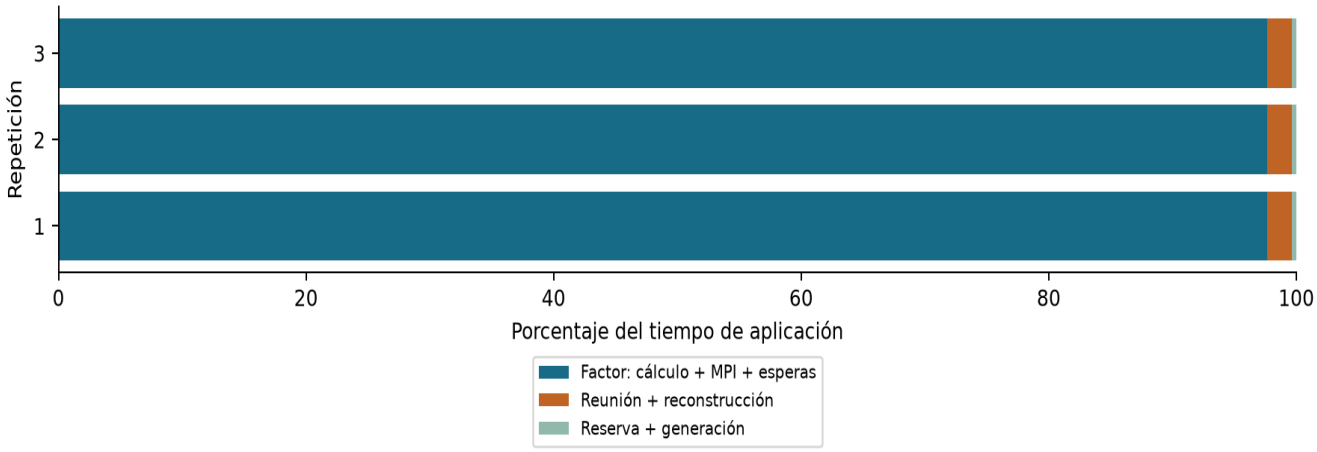


Figura 3. Porcentajes calculados por ejecución desde los CSV originales de los tres controles MPI; el resto se obtiene como application-factor-gather y corresponde a reserva más generación.

Mediana de la fracción de factor: 97.64%; reunión/reconstrucción: 1.99%. El mayor tiempo está dentro del kernel MPI, pero esta medición no separa cálculo, difusión y espera. Optimizar únicamente la reunión tendría un margen cercano al 2% del total en estos casos, aun suponiendo eliminarla por completo.

Afinidad observada, no exclusividad

El diagnóstico con /bin/true y P=6 informó seis cores distintos del socket 0. Es evidencia del mapeo de ese lanzamiento. No demuestra retrospectivamente el binding de cada benchmark ni que P=4 use exactamente la misma colocación. Compartir socket tampoco demuestra ubicación efectiva de páginas de memoria. Conviene guardar bindings por P en la campaña formal.

Throttling comprobado, causalidad abierta

Contador	Incremento
usage_usec	37.260.596 (37,261 CPU-s agregados)
nr_periods	66
nr_throttled	28 (42,42% de los períodos observados)
throttled_usec	119.281 (0,119281 s del contador)

Los contadores cubren el intervalo entre dos lecturas del cgroup, incluidos lanzamiento, validación y procesos auxiliares; application_s=4,127776 s solo cubre la aplicación en rango 0. El 42,42% NO es porcentaje de tiempo perdido. Tampoco corresponde restar 0,119281 s al factor ni dividir los CPU-s por application_s para inferir utilización. No hay un control equivalente sin cuota que permita atribuirle la pérdida de rendimiento.

Decisión

Mantener P<=6 y registrar cuota/contadores sin modificar el algoritmo. La evidencia todavía no permite escoger entre cuota, interferencia, localidad y colectivas como causa de la no monotonía.

Cruce pequeño: hipótesis y campaña pendiente

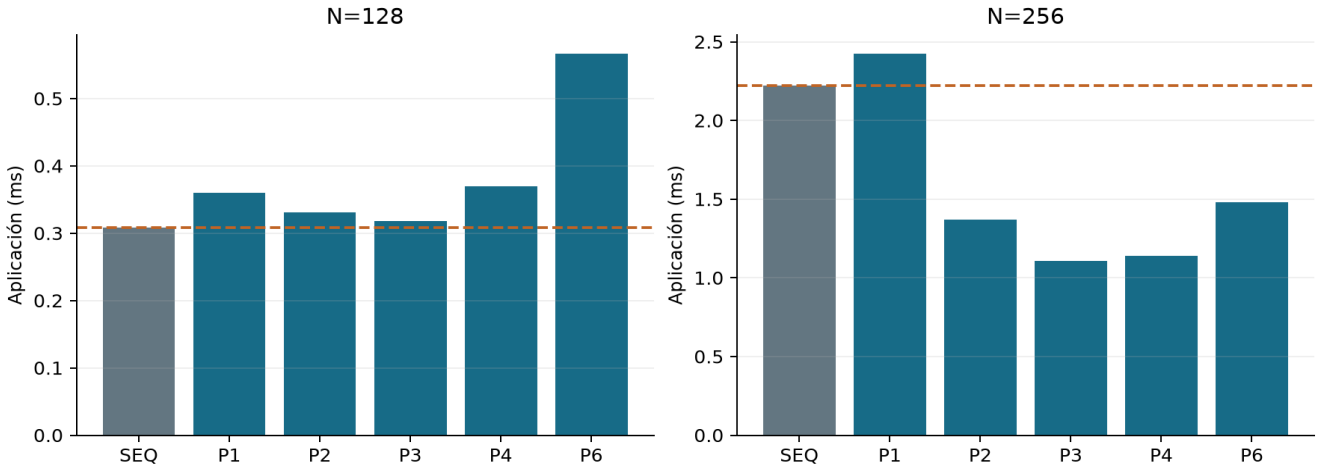


Figura 4. Sección 17: una observación por método, con SEQ fijado a CPU0. No se combina con el escaneo mayor ni con las validaciones anteriores.

Qué se puede concluir ahora

En N=128 todos los tiempos MPI superan el secuencial de esa prueba. En N=256, P=2,3,4,6 mejoran; el menor tiempo observado es P=3, con cociente 2,01. Esto localiza una región de interés entre dos tamaños ensayados. No demuestra que exista un único umbral ni que todos los tamaños posteriores mejoren.

Protocolo que ya quedó definido

N=128,160,192,224,256; SEQ y MPI P=1,2,3,4,6; una ronda descartada y siete rondas medidas. Son 30 configuraciones, 240 lanzamientos y 210 observaciones útiles. Se rota el orden de métodos; el orden de tamaños permanece fijo. Siete rondas no equilibran exactamente las seis posiciones posibles. Cada observación abre un proceso nuevo: la ronda descartada no convierte al programa en un benchmark de runtime persistente.

Métrica primaria: application_s. S=mediana(SEQ)/mediana(MPI); E=S/P. Mejora clara descriptiva si S>1,10 y Q3(MPI)<Q1(SEQ). Fijar Q1/Q3 inclusivos antes del análisis. No es un test estadístico, y comparar muchas configuraciones hace especialmente necesaria una confirmación posterior de los puntos elegidos.

Ajustes operativos antes de ejecutarla

Usar una carpeta nueva y fallar si ya existe: mkdir -p y tee sobre nombres repetidos pueden sobrescribir una campaña. Guardar fuentes/hashes, entorno, bindings, errores y código de salida. Mantener una sola campaña activa. Si ya se ejecutó después de escribir la bitácora, conservarla como fue y analizarla antes de decidir si requiere repetición.

Pregunta que seguirá abierta

El cruce de application_s no es el cruce del comando completo. Un MPI rápido dentro del programa puede seguir siendo más lento cuando se incluyen arranque, validación y cierre. Falta medir ese segundo reloj en el clúster.

Conclusión de etapa y camino al cierre

Mi evaluación

El avance es productivo: se confirmó el candidato, se identificó el entorno real y los pilotos descubrieron preguntas que una campaña automática podría haber ocultado. Ya hay evidencia suficiente para fijar las siguientes mediciones. Seguir acumulando ejecuciones aisladas aportaría menos que completar una grilla repetida y una confirmación en otra sesión.

Orden recomendado

Paso	Pregunta y alcance
1. Evidencia resguardada	Completado: paquete original archivado; CSV, archivos de error, logs y fuentes contrastados. No hace falta repetir los pilotos para recuperar esta evidencia.
2. Completar el cruce	Ejecutar o recuperar la campaña N=128..256 fijada. Reportar medianas, Q1-Q3 y criterio descriptivo por N y P.
3. Escalabilidad mayor	N=512,1024,2048,4096; mismos seis métodos; siete repeticiones y una ronda descartada: 192 lanzamientos, 168 útiles. Afinidad controlada y orden rotado.
4. Segundo reloj	Medir aplicación y comando externo en los mismos lanzamientos, tras verificar el cronómetro. Priorizar esta incorporación para la campaña mayor; conservar validación aparte.
5. Confirmar y cerrar	Otra sesión para el cruce y configuraciones decisivas, especialmente N=2048/4096 con P=3,4,6 y SEQ. Explicar límites sin exigir resolver cada causa.

La grilla mayor es una propuesta para el siguiente bloque, no un resultado ni una orden de ejecución inmediata. Si el costo es alto, hacer primero N=2048 y N=4096 y completar después los otros tamaños. Comparar sesiones por separado y medir el secuencial en cada una. No hace falta ampliar a 48/96 cores para responder el alcance actual.

Conclusión que hoy puede defenderse

En la sesión con cuota de seis CPUs equivalentes y 24 GiB, el candidato conserva la corrección reportada y muestra ganancias del tiempo hasta L en varios casos explorados. Para N=4096, tres repeticiones por método dan 10,327 s secuenciales y 4,123 s MPI P6 de mediana, con cociente 2,50. No se ha establecido aún el cruce reproducible, la curva completa de escalabilidad ni la ganancia del comando externo. Los resultados corresponden al entorno de un host; no acreditan ejecución entre nodos.

No es necesario que MPI gane siempre para cerrar el proyecto. El cierre requiere describir dónde gana, dónde deja de convenir y qué costo agrega el flujo completo, con datos repetidos y límites explícitos. Bloques, BLAS, OpenMP y distribución 2D quedan como extensiones justificables después de esa respuesta.

Trazabilidad y precisiones de la revisión

Qué se verificó y qué permanece declarado

Se auditaron los 53 CSV originales y los 113 hashes del paquete. Las cinco fuentes coinciden byte por byte con las copias archivadas. Los 53 archivos .err y ambos logs de compilación están vacíos: no contienen mensajes capturados, pero no sustituyen códigos de salida explícitos. No se ejecutaron nuevos benchmarks. Los diagnósticos de afinidad y cpu.stat siguen procediendo de la bitácora: el paquete no incluye esos logs completos.

Precisiones frente a interpretaciones anteriores

1. Coincidencia de residuos no equivale a igualdad de factores. 2. El binding observado para P=6 no descarta de forma general efectos NUMA ni acredita P=4. 3. La repetibilidad en una sesión no demuestra estabilidad entre fechas. 4. La fracción dominante de factor no equivale a cómputo puro. 5. La cuota de seis CPUs no es una reserva exclusiva. 6. El throttling observado no cuantifica la pérdida de tiempo de Cholesky.

Archivos conservados con esta etapa

Archivo o carpeta	Contenido
evidencia_preformal_20260928.tar.gz	Paquete original íntegro del clúster.
bitacora_original.md	Contexto y protocolo experimental.
fuentes/ y auditoria.json	Fuentes, hashes y comprobaciones actualizadas.
observaciones_originales.csv	53 filas extraídas del paquete original.
observaciones_tablas.csv	Reconstrucción anterior conservada para contraste.
cocientes_escaneo.csv	Cocientes recalculados con los originales.
estadisticas_control.csv	Mediana, cuartiles, desvío y CV del control.
fracciones_control.csv	Porcentajes por ejecución desde originales.
throttling_transcrito.csv	Contadores de la bitácora; diferencias verificadas.
informe.md y cuatro PNG	Análisis y gráficos de esta edición revisada.

Generador: desde_cero/docs/generar_informe_cluster_exploratorio.py. Verifica el paquete, contrasta la bitácora y recalcula desde los originales. Esta revisión reemplaza la edición que indicaba ausencia de CSV y logs. No añade repeticiones ni convierte los pilotos en una campaña formal. Análisis y documentación realizados con asistencia de IA.

Etapa 16: campaña formal de cruce

Clúster | 28/09/2026 | Candidato Cholesky right-looking MPI cíclico 1D. Continuación de la exploración de la Etapa 15. Análisis independiente del paquete original entregado por el usuario; los resultados pegados en el mensaje se contrastaron con los CSV.

Pregunta y resultado

¿En qué tamaños ensayados MPI reduce claramente el tiempo hasta disponer de L frente al secuencial propio? Bajo el criterio prefijado, P=3 presenta la primera mejora en N=192. P=4 la presenta en N=224; P=2 y P=6, en N=256. MPI P=1 no cumple el criterio en la grilla. Son cruces descriptivos de una sesión, no umbrales universales.

Control de evidencia	Resultado
Cantidad de muestras	210 útiles y 30 warmups; siete útiles por cada una de 30 configuraciones.
Integridad	494 archivos regulares; 493 hashes del manifiesto general verificados. El manifiesto no se incluye a sí mismo.
Errores y compilación	240 archivos .err y dos logs de compilación vacíos. No hay códigos de salida explícitos archivados.
Corrección numérica	240 filas con valid=1. Residuo proyectado máximo útil: 5.764e-16.
Fuentes	Cinco fuentes idénticas a las de la Etapa 15. Se preservan también los dos ejecutables originales sin ejecutarlos.

Protocolo

N=128,160,192,224,256; SEQ y MPI P=1,2,3,4,6. Una ronda descartada y siete medidas. Orden de métodos rotado; tamaños en orden creciente. Afinidad prevista: SEQ con taskset CPU0; MPI --bind-to core --map-by core. El entorno conserva un diagnóstico de binding P6 en el socket 0, no un registro independiente de cada lanzamiento.

AMD EPYC 7B12, GCC 13.3, Open MPI 4.1.6, cuota de seis CPUs equivalentes y 24 GiB. Fuente de entorno fechada 2026-09-28T16:06:20Z. No se interpreta la cuota como seis cores exclusivos ni como acceso a varios nodos.

Resultado del criterio fijado antes de medir

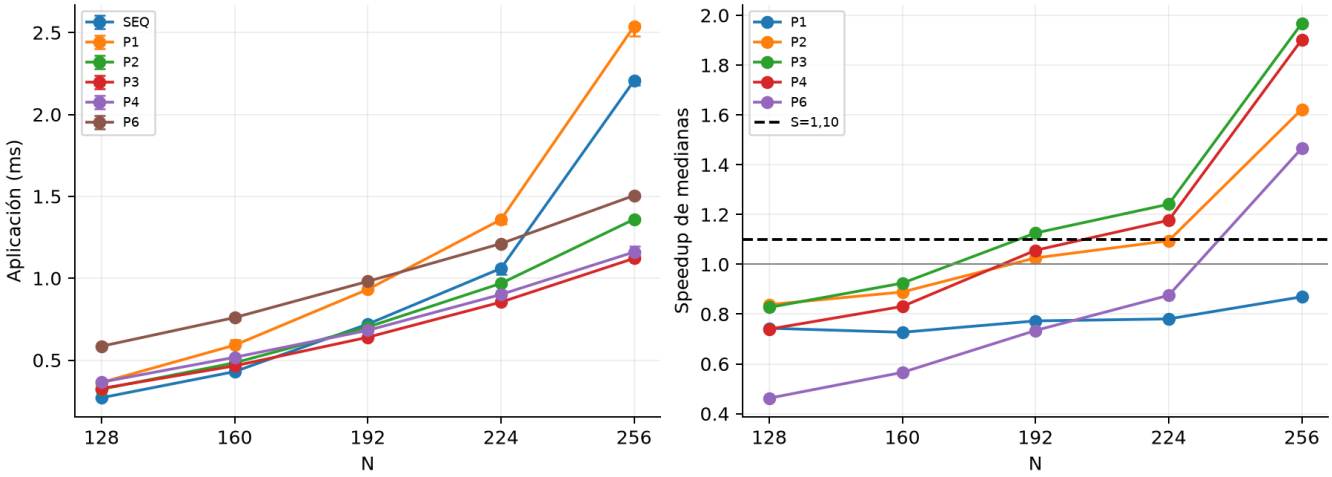


Figura 1. Medianas y barras Q1-Q3 inclusivos en tiempos, siete observaciones por punto. Los speedups son cocientes de medianas, sin intervalos de confianza. Las líneas unen únicamente los puntos medidos. Warmups y pilotos quedan excluidos.

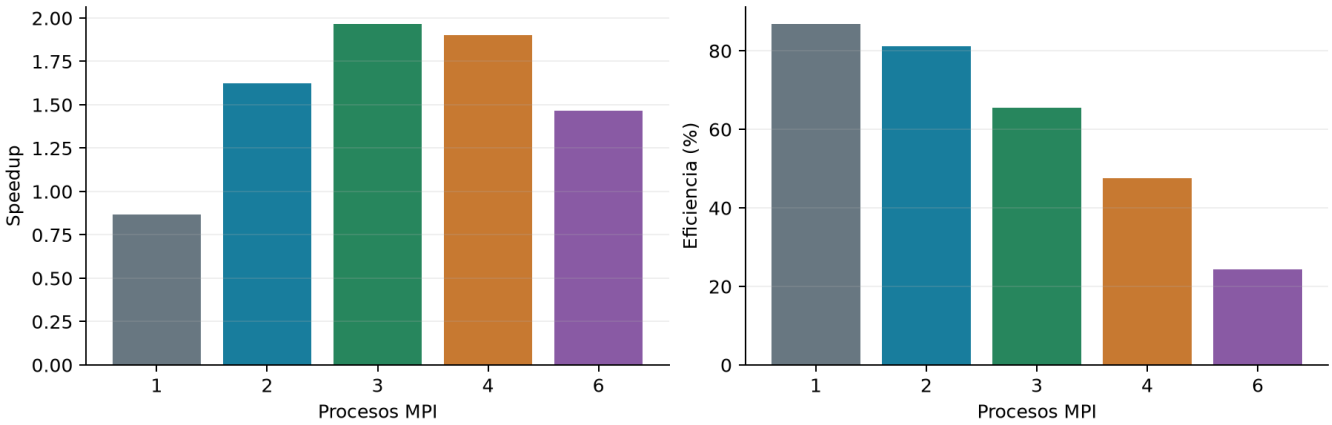
S=mediana(T_SEQ)/mediana(T_MPI). E=S/P. Mejora clara descriptiva si S>1,10 y Q3_MPI<Q1_SEQ, ambas desigualdades estrictas. Cuartiles por interpolación inclusiva. No es una prueba formal de significación y no demuestra monotonicidad para tamaños no ensayados.

N	P1	P2	P3	P4	P6
128	No	No	No	No	No
160	No	No	No	No	No
192	No	No	Sí	No	No
224	No	No	Sí	Sí	No
256	No	Sí	Sí	Sí	Sí

Procesos	Primer N ensayado con mejora clara
1	Ninguno en la grilla
2	256
3	192
4	224
6	256

La campaña responde el cruce del reloj application_s para esta entrada, implementación y sesión. No responde aún si mpirun termina antes que el comando secuencial: el reloj excluye arranque, validación y cierre.

N=256: rapidez y eficiencia son preguntas distintas



Método	Q1 (ms)	Mediana (ms)	Q3 (ms)	S	E (%)
SEQ	2.177	2.206	2.219	-	-
P1	2.480	2.538	2.556	0.87	86.9
P2	1.352	1.360	1.374	1.62	81.1
P3	1.112	1.121	1.147	1.97	65.6
P4	1.154	1.160	1.194	1.90	47.6
P6	1.493	1.505	1.508	1.47	24.4

P=3 tiene la menor mediana MPI observada. P=2 obtiene la mayor eficiencia entre las configuraciones que aceleran (P>=2). P=1 da 86,9% de eficiencia numérica, pero S=0,87: es más lento que SEQ. P=6 mejora frente a SEQ con eficiencia 24,4%. Son descripciones de esta campaña; no prueban que P=3 sea siempre óptimo ni que su diferencia frente a P=4 sea estadísticamente concluyente.

Comparación justa

La referencia es el secuencial autónomo medido aquí. MPI P1 no ejecuta exactamente la misma ruta: utiliza almacenamiento distribuido y reunión incluso con un proceso. Por eso se conserva también MPI1/MPIP en resumen.csv como escalabilidad interna, sin sustituir el speedup frente a SEQ.

Conclusión parcial

Aumentar procesos no asegura menor tiempo. En matrices pequeñas los costos adicionales compiten con el trabajo que se reparte; una configuración intermedia puede terminar antes que utilizar toda la cuota disponible.

Cómo se amortizan los costos adicionales

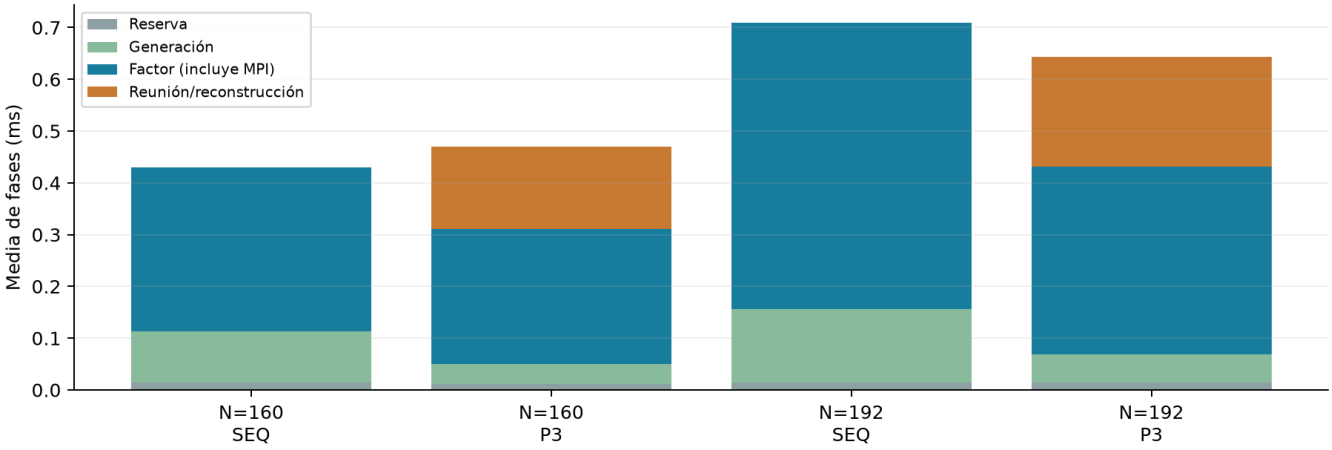


Figura 3. Medias de fases en las siete repeticiones, desde el reloj del rango 0. Aquí se usan medias para que las barras apiladas sumen la media total. El criterio principal sigue usando medianas del total medido, no la suma de medianas.

N / método	Generación ms	Factor ms	Reunión ms	Total ms
160 / SEQ	0.0989	0.3182	0.0000	0.4298
160 / P3	0.0382	0.2607	0.1544	0.4651
192 / SEQ	0.1425	0.5638	0.0000	0.7200
192 / P3	0.0535	0.3599	0.2088	0.6396

Tabla: medianas de cada campo por separado. En N=160/P3, generación y factor disminuyen respecto de SEQ, pero el costo de reunión/reconstrucción impide mejorar el total. En N=192/P3 el ahorro compensa esos costos y se cumplen las dos condiciones de mejora. Esto describe los intervalos medidos; no aísla causas de hardware.

factor_s incluye cálculo, colectivas y esperas. gather_s incluye MPI_Gatherv y reconstrucción serial. Las fases no son máximos por proceso. La comparación de las fases sirve para interpretar el cruce; no demuestra cuánto tiempo puro de red o de aritmética se consumió.

Cuota durante esta campaña

Los registros antes/después muestran incrementos de nr_throttled=0 y throttled_usec=0; nr_periods aumenta 941. No se registró throttling adicional en ese intervalo. Esto no demuestra ausencia de interferencia, migraciones o variación de frecuencia. El diagnóstico previo con N=4096 sigue siendo otro experimento y no se contradice con este resultado.

Auditoría de nombres y alcance de la evidencia

Qué se recuperó sin cambiar los originales

Los nombres contienen uno o varios prefijos rep seguidos del número de ronda. Se extrajeron ronda, N y método; se contrastaron N/P con el contenido. Cada configuración tiene exactamente rondas 1..7 y un warmup. No hay colisiones en los 240 nombres normalizados. El número de prefijos coincide con las posiciones 1..30 de cada ronda bajo el orden rotado previsto.

Esto respalda la recuperación inequívoca de la grilla y la coherencia del orden codificado. No reemplaza un registro temporal de los comandos: el script ejecutado no está incluido en el paquete. No se renombraron ni sobrescribieron archivos originales; observaciones_normalizadas.csv conserva el nombre original y el normalizado.

Causa compatible con el script documentado

La función de la bitácora asigna rep="$3" sin local, y el bucle la llama con "rep${rep}". La asignación modifica la variable exterior y acumula prefijos. El orden de métodos se calcula con start antes de esas llamadas; el bucle exterior asigna el siguiente número al comenzar otra ronda. Este mecanismo explica los nombres sin implicar pérdida de muestras.

Para el próximo script: declarar local n="$1", local method="$2" y local rep_tag="$3" dentro de run_case; usar rep_tag al formar nombres. Mantener una variable distinta para la ronda exterior. Usar una carpeta nueva, registrar estados de salida y probar primero la generación de nombres y órdenes sin lanzar benchmarks.

Integridad y trazabilidad

El manifiesto general verificó 493 entradas; el manifiesto de provenance verificó nueve. Son comprobaciones de integridad del contenido recibido, no certificaciones independientes de las condiciones de ejecución. Los logs vacíos indican ausencia de mensajes capturados. valid=1, los residuos y la coherencia de fases respaldan la validez numérica de los resultados archivados.

Archivos derivados: observaciones_normalizadas.csv (210 filas), warmups.csv (30), resumen.csv (30 configuraciones), fases.csv, primer_cruce.csv, cpu_stat_diferencias.csv y auditoria.json. El tar original queda archivado íntegro; no se ejecutan los binarios recibidos.

Conclusión y siguiente pregunta experimental

Qué queda respondido

Bajo el criterio descriptivo prefijado, la primera mejora del tiempo hasta L aparece en N=192 con P=3 entre los tamaños ensayados. Para N=256 mejoran P=2,3,4,6. Los datos formales confirman que los costos adicionales deben amortizarse antes de obtener una ganancia, y que seis procesos no son la opción más rápida en esta grilla.

Mi evaluación

Los cruces del resumen coinciden con el análisis independiente; la afirmación sobre eficiencia de P=2 requiere limitarse a las configuraciones que aceleran. La campaña es aprovechable pese al defecto de nombres: no hace falta repetirla por ese motivo. Tampoco hace falta explorar más tamaños pequeños ahora. Una confirmación selectiva en otra sesión puede reforzar la estabilidad alrededor del primer cruce, sin rehacer toda la grilla para avanzar.

Siguiente bloque, de forma gradual

Paso	Acción y propósito
1. Preparar el lanzamiento	Corregir variables locales y evitar sobrescritura. Guardar script, fuentes, compilación, binding y estados de salida.
2. Verificar dos relojes	Probar una configuración con el cronómetro externo existente y revisar sus dos salidas antes de lanzar la campaña. El tiempo externo incluye validación y runtime.
3. Medir escalabilidad	N=512,1024,2048,4096; SEQ y MPI P=1,2,3,4,6. Una ronda descartada y siete útiles: 192 lanzamientos, 168 útiles.
4. Confirmar y cerrar	Repetir en otra sesión los casos decisivos. Separar velocidad hasta L, tiempo externo y eficiencia; mantener registrados resultados desfavorables.

Mantener afinidad controlada, orden rotado, semilla 2026 y candidato sin cambios. Registrar cpu.stat antes/después de cada bloque; no convertir su delta directamente en tiempo perdido de Cholesky. Los dos relojes deben provenir del mismo lanzamiento. No lanzar aún toda la campaña sin verificar ese mecanismo.

La siguiente pregunta es cuánto escala el candidato cuando crece el trabajo y si la ganancia sobrevive al comando completo. No se necesita agregar BLAS, OpenMP o una distribución 2D para responderla. Esta etapa conserva alcance de una sesión en un host bajo cuota; no establece resultados multinodo.

Reproducción: desde_cero/docs/generar_informe_cruce_cluster.py. Esta Etapa 16 continúa la 15; sus frases históricas sobre campaña pendiente corresponden al estado anterior. Estadísticas recalculadas desde el paquete original. Documentación y análisis con asistencia de IA.

Etapa 17: escalabilidad y comando completo

Pregunta: cuando el trabajo aumenta, ¿cómo cambia el rendimiento al pasar de uno a seis procesos y cuánto de la mejora sobrevive al medir el comando completo? Campaña del 28/09/2026, posterior al cruce formal. Análisis desde el paquete original, sin incorporar pilotos ni muestras de otras sesiones.

Protocolo y evidencia

Elemento	Verificación
Grilla	N=512,1024,2048,4096; SEQ y MPI P=1,2,3,4,6.
Muestras	192 lanzamientos: 24 warmups y 168 útiles. Siete útiles por configuración. Cada interno tiene su externo y archivo .err.
Integridad y código	592 archivos; 12 hashes de provenance coinciden. Las cinco fuentes candidatas coinciden con etapas previas. No hay manifiesto global de las mediciones.
Validez	192 valid=1 y exit_code=0; residuos aceptables; fases coherentes; errores y logs de compilación vacíos.
Entorno	EPYC 7B12, GCC 13.3, Open MPI 4.1.6; cuota 6 CPUs y 24 GiB. Entorno fechado 16:40:52 UTC.

Dos relojes del mismo lanzamiento

application_s mide desde reserva hasta L completo en rango 0, con generación, factor y reunión. external_s proviene de measure_command.c: reloj monotónico antes de fork hasta waitpid. Incluye arranque del comando, MPI, validación, salida, liberación y cierre; excluye la escritura del CSV del cronómetro. El tiempo de cola o inicio del servicio no pertenece a este reloj.

Se verificó external_s >= application_s + validation_s. El resto se calcula por observación antes de resumirlo; no equivale a tiempo puro de arranque. Medianas y cuartiles inclusivos usan solo las siete rondas útiles. Q1-Q3 describen dispersión, no intervalos de confianza.

Los nombres de rondas están corregidos. El paquete no incluye el script de lanzamiento ni registros de posición temporal por ejecución: la rotación y la afinidad secuencial no pueden auditarse desde esos nombres. El entorno sí contiene un diagnóstico de binding P6, todos en el socket 0. Esta limitación no se confunde con un fallo numérico.

Ganancia hasta obtener L

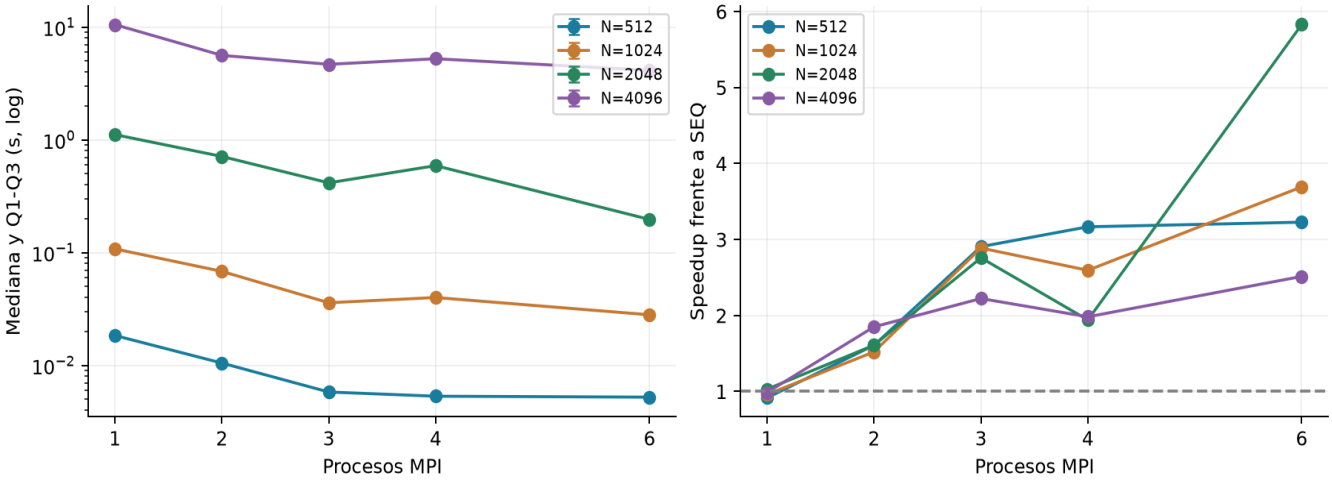


Figura 1. Tiempo de aplicación MPI y cociente mediana(SEQ)/mediana(MPI). Cada N usa su propio secuencial contemporáneo. P1 es la ruta MPI, no una copia del secuencial autónomo.

N	SEQ s	P mínimo	MPI s	S	E %
512	0.016851	6	0.005221	3.23	53.8
1024	0.103648	6	0.028074	3.69	61.5
2048	1.146293	6	0.196693	5.83	97.1
4096	10.391210	6	4.133791	2.51	41.9

Interpretación

La curva no es monótona: utilizar más procesos no garantiza terminar antes. El mínimo entre configuraciones es descriptivo y requiere contexto; la eficiencia S/P no es utilización de CPU. Un valor cercano o superior al ideal lineal tampoco identifica por sí solo una causa como caché o balance.

En resumen.csv quedan los 24 grupos con sus medianas, cuartiles y cocientes, para ambos relojes y las fases. La grilla alcanza seis procesos bajo cuota; no permite extrapolar a 24 o 96 procesos ni a varios nodos.

Pregunta siguiente

¿El mínimo del tiempo interno sigue siendo útil para quien espera que termine el comando? La validación secuencial y el costo de lanzar MPI pueden cambiar la respuesta, especialmente en los tamaños menores.

Cuánto sobrevive en el reloj externo

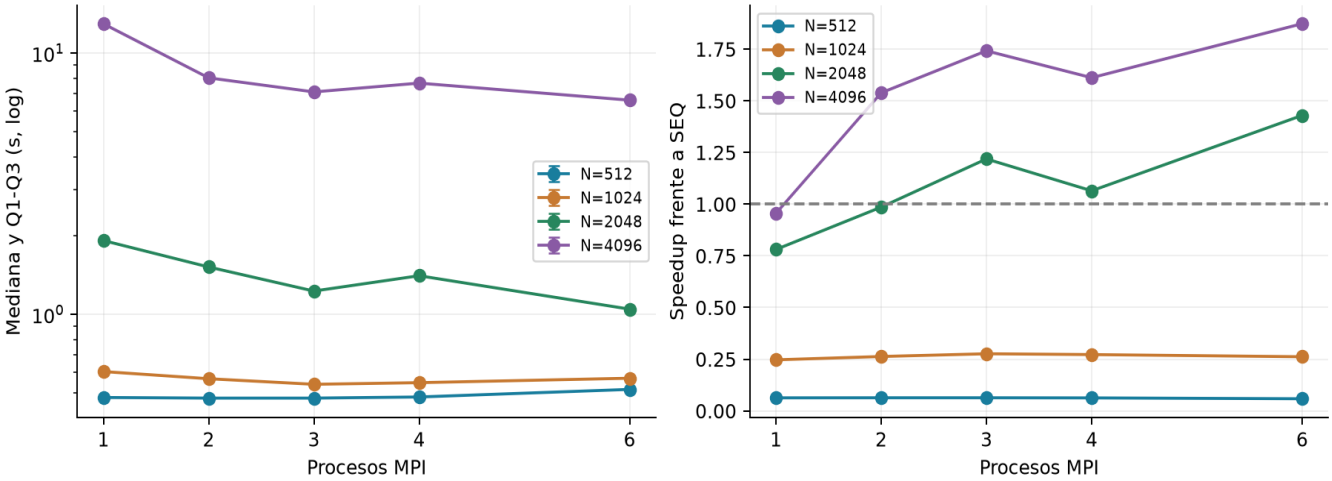


Figura 2. Tiempo del comando y speedup externo. Las barras representan Q1-Q3 del tiempo; no se transfieren las ganancias del reloj interno al externo.

N	SEQ ext s	P mínimo	MPI ext s	S externo
512	0.0302	3	0.4774	0.06
1024	0.1489	3	0.5392	0.28
2048	1.4950	6	1.0459	1.43
4096	12.3637	6	6.6011	1.87

Una comparación explícita con P=6

N	S aplicación	S externo	E externa %
512	3.23	0.06	1.0
1024	3.69	0.26	4.4
2048	5.83	1.43	23.8
4096	2.51	1.87	31.2

S externo mayor que uno indica que el comando terminó antes por mediana. Un speedup externo menor no contradice la aceleración del kernel: mide una tarea más amplia. El primer tamaño de esta grilla que mejora no es un umbral universal ni necesariamente el mismo que el cruce N=192 de application_s.

Costos del flujo y conclusión de escalabilidad

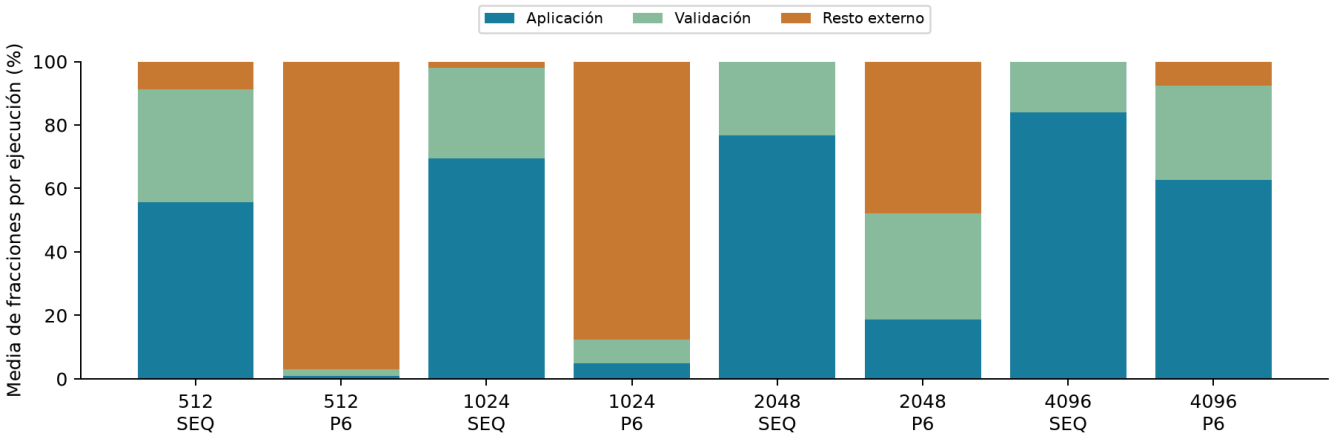


Figura 3. Media de las fracciones calculadas por lanzamiento; cada barra suma 100%. El resto externo combina inicialización, coordinación, cierre y otros costos. No es comunicación pura ni se obtiene restando medianas.

Cuota observada

cpu.stat cambia en 5514 períodos, 197 con throttling adicional y 0.764111 s del contador throttled_usec. Las lecturas rodean toda la campaña y procesos auxiliares: no asignan ese costo a una configuración ni autorizan a restarlo de sus tiempos.

Respuesta experimental

N=2048/P6: S interno=5.83; S externo=1.43. La comparación mide beneficio frente al programa secuencial propio en esta sesión. La validación permanece fuera del reloj interno y dentro del externo.

N=4096/P6: S interno=2.51; S externo=1.87. La comparación mide beneficio frente al programa secuencial propio en esta sesión. La validación permanece fuera del reloj interno y dentro del externo.

La campaña ya permite distinguir paralelismo del algoritmo y beneficio de extremo a extremo. No hace falta ampliar recursos para responder la pregunta planteada. Una repetición selectiva puede reforzar estabilidad, pero no es necesario investigar cada anomalía para cerrar un trabajo universitario.

Archivos: observaciones.csv, warmups.csv, resumen.csv, auditoria.json y el paquete intacto. Los hashes calculados al recibir el archivo permiten identificar esta copia; no reemplazan un manifiesto de mediciones que no se archivó. La siguiente etapa compara el secuencial propio con una biblioteca optimizada en un banco aparte.

Etapa 18: referencia LAPACK en el clúster

Pregunta: ¿cuánto margen de rendimiento local queda frente a una biblioteca numérica optimizada? Se comparan el kernel secuencial propio y LAPACKE_dpotrf_work, enlazado con OpenBLAS 0.3.34, dentro del mismo ejecutable y con la misma matriz. Es una comparación local de implementaciones; no es ScaLAPACK ni una prueba MPI de la biblioteca.

Diseño comprobado en el código archivado

Aspecto	Implementación
Tamaños principales	N=2048 y 4096; seis repeticiones por método y tamaño: 24 filas. Control N=512: otras 12 filas, separado.
Orden	Alternancia own/lapack y lapack/own; tres posiciones iniciales para cada método. Una ronda inicial por método sin exportar.
Hilos	openblas_set_num_threads(1) y comprobación por API. Los logs informan threads=1. OpenBLAS pthreads, no OpenMP añadido al candidato.
Entrada y salida	Matriz SPD con semilla 2026. Mismo generador; factor triangular inferior interpretado en row-major.
Validación	Ocho proyecciones, comparación con factor propio de referencia y pruebas pequeñas con residuo completo.

Qué mide cada campo

factor_s mide únicamente la llamada al kernel. total_s incluye reserva del arreglo y workspace, generación y factorización. Excluye preparar la matriz original/referencia del banco, validación y liberación. No existe aquí un reloj externo. Cada tamaño se ejecuta en un proceso con repeticiones internas, a diferencia de los lanzamientos nuevos de la campaña MPI.

LAPACK recibe la matriz simétrica como column-major y triángulo U: ese almacenamiento representa L en row-major. La variante _work evita la conversión row-major y el chequeo opcional de NaN de la interfaz. Se verifica un factor conocido en el test. La comparación conserva el mismo arreglo lógico sin cobrar una transposición adicional.

Resultados: kernel y flujo completo del banco

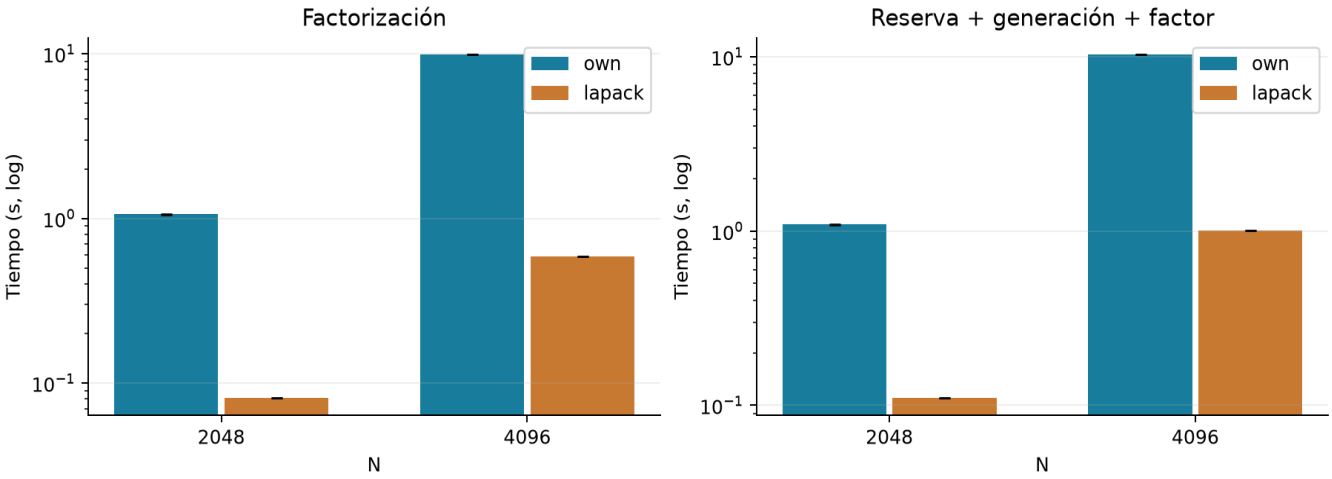


Figura 1. Medianas y Q1-Q3 inclusivos, seis muestras por método. Escala logarítmica. El total se resume directamente, sin sumar medianas de fases. La mejora es own/LAPACK dentro de este banco.

N	Reloj	Propio s	LAPACK s	Cociente
2048	Factor	1.060705	0.081292	13.05
2048	Total	1.089732	0.110383	9.87
4096	Factor	9.854616	0.589960	16.70
4096	Total	10.269754	1.006013	10.21

Qué responde la comparación

La biblioteca reduce ampliamente el tiempo incluso con un único hilo. La mejora del total es menor que la del kernel porque la generación permanece en el flujo. Esto demuestra margen de optimización local del candidato; no demuestra que agregar procesos pueda alcanzar ese margen ni aísla qué parte corresponde a cada optimización de la biblioteca.

Los residuos y diferencias de factor pasan el criterio 1e-12 en las 24 filas. La diferencia normalizada máxima entre factores es aproximadamente 6,595e-15: resultados numéricamente consistentes no requieren igualdad bit a bit. Los tests pequeños figuran PASS.

Auditoría y límites de la referencia

Lo que contiene el paquete

Doce archivos: dos CSV principales, dos salidas .err, código C, headers, ejecutable, logs de prueba/compilación y entorno. Se verificaron los cuatro hashes de fuentes/ejecutable registrados en entorno.txt. No hay un manifiesto original que cubra todos los CSV; se conserva el tar íntegro y su SHA-256 de recepción.

Los archivos .err contienen el mensaje informativo de OpenBLAS threads=1, no un error de cálculo. El log de compilación contiene una advertencia -Wpedantic sobre _Float16 en openblas_config.h. Debe conservarse como advertencia de un encabezado de la biblioteca; no equivale a fallo del test ni autoriza a decir que compiló sin warnings.

El código acumula el estado de fallo y lo devuelve al finalizar. Las filas no incluyen valid ni el código de salida y el paquete no registra ese estado del shell. Las comprobaciones numéricas exportadas son satisfactorias y el log de tests dice PASS, pero no se inventa un exit_code para las corridas principales.

Afinidad y condiciones

El límite de un hilo está comprobado por código y logs. El paquete LAPACK no conserva el comando de ejecución ni un registro de afinidad CPU: no se afirma que corriera fijado a CPU0. El banco alterna ambos métodos en el mismo proceso, lo que fortalece su comparación interna. El entorno registra EPYC 7B12 y bibliotecas conda-forge, fechado 17:25:42 UTC.

Comparación con la etapa MPI

No se construye un speedup formal LAPACK/MPI mezclando estas campañas. Tienen distintos ejecutables, tratamiento de calentamiento, validaciones y sesiones de medición. La lectura conjunta sí es válida: nuestro MPI acelera una base propia y la biblioteca muestra que esa base conserva un margen local grande. Acelerar una implementación no demuestra que sea competitiva con una biblioteca especializada.

El control N=512 se conserva separado en control_n512.csv. No se suma a la muestra principal ni se usa como repetición independiente. Los seis resultados por método provienen del mismo lanzamiento; no son seis sesiones del clúster.

Conclusión integrada y cierre universitario

Tres respuestas complementarias

La Etapa 16 localizó el cruce de application_s: primera mejora clara ensayada en N=192/P3. La Etapa 17 midió cómo cambian las ganancias con N y P y cuánto sobrevive al comando completo. Esta Etapa 18 muestra el margen del kernel propio frente a LAPACK/OpenBLAS con un hilo. Cada resultado conserva su denominador y su reloj.

N=2048: LAPACK mejora el kernel propio 13.05 veces y el flujo interno del banco 9.87 veces. Es evidencia de rendimiento local, sin MPI en la biblioteca.

N=4096: LAPACK mejora el kernel propio 16.70 veces y el flujo interno del banco 10.21 veces. Es evidencia de rendimiento local, sin MPI en la biblioteca.

Mi recomendación

Estas campañas cubren las preguntas centrales de un proyecto universitario: corrección, cruce, escalabilidad, costo completo y referencia optimizada. No abriría ahora una nueva implementación con OpenMP, bloques o distribución 2D ni una campaña hasta 96 cores. Priorizaría redactar la conclusión y preparar la defensa.

Si se desea una confirmación adicional, bastaría una sesión corta con SEQ y una o dos configuraciones MPI decisivas para N=2048/4096. Es una mejora de robustez, no una obligación de repetir todo. Guardar el script y la afinidad real solucionaría las principales carencias de trazabilidad detectadas.

Límite que debe quedar explícito

La referencia aportada es LAPACK, no ScaLAPACK. Si la consigna docente exige explícitamente una biblioteca distribuida, esa parte sigue pendiente y requiere un experimento distinto. Si pide una referencia numérica optimizada, esta campaña ya aporta evidencia pertinente.

Conclusión defendible: el diseño MPI propio puede reducir el tiempo respecto de su versión secuencial cuando hay suficiente trabajo, pero los costos externos reducen la ganancia percibida y más procesos no siempre convienen. La comparación con LAPACK muestra que la eficiencia del kernel local importa tanto como repartir el trabajo. Estas observaciones se limitan a las matrices, grillas y entorno medidos.

Reproducción de ambas etapas: desde_cero/docs/generar_informes_escalabilidad_lapack_cluster.py. Paquetes originales preservados; CSV derivados y auditorías junto a cada campaña. Análisis y documentación con asistencia de IA.

Etapa 19: MPI frente a LAPACK en matrices grandes

Pregunta final: al crecer hasta N=6144 y N=8192, ¿MPI con seis procesos se acerca o supera a LAPACK/OpenBLAS con un hilo? Se analiza el paquete original del 28/09/2026 y se contrasta de forma independiente el ZIP de análisis recibido.

Respuesta

MPI reduce el tiempo frente al C propio secuencial, pero no alcanza a LAPACK en ninguno de los dos tamaños. La razón MPI/LAPACK disminuye de 8,58 a 6,25: hay acercamiento relativo entre esos dos puntos, sin evidencia de un cruce futuro. La diferencia absoluta en segundos, en cambio, aumenta.

Diseño y trazabilidad

Aspecto	Evidencia
Métodos y tamaños	C propio, MPI P6 y LAPACK/OpenBLAS 1 hilo; N=6144 y 8192.
Repeticiones	Seis útiles por método y tamaño: 36 observaciones. Dos warmups MPI exportados; warmups own/LAPACK implementados dentro del banco, no exportados.
Integridad	44 archivos regulares; 43 hashes del manifiesto verificados, excluido el propio manifiesto.
Continuidad	Los cinco fuentes candidatos coinciden con las etapas anteriores; compare_lapack.c coincide con la Etapa 18.
Entorno	EPYC 7B12, Open MPI 4.1.6, OpenBLAS 0.3.34; cuota de 6 CPUs y 24 GiB. Inicio 19:38:51 UTC; fin 20:05:20 UTC.

Qué hace comparable factor_s

El reloj principal abarca la fase que factoriza la misma familia de matrices SPD generadas con semilla 2026. En C/LAPACK es la llamada al kernel; en MPI es el intervalo del rango 0 alrededor del kernel distribuido, que incluye comunicaciones y esperas. No es un máximo reducido por fase ni una comparación de aritmética pura.

Generación y reunión quedan fuera de factor_s; los relojes secundarios tienen límites distintos. El banco C/LAPACK alterna métodos dentro de un proceso; MPI se lanza por separado. Por eso el cociente principal es descriptivo del costo de esa fase en los dos programas, no una prueba que aísle únicamente el efecto del paralelismo.

Resultados de factorización y dispersión

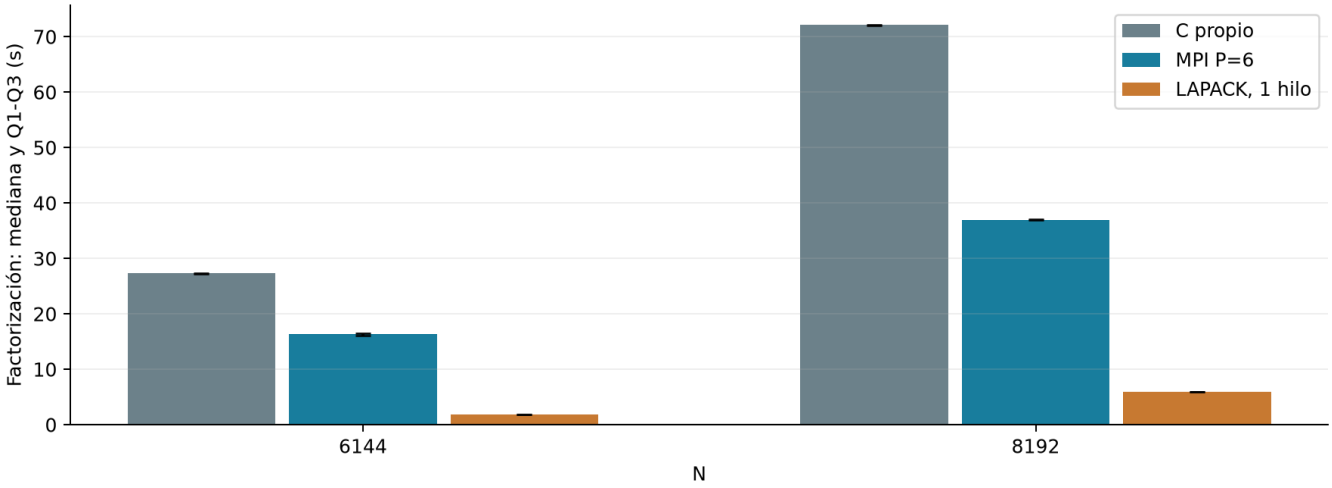


Figura 1. Medianas y Q1-Q3 inclusivos, seis observaciones útiles por grupo. Barras de cuartiles descriptivas; no intervalos de confianza. Los warmups no se incluyen.

N	Método	Q1 s	Mediana s	Q3 s	CV %
6144	own	27.2602	27.2690	27.2713	0.027
6144	mpi_p6	16.0909	16.2985	16.4927	1.424
6144	lapack	1.8987	1.8992	1.8996	0.076
8192	own	72.0020	72.0915	72.1250	0.131
8192	mpi_p6	36.9214	36.9539	36.9942	0.211
8192	lapack	5.9129	5.9159	5.9181	0.065

CV usa desviación muestral dividida por la media. La variación MPI en N=6144 es mayor que la de los otros grupos, pero la separación respecto de LAPACK es amplia: incluso el menor tiempo MPI supera al mayor LAPACK en ambos tamaños. Esta observación no necesita transformarse en un test de significación para describir la diferencia de esta sesión.

Corrección numérica

Las 12 mediciones MPI y los dos warmups registran valid=1. Las 24 filas own/LAPACK cumplen los controles exportados: residuo proyectado y diferencia de factor <=1e-12. Máximo residuo útil: 3.240e-15; máxima diferencia de factor: 8.866e-15. No se calcula residuo completo para estos tamaños.

El ZIP adjunto coincide con los seis grupos estadísticos y con las razones recalculadas desde los originales. No se utilizó su narrativa como sustituto de la verificación de datos.

Acercamiento relativo, no cruce

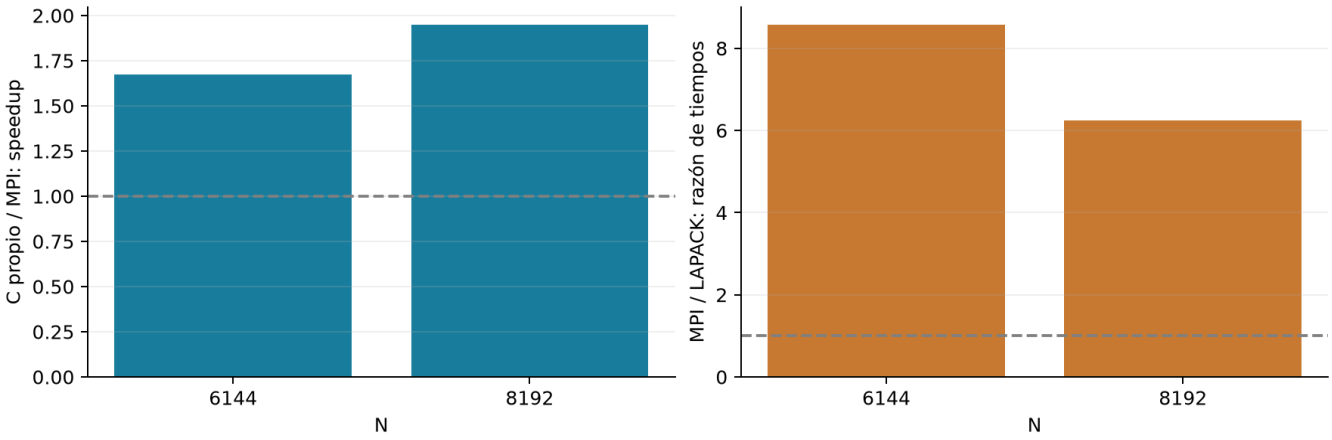


Figura 2. Izquierda: valores mayores que uno indican mejora MPI frente al C propio. Derecha: valores mayores que uno indican que MPI tarda más que LAPACK. Los denominadores responden preguntas distintas.

N	C / MPI	E MPI %	C / LAPACK	MPI / LAPACK
6144	1.673	27.9	14.359	8.582
8192	1.951	32.5	12.186	6.247

La eficiencia respecto del C propio es S/6: 27,9% y 32,5%. No equivale a utilización de CPU. LAPACK sigue por delante usando un hilo; no se ejecutó ScaLAPACK. La cuota de seis CPUs equivalentes no implica seis cores exclusivos.

La diferencia absoluta MPI menos LAPACK pasa de 14.399 s a 31.038 s. Por tanto, 'se acerca' solo es correcto en la razón relativa entre estos tamaños. Dos puntos no permiten extrapolar un tamaño donde MPI alcance a la biblioteca ni establecer una tendencia monótona.

Interpretación y límite causal

Repartir el trabajo del kernel propio obtiene una mejora, pero no compensa la ventaja local de LAPACK/OpenBLAS en esta medición. La campaña no separa las contribuciones de localidad, vectorización, algoritmos internos de la biblioteca o comunicación. No hace falta identificar cada causa para responder la comparación experimental.

Relojes secundarios y condiciones del entorno

N	Método	Generación s	Reloj secundario	Mediana s
6144	own	0.4023	total_s	27.6713
6144	mpi_p6	0.1322	application_s	16.6069
6144	lapack	0.4023	total_s	2.3017
8192	own	3.9806	total_s	76.0697
8192	mpi_p6	0.0614	application_s	37.3368
8192	lapack	3.9786	total_s	9.8945

Estas medianas se presentan como contexto: no se construyen cocientes entre application_s y total_s. MPI application_s incluye reserva, generación, factorización y reunión/reconstrucción; total_s del banco own/LAPACK incluye reserva, generación y factor. Ninguno es el tiempo externo del comando. No se suman medianas para reconstruir totales.

Salto en la generación

La generación secuencial pasa de aproximadamente 0,402 s a 3,98 s al cambiar N=6144 por N=8192. El salto es observado, pero su causa no se aisló. La hipótesis del stride de 65536 bytes por fila puede motivar trabajo futuro; no se presenta como explicación demostrada ni exige una nueva campaña para cerrar esta pregunta.

Su duración no se suma a factor_s. Sin embargo, la generación previa puede afectar caché y colocación de memoria al entrar al kernel: separar cronómetros no elimina toda influencia indirecta. Esa salvedad delimita la comparación, sin borrar la amplia diferencia observada.

Cuota y procedencia

Entre lecturas de cpu.stat hubo 1676 períodos adicionales con throttling y 4.376492 s adicionales en throttled_usec. Son contadores de toda la sesión durante el intervalo, no tiempos perdidos atribuibles a una repetición. No se corrigen los resultados restando ese valor.

El paquete conserva fuentes, CSV y entorno, pero no script, binarios, códigos de salida del shell ni binding por ejecución. Los hashes de binarios mencionados en entorno.txt no pueden cotejarse sin esos binarios. Los .err MPI están vacíos y los de own/LAPACK informan threads=1. El estado numérico MPI es explícito; en own/LAPACK se verifican los residuos y diferencias exportados.

Conclusión final de la comparación

Respuesta a la pregunta planteada

En N=6144 y N=8192, con la sesión limitada a seis CPUs equivalentes, MPI P6 acelera el kernel propio 1,67 y 1,95 veces, respectivamente. Aun así, sus medianas de factorización son 8,58 y 6,25 veces las de LAPACK/OpenBLAS a un hilo. No se observó que MPI alcanzara ni superara a LAPACK en ninguno de los dos tamaños ensayados.

Cómo completa el proyecto

La investigación muestra cuatro resultados complementarios: hay un tamaño mínimo ensayado donde el MPI propio empieza a compensar; la escalabilidad depende de N y P; el comando completo conserva menos ganancia que el reloj interno; y una biblioteca optimizada puede superar al kernel propio aun cuando este usa varios procesos. No se mezclan clocks ni referencias para obtener esas conclusiones.

La ganancia MPI es real respecto de la base propia medida, pero no demuestra superioridad frente a implementaciones especializadas. Esta distinción es un resultado valioso del proyecto, no una razón para seguir ampliando matrices hasta encontrar un resultado favorable.

Mi recomendación de cierre

Con esta campaña, la pregunta final queda respondida para el alcance acordado. Recomiendo cerrar la experimentación y consolidar la conclusión general y la defensa. No es necesario probar 96 cores, implementar OpenMP o investigar en detalle el salto del generador. Se pueden mencionar como extensiones, sin convertirlas en requisitos del trabajo universitario.

El límite principal sigue siendo una familia de matrices SPD y una sesión bajo cuota compartida. La campaña no prueba comportamiento multinodo, ni ScaLAPACK, ni un cruce futuro. La diferencia relativa observada es amplia, aunque los bancos no comparten exactamente el mismo régimen de lanzamiento y calentamiento.

Reproducibilidad

Se conservan los dos paquetes recibidos. observaciones.csv contiene las 36 mediciones útiles; warmups_mpi.csv conserva las dos filas descartadas. resumen_factor.csv, razones.csv, contexto.csv y auditoria_independiente.json se regeneran con desde_cero/docs/generar_informe_final_grande_cluster.py. El proceso no ejecuta código ni binarios recibidos. Estadísticas independientes coincidentes con el análisis adjunto. Documentación asistida por IA.